arXiv:2609.18126v1 [cs.AI] 16 Sep 2026

Designing Agentic AI Workflow Portfolios under Imperfect Selection and Compute Cost

Mojtaba Abdolmaleki
Ross School of Business, University of Michigan, United States, mojtabaa@umich.edu

Stefanus Jasin
Ross School of Business, University of Michigan, United States, sjasin@umich.edu

Boyu Wang
TrueFoundry, boyu.wang@truefoundry.com
School of Management, University of San Francisco, United States, bwang62@usfca.edu

Agentic AI systems often approach the same task through multiple workflows that differ in reasoning strategy, verification structure, and compute cost. A natural deployment policy is to use the workflow with the highest average performance, but this can be suboptimal because different workflows may succeed on different instances. We study a portfolio-and-selector paradigm in which a firm runs multiple workflow executions and selects the final answer after observing their outputs. Additional executions may uncover correct answers that the best standalone workflow misses, but they consume compute and introduce plausible distractors that complicate final selection. We formulate this as a workflow portfolio problem in which the firm jointly chooses run size and allocation across workflow types. We summarize selector quality through an odds-lift index and derive sharp bounds on the value of workflow variety. For finite workflow pools, we develop exact formulations, linear programming relaxations, randomized rounding procedures, and computable performance certificates. For large implicit workflow classes, we derive a finite-dimensional dual and an ellipsoid method using a pricing oracle to identify workflows with high weighted accuracy net of recurring compute cost. Under a weak condition, the method obtains a near-optimal solution to the relaxation with polynomially many oracle calls. We evaluate the framework on three datasets: ABCD, Schema-Guided Dialogue, and HotpotQA. Relative to the best standalone workflow, portfolio optimization improves held-out selector accuracy by 3.1, 7.5, and 0.9 percentage points, respectively. Dual-guided workflow generation adds 3.5 points on ABCD and 24.1 on HotpotQA, with no additional gain on Schema-Guided Dialogue.

Key words: agentic AI; workflow portfolios; large language models; endogenous portfolio size; compute allocation; selector design; selector strength; odds lift; random utility; ellipsoid method; separation oracle; repeated execution

1. Introduction

Artificial intelligence is moving beyond individual productivity tools and into the operating core of organizations. Enterprise software vendors are embedding task-specific AI capabilities into business applications, while firms are beginning to delegate not only information retrieval and drafting, but also parts of operational decision processes to AI systems (Gartner 2025a,b). Recent business surveys report that many organizations are experimenting with or deploying agentic AI, while

also emphasizing that realizing value requires redesigning workflows, establishing governance, and building reliable orchestration layers rather than simply giving employees access to models (McKinsey & Company 2025, 2026, Ransbotham et al. 2025, Harvard Business Review Analytic Services 2026). This shift raises a natural operations-management question: once AI workflows become part of recurring business processes, how should firms decide which workflows to deploy and how much compute to devote to them?

The setting we study arises in service operations, compliance, claims processing, internal knowledge work, and decision support. A customer-service system may classify a ticket, retrieve relevant policies, identify the customer's need, draft a response, check the response for compliance, and decide whether to escalate the case to a human. A claims-processing system may summarize a claim, compare it with contract language, identify relevant exceptions, and recommend whether to approve or deny it. A compliance assistant may retrieve applicable rules, analyze a transaction, flag inconsistencies, and recommend an appropriate action. In each of these settings, the relevant operational unit is not a single model call, but a workflow: a sequence or graph of prompts, model calls, retrieval steps, tools, verifiers, and aggregation rules that produces a candidate answer.

At first glance, a natural approach is to identify the workflow with the highest average accuracy and deploy it alone. Average performance, however, can conceal substantial heterogeneity across tasks: different workflows may succeed on very different cases. Relying on a single workflow can therefore leave important gaps that another workflow could fill. A firm may have many workflows that it could develop and maintain. Some are cheap and fast, while others are more expensive but more careful. Some rely on retrieval, some on decomposition or repeated reasoning, and others on verification or critique. A workflow that performs poorly on average may still be valuable if it solves cases that stronger workflows miss, whereas a high-performing workflow may add little if it succeeds mainly on cases that are already easy. The firm must therefore decide which workflow types to maintain, how many times to execute each type, and which candidate answer to select as the final output, while accounting for the recurring compute cost of these decisions.

Much of the existing literature follows a plan, then execute paradigm (Shen et al. 2023, Liang et al. 2024, Lu et al. 2023, Zhang et al. 2025b, Hu et al. 2025, Zhang et al. 2025a). A large model, router, or planner first selects or designs a single workflow, which is then executed to produce the final answer. This plan–then–execute paradigm is illustrated in Figure 1. This approach is natural when the system can reliably predict, before execution, which workflow is best suited to the task. In many operational settings, however, the firm may not know in advance which workflow will succeed. It may be easier to recognize a correct answer after observing the candidate outputs and their supporting evidence, such as execution traces, retrieved sources, agreement patterns, confidence signals, or verifier results, than to identify ex ante the single workflow that will generate it.

In this paper, we propose a portfolio and selector paradigm. The firm first builds and maintains a set of workflow types. For each task class or deployment setting, it decides how many execution slots to use and how to allocate those slots across the available workflows. The same workflow may receive more than one slot. The selected executions are then run in parallel, and a selector chooses

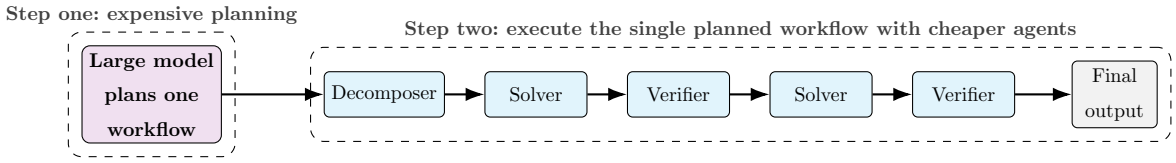


Figure 1 **Traditional orchestration paradigm: a large model synthesizes one workflow, which is then executed by cheaper specialized agents.**

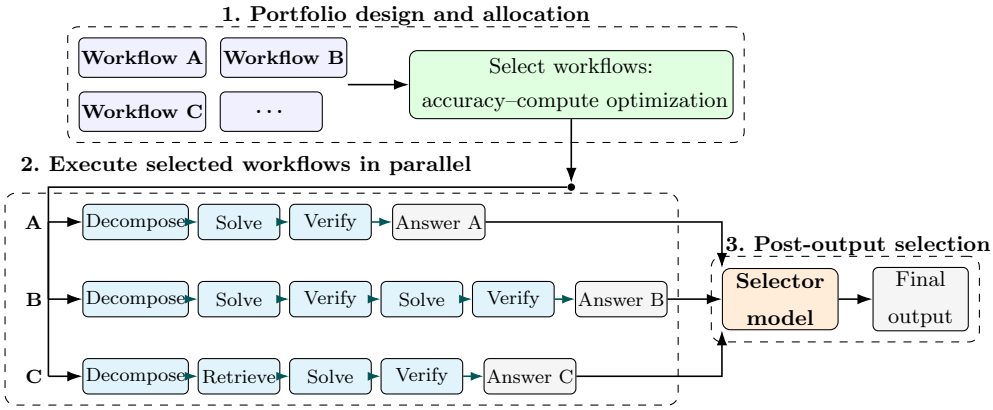


Figure 2 **Portfolio-based orchestration: the decision maker selects workflow types and allocates execution slots, runs the selected workflows in parallel, and chooses the final answer after observing their realized outputs.**

the final answer from the resulting candidates. This operating model creates a fundamental tradeoff. An additional execution may produce a correct answer that would otherwise be unavailable, or it may increase the representation of a workflow that performs well on many tasks. At the same time, every execution consumes tokens, latency, tool calls, and money. If its output is wrong, it also adds another plausible distractor for the selector. More executions are therefore not necessarily better.

The central question of this paper is: *How should a firm decide how many workflow executions to run and how to allocate them across workflow types, when additional executions can improve the quality of the candidate set but also increase selector confusion and compute cost?*

Related literature. Our work brings together several related streams, which we discuss in detail in Section 2. One stream studies how to design effective agentic workflows and inference time architectures (Zhang et al. 2025b, Hu et al. 2025, Zhuge et al. 2024, Saad-Falcon et al. 2025, Zhang et al. 2025a). A second stream studies how to route queries across models or workflows to balance quality and cost (Chen et al. 2024, Ong et al. 2025, Hu et al. 2024, Huang et al. 2025). A third stream generates multiple candidate answers and uses ranking, fusion, self consistency, or multi-agent aggregation to choose among them (Wang et al. 2023, Jiang et al. 2023, Wang et al. 2025). Together, these studies address important parts of the deployment problem, namely workflow design, pre-execution allocation, and post-output aggregation. Once these decisions are considered jointly, however, the problem becomes one of choosing and allocating a portfolio of alternatives under limited resources and imperfect final selection. This perspective relates to several established areas in operations research, including choice modeling, assortment optimization, coverage, submodular optimization, column generation, and optimization through separation oracles

(Luce 1959, McFadden 1974, van Ryzin and Mahajan 1999, Talluri and van Ryzin 2004, Bertsimas and Mišić 2019, Dong et al. 2023, Wang et al. 2024, Nemhauser et al. 1978, Barman et al. 2021, Alaei et al. 2010, Dantzig and Wolfe 1960, Desaulniers et al. 2005, Grötschel et al. 1981, 1988). What remains missing is an integrated framework for designing workflow portfolios when both execution decisions and final answer selection affect performance and cost.

The challenge is not limited to choosing among workflows that have already been evaluated. In practice, the feasible workflow space is large and cannot usually be enumerated in advance. We therefore need a way to model how the firm accesses workflows outside its current evaluated pool. In this paper, we assume that new workflows can be proposed by an oracle, which may take the form of a teacher model, an automated workflow generator, a structured search procedure, or a human engineering team. The firm guides the oracle toward generating workflows that perform well on tasks that the current portfolio handles poorly, while also accounting for the recurring cost of executing those workflows. This raises two questions: how should the firm use the oracle to search the implicit workflow space, and how can it certify the quality of the resulting portfolio?

We address these questions through a selector aware workflow portfolio model. The firm decides how many times to execute each workflow type, which jointly determines the composition and total size of the candidate set. Each execution produces a candidate answer, and a selector chooses the final output after observing the resulting candidates. This portfolio-and-selector paradigm is illustrated in Figure 2. We summarize selector performance through recovery curves that relate the number of correct candidates in the set to the probability that the final selected answer is correct. The firm's objective is to maximize deployed decision quality net of recurring compute cost.

Our contributions. Our first contribution develops a general theory of selector strength. We introduce an odds lift index that measures how much the selector increases the odds of returning a correct answer relative to random selection. A finite bound on this index places the selector's recovery curve below a common concave envelope. This envelope gives a sharp limit on how much any workflow portfolio can improve on the best single workflow, regardless of how many workflows are available or how complementary they appear. It also yields simple screening rules that identify when selector quality and execution cost make a multi-workflow portfolio unattractive, so that the best decision is to use a single workflow or not deploy the system.

Our second contribution develops an optimization framework for finite workflow pools when the total number of workflow executions, which we call the *run size*, is itself a decision. The framework determines both the run size and the allocation of execution slots across workflow types, recognizing that an additional execution may improve accuracy on some tasks while increasing selector confusion on others. For any fixed run size, the problem becomes a concave coverage problem. This structure leads to exact integer programming formulations, linear programming relaxations, randomized rounding procedures, and computable optimality certificates. We then develop a sparse grid method for choosing the run size. Rather than solving a separate optimization problem for every possible size, the method evaluates only a small collection of candidate values while retaining a provable approximation guarantee. This provides a practical approach for jointly choosing how many workflow executions to run and how to allocate them across workflow types.

Our third contribution develops an optimization method over the *implicit workflow class*, by which we mean the full set of feasible workflows that the oracle can search over but the firm cannot enumerate in advance. Because the same workflow type may occupy multiple execution slots, the linear program for a fixed run size does not require an upper bound on the number of times each type can be used. Although this linear program contains one variable for every workflow type in the implicit class, its dual has only finitely many task price variables, together with one constraint for each workflow. A global pricing oracle can therefore identify a workflow whose constraint is violated, or certify that no important violation remains. Using the classical equivalence between optimization and separation, the ellipsoid method computes a solution within ε of the full linear programming relaxation using a number of oracle calls that is polynomial in the problem encoding and $\log(1/\varepsilon)$. Combining this optimization error with the losses from the cardinality grid and randomized rounding yields a high probability approximation guarantee relative to the entire implicit workflow class, without requiring that class to be enumerated.

Our fourth contribution is empirical. We first calibrate selector recovery across the ABCD and Schema-Guided Dialogue service-operations tasks (Chen et al. 2021a, Rastogi et al. 2020) and the HotpotQA question-answering benchmark (Yang et al. 2018). Selector strength is substantially above random selection in the two service domains and remains positive, although weaker, on HotpotQA. We then evaluate the complete stochastic workflow- generation and portfolio-optimization pipeline in all three domains. On fresh held-out tasks, actual selector accuracy rises from 43.625% for the best initial singleton to 50.250% for the final ABCD portfolio, from 85.250% to 92.750% on Schema-Guided Dialogue, and from 30.250% to 55.250% on HotpotQA. Dual-guided generation incorporates four workflows on ABCD, none on Schema-Guided Dialogue, and one on HotpotQA. This variation is consistent with the model's central implication: a generated workflow creates value only when its incremental coverage and selection benefit justify its recurring compute cost.

The central message is that deploying agentic AI is neither a search for the single best workflow nor a simple rule that more candidates are always better. Workflow variety creates value only when the selector can identify useful outputs reliably enough to justify the additional compute cost and selection difficulty. Effective deployment therefore requires firms to manage workflow generation, run size, the allocation of executions across workflow types, selector strength, and compute expenditure as a joint decision.

Organization of the paper. The rest of the paper is organized as follows. Section 2 reviews the related literature. Section 3 formulates the portfolio and selector problem and introduces the oracle interface for searching the implicit workflow class. Section 4 studies how selector strength limits the value of workflow variety. Section 5 develops the finite pool optimization framework, including exact integer programs, linear programming relaxations, randomized rounding, and optimality certificates. Section 6 derives the dual formulation for the implicit workflow class and uses a workflow pricing oracle together with the ellipsoid method to obtain performance guarantees without enumerating all feasible workflows. Section 7 extends the model and its algorithmic guarantees to stochastic workflow execution. Section 8 presents the numerical experiments, and Section 9 concludes the paper. Unless otherwise noted, all proofs are provided in the appendix.

2. Related Literature

Our work relates to several streams of literature. The first studies the automated design of agentic workflows, compound AI systems, and LLM generated algorithms or heuristics. The second examines model routing, cascades, and methods that generate and aggregate multiple candidate outputs. The third includes work on random utility, assortment choice, and selection among competing alternatives. The fourth concerns coverage, submodular optimization, and optimization over large implicitly represented decision spaces. Finally, our paper contributes to the emerging operations management literature on the deployment, allocation, and governance of AI systems. We discuss these streams in turn and explain how they inform, but do not resolve, the joint problem of workflow generation, execution allocation, and final answer selection studied here.

Automated design of agentic workflows and compound AI systems. A growing computer science literature studies how to design and optimize multistep language model systems. AFlow searches over workflows represented as code, using execution feedback and tree search (Zhang et al. 2025b). Automated Design of Agentic Systems treats agent design as a search problem in which a meta agent proposes new agentic systems (Hu et al. 2025). GPTSwarm represents interacting language agents as a graph whose structure can be optimized (Zhuge et al. 2024). Archon searches over inference architectures that combine repeated sampling, ranking, critique, verification, fusion, and model choice (Saad-Falcon et al. 2025). MaAS constructs an agentic supernet and selects task dependent multiagent architectures to balance performance and resource use (Zhang et al. 2025a). LLMSelector studies model assignment within a compound AI system, asking which language model should be used in each module of a fixed multicall architecture (Chen et al. 2025). Collectively, these studies show that workflow structure, module choice, and automated search can have substantial effects on system performance. Our work treats all these methods as potential mechanisms for generating candidate workflows, but studies a different operational decision. The primary objective in automated workflow design is typically to discover a strong workflow, architecture, or assignment of models to modules. We instead ask which workflow types should be retained, how many execution slots should be allocated to each, and how much compute should be spent when the final answer is chosen only after the candidate outputs are observed. Under this perspective, a workflow that is not the strongest on average may still be valuable if it solves cases missed by the existing portfolio and the selector can recognize its contribution. The same workflow may be harmful if it mainly produces costly distractors that make final selection more difficult.

LLM based algorithm design and heuristic portfolio generation. A related literature studies the use of large language models to design algorithms and heuristics. Recent surveys organize this work around the roles of language models as optimizers, designers, predictors, and components of algorithmic search (Liu et al. 2026b). AlphaEvolve combines language model generated code modifications with evaluator feedback and evolutionary search to improve algorithms (Novikov et al. 2025). EoH-S moves beyond the search for a single heuristic and instead constructs a small set of complementary heuristics (Liu et al. 2026a). The recurring process of generating candidates, evaluating their performance, and reoptimizing the system also has methodological parallels in self adjusting control and joint learning and optimization (Jasin 2014, Chen et al. 2019, 2021b,

Zhang and Jasin 2022, Agrawal et al. 2014, Keskin and Zeevi 2014, Elmachtoub and Grigas 2022, Faradonbeh and Faradonbeh 2023). The above literature is closely related to our workflow generation layer. In our setting, however, the generated objects are executable AI workflows drawn from an implicit workflow class, and their value depends on how they are deployed together. The objective therefore accounts not only for the performance of each generated workflow, but also for its contribution to a portfolio under imperfect final selection and its recurring execution cost.

LLM routing, model cascades, and efficient inference. Another stream studies how to allocate queries across models with different cost and quality profiles. FrugalGPT develops adaptive strategies, including model cascades, to reduce inference cost while preserving or improving performance (Chen et al. 2024). RouteLLM learns routers that choose between stronger and weaker models using preference data (Ong et al. 2025), while RouterBench provides a benchmark for evaluating systems that route queries across multiple language models (Hu et al. 2024). ThriftLLM studies the cost conscious selection of LLM ensembles for classification tasks (Huang et al. 2025). Recent operations research also treats heterogeneous LLM use as a resource allocation problem. Dean et al. (2026) optimize parallel query counts under reliability constraints, Li et al. (2026) study sequential model choice and stopping with query and waiting costs, and Guo et al. (2026) analyze static cascades under congestion and latency. Together, these papers determine how model calls should be allocated, sequenced, or stopped. Our paper studies a complementary problem. Rather than deciding which model to query next, we choose how many workflow executions to run, how to allocate those executions across workflow types, and which answer to select from the resulting candidates. Routing and cascade methods govern the process of acquiring model outputs, while our framework governs the design and evaluation of the candidate portfolio presented to the selector.

Output ensembling, ranking, self consistency, and inference time aggregation. A related literature improves language model performance by generating multiple candidate outputs and then selecting, combining, or refining them. Self consistency samples several reasoning paths and returns the answer with the greatest agreement across paths (Wang et al. 2023). LLM Blender ranks outputs from multiple language models and combines the highest quality candidates through generative fusion (Jiang et al. 2023). Mixture of Agents uses a layered architecture in which agents at later stages build on outputs produced by agents in earlier stages (Wang et al. 2025). Archon treats ranking, fusion, critique, verification, and related inference techniques as components of an architecture that can be selected through search (Saad-Falcon et al. 2025). Our paper abstracts from the details of the selector itself. The selector may use ranking, fusion, consistency, critique, verification, or some other mechanism. We instead study the upstream portfolio decision: which workflows should generate candidates, how many executions should be run, and when an additional candidate is worth its compute cost and its effect on final selection.

Choice modeling, coverage, and implicit optimization. Our selector model draws on random utility and assortment choice. Multinomial logit and Plackett–Luce provide tractable models of selection and ranking (Luce 1959, McFadden 1974, Plackett 1975, van Ryzin and Mahajan 1999, Talluri and van Ryzin 2004). Related work estimates preference parameters from observed choices and response times (Echenique et al. 2025). In our setting, the alternatives are workflow outputs rather

than products, and their value depends on whether they are correct. A wrong output enlarges the selector's choice set without adding correct attraction. We use Plackett–Luce as an estimable specialization, while our main variety bound relies only on a general odds lift envelope. This choice structure leads directly to the optimization problem studied in the paper. Once the run size is fixed, portfolio composition affects selector performance only through the number of correct outputs on each task, yielding a concave coverage problem and allowing us to use established linear programming and rounding tools (Nemhauser et al. 1978, Barman et al. 2021, Ageev and Sviridenko 2004, Chekuri et al. 2010). When run size is endogenous, however, adding an execution changes both the correct count and the total number of candidates, so the global objective is neither monotone nor generally submodular. We handle this additional decision through a sparse cardinality grid. Finally, when the feasible workflow class cannot be enumerated, a global workflow pricing oracle separates the workflow indexed dual constraints, allowing the ellipsoid method to optimize over the implicit class (Grötschel et al. 1981, 1988).

Operations management and AI deployment. An emerging operations management literature studies how AI systems should be adopted, allocated, priced, and governed. Recent work examines query allocation across heterogeneous language models (Dean et al. 2026, Li et al. 2026, Guo et al. 2026), the timing of AI adoption and access (Abdolmaleki and Duenyas 2026), pricing and delay in agentic AI services (Abdolmaleki et al. 2026), and the provenance of language model generated content (Radvand et al. 2026). Baek et al. (2026) study complementarity among humans, large language models, and operations research algorithms in inventory control. We contribute to this literature by studying workflow portfolio design. Our focus is how a firm should choose the number and allocation of workflow executions, generate new workflow types, and select among their outputs when performance depends jointly on workflow complementarity, selector strength, and recurring compute cost.

3. Model

This section develops the operational model for selector-aware workflow portfolio design. We first define the task environment, workflow executions, selector recovery, and the firm's objective of choosing both the composition and size of the deployed portfolio. We then distinguish optimization over a finite evaluated workflow pool from search over the larger implicit workflow class and formalize the cost-aware pricing interface used to generate new workflows.

3.1. Deployment setting

We study an operational setting in which a firm handles a stream of similar tasks. Although the tasks share a common objective, individual instances may differ substantially in the information, reasoning, and verification required to solve them. A workflow designed for one source of difficulty may therefore perform poorly on cases that require a different approach. This heterogeneity creates a role for workflow portfolios: by running several complementary workflows and selecting among their outputs, the firm may obtain a correct answer on cases that no single workflow handles reliably. As a running example, consider customer-service routing. A customer initiates a service conversation, and the system observes the customer's initial messages together with rel-

evant account information, order status, and policy context. Determining the appropriate route may require the system to infer the customer's objective, identify the current service state, retrieve the applicable policy, and verify that the proposed action is feasible. The final decision may be, for example, to initiate an order cancellation, address a refund-status inquiry, modify a subscription, begin troubleshooting, or escalate the case to a human agent. Different workflows may emphasize different parts of this reasoning process. Running multiple workflows can increase the likelihood that at least one produces the correct route, but it also incurs additional compute cost and creates more candidate outputs for the selector to distinguish. The model below formalizes this tradeoff. (Remark 1 discusses the extension to settings with multiple task types.)

Tasks. Let $X \in \mathcal{X}$ denote a task instance drawn from a population $\mathcal{D}$, and let $Y^\star \in \mathcal{Y}$ denote the correct answer, target decision, or benchmark solution. The output space $\mathcal{Y}$ may be finite, numerical, structured, or textual. In the baseline model, a returned answer y receives payoff $\mathbf{1}\{y = Y^\star\}$. In the customer-service routing example, X contains the observed conversation, account information, order status, and relevant context, while the target $Y^\star$ is the correct routing label, such as "cancel order," "refund status," "subscription change," or "escalate to human."

Workflows. A workflow $g \in \mathcal{G}$ is an executable AI procedure that maps a task instance to a candidate answer and an observable trace. Workflows can take many forms: a single prompt, a chain of specialized agents, a retrieval-augmented routine, a verify-and-revise loop, or a graph of agents and tools. The feasible space $\mathcal{G}$ contains all workflows consistent with the firm's operational constraints, such as allowed models, tools, retrieval sources, context length, latency, and termination rules. The main analysis does not require a particular graph representation of workflows; it only uses each workflow's evaluated correctness vector and running cost. Appendix A gives one possible graph-based representation of $\mathcal{G}$ for readers who want a concrete finite workflow class.

When workflow g is run on task X, it produces

$$O_g(X) = \big(A_g(X), T_g(X)\big), \tag{1}$$

where $A_g(X) \in \mathcal{Y}$ is the candidate answer and $T_g(X)$ is the observable execution trace, potentially including intermediate messages, retrieved documents, tool outputs, confidence scores, verifier signals, token counts, and latency. We suppress the dependence on X when it is clear from context. Because $\mathcal{G}$ is typically too large to enumerate, the firm must search over an "implicit" workflow space rather than selecting from a fixed list; we return to this search layer in Sections 3.2 and 3.3. In the routing example, a workflow might first retrieve relevant policy text, infer the customer's intent, and propose a routing label. A verifier then checks the proposal against the conversation and retrieved policy. If the proposal fails verification, a revision agent updates it using the verifier's feedback, and the verify–revise loop continues until the proposal passes verification or a prespecified iteration limit is reached.

Workflow accuracy. Each workflow has a recurring per-task running cost $c_g \geq 0$. This cost may represent dollars, tokens, latency-adjusted compute, tool-call charges, or an additive resource index. The cost is recurring because it is paid every time the workflow is run on an incoming task. Let $(x_i, y_i^\star)_{i=1}^n$ be a labeled development sample, i.e., the training or validation tasks used to evaluate

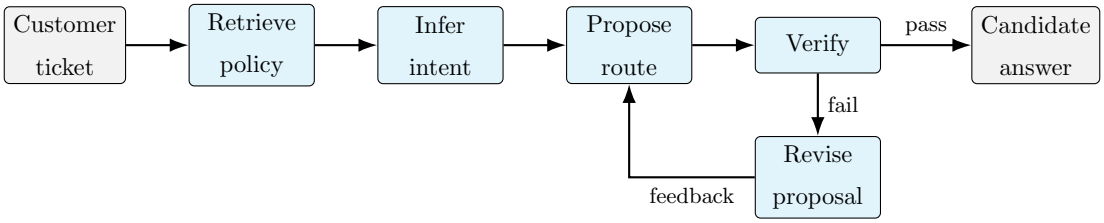


Figure 3 A customer-service routing workflow with a bounded verify–revise loop.

candidate workflows before deployment. The portfolio optimization problem below is defined on this training sample: workflow correctness, and portfolio values are computed from $(x_i, y_i^\star)_{i=1}^n$. For workflow g, define its task-level correctness indicator by

$$a_{ig} = \mathbf{1}\{A_g(x_i) = y_i^\star\}, \qquad i = 1, \dots, n, \quad g \in \mathcal{G}. \tag{2}$$

Thus, $a_{ig} = 1$ if workflow g returns the correct routing label for task i, and $a_{ig} = 0$ otherwise. Throughout the main analysis, we treat a_{ig} as deterministic, so each workflow–task pair has a fixed evaluated correctness outcome on the development sample. Section 7 extends the model to stochastic workflow execution, where $a_{ig} \in [0, 1]$ denotes the probability that one execution of workflow g is correct on task i.

Portfolio and execution cost. We use $g \in \mathcal{G}$ to index workflow types. For each incoming task, the firm chooses an execution-count vector

$$\boldsymbol{m} = (m_g)_{g \in \mathcal{G}} \in \mathbb{Z}_+^{\mathcal{G}}$$

with finite support, where m_g is the number of execution slots assigned to workflow type g. It may appear natural to impose $m_g \le 1$ for every workflow type. Under an imperfect selector, however, repeated execution of a strong workflow may raise the share of correct candidate outputs on the tasks it already solves and thereby improve selector recovery; see Appendix C. Under stochastic execution, repeated runs can also yield different outcomes, making multiplicity even more natural; Section 7 extends most of our results to that setting.

The total number of workflow executions is

$$k(\boldsymbol{m}) = \sum_{g \in \mathcal{G}} m_g, \tag{3}$$

and must satisfy $k(\boldsymbol{m}) \le K_{\max}$, where $K_{\max}$ reflects operational limits such as latency, context-window capacity, compute budget, or risk policy.

Each execution occupies a separate slot, incurs cost c_g, and produces a separate candidate for the selector. In the deterministic development-sample model, all executions of workflow type g share the evaluated correctness vector $(a_{1g}, \dots, a_{ng})$. The zero vector $\mathbf{0}$ represents the outside option of not deploying the AI system and has value zero. For $g \in \mathcal{G}$, let $\boldsymbol{e}_g$ denote the execution-count vector containing one execution of workflow g and zero executions of every other workflow.

For a feasible execution-count vector $\boldsymbol{m}$, let

$$r_i(\boldsymbol{m}) = \sum_{g \in \mathcal{G}} a_{ig} m_g \tag{4}$$

denote the number of correct candidate outputs produced on task i, and let

$$C(\boldsymbol{m}) = \sum_{g \in \mathcal{G}} c_g m_g \tag{5}$$

denote the total recurring execution cost.

Selector. Consider a feasible nonzero execution-count vector $\boldsymbol{m}$. Choose any labeling $g_1, \ldots, g_{k(\boldsymbol{m})}$ of its execution slots satisfying $|\{j : g_j = g\}| = m_g$ for all $g \in \mathcal{G}$. After all workflow executions have been completed, the firm observes

$$O_{\boldsymbol{m}} = \big(O_{g_j} : j = 1, \ldots, k(\boldsymbol{m})\big). \tag{6}$$

A post-output selector then chooses one of the candidate outputs:

$$\mathrm{Sel}(X, O_{\boldsymbol{m}}) \in \left\{A_{g_j} : j = 1, \ldots, k(\boldsymbol{m})\right\}. \tag{7}$$

The selector may be an LLM judge, a verifier, a ranking model, a rule-based policy checker, a human reviewer, or a combination of these mechanisms. In the customer-service routing example, the selector observes the proposed routing decisions and their execution traces and chooses the route submitted to the service system.

We summarize selector performance through a family of *recovery curves* $\{\psi_k(r)\}_{k=1}^{K_{\max}}$, one for each candidate-set size k. Each curve

$$\psi_k : \{0, 1, \ldots, k\} \to [0, 1], \qquad \psi_k(0) = 0, \quad \psi_k(k) = 1, \tag{8}$$

maps the number of correct candidate outputs r to the probability that the selector returns a correct final output. The endpoint conditions are natural: if no candidate is correct, the selector cannot recover one, and if all candidates are correct, any choice succeeds.

The empirical selector-aware accuracy of a nonzero execution-count vector $\boldsymbol{m}$ is

$$J_\psi(\boldsymbol{m}) = \frac{1}{n} \sum_{i=1}^{n} \psi_{k(\boldsymbol{m})}\big(r_i(\boldsymbol{m})\big), \tag{9}$$

with $J_\psi(\mathbf{0}) = 0$ by convention.

Deployment objective. The firm's goal is to choose an execution-count vector $\boldsymbol{m}$ that maximizes selector-aware accuracy on the development sample net of recurring execution cost. To place accuracy and cost on a common scale, let $\gamma \geq 0$ denote the cost price. If a correct decision is worth $v > 0$ monetary units relative to an incorrect decision, dividing monetary value by v gives $\gamma = 1/v$. The net deployed value of $\boldsymbol{m}$ is

$$\Pi_{\psi,\gamma}(\boldsymbol{m}) = J_\psi(\boldsymbol{m}) - \gamma C(\boldsymbol{m}), \qquad \Pi_{\psi,\gamma}(\mathbf{0}) = 0. \tag{10}$$

The ideal deployment objective is

$$\mathrm{OPT}_{\psi,\gamma}(\mathcal{G}) = \max_{\substack{\boldsymbol{m} \in \mathbb{Z}_+^{\mathcal{G}} \\ \boldsymbol{m} \text{ has finite support} \\ k(\boldsymbol{m}) \leq K_{\max}}} \Pi_{\psi,\gamma}(\boldsymbol{m}). \tag{11}$$

This is the best net value achievable over the full workflow space $\mathcal{G}$, and serves as the benchmark for the algorithms developed in subsequent sections.

Remark 1 (Multiple task types). The model is written for a fixed task type. In settings with several task types, the same formulation can be applied separately within each type, so that the firm chooses a type-specific workflow portfolio. A task type may be defined by operational information available before execution, such as the claim category, product line, customer segment, language, channel, risk tier, or required output format. Thus, task-type information can be used to select the relevant portfolio before workflows are run. Throughout the paper, we fix one task type and suppress the type index.

3.2. Restricted pools and the inner–outer view

Solving (11) directly is generally infeasible because the workflow class $\mathcal{G}$ is large and implicit. We therefore distinguish two objects. The first is a finite evaluated pool of workflows, over which the portfolio problem can be solved directly. The second is the larger implicit workflow class, over which the algorithm must search for additional useful workflows.

For any finite evaluated pool $\mathcal{M} \subseteq \mathcal{G}$, define the restricted benchmark

$$\mathrm{OPT}_{\psi,\gamma}(\mathcal{M}) = \max_{\substack{\boldsymbol{m}\in\mathbb{Z}_+^{\mathcal{M}} \\ k(\boldsymbol{m})\le K_{\max}}} \Pi_{\psi,\gamma}(\boldsymbol{m}). \tag{12}$$

Here, $\boldsymbol{m}$ is the execution-count vector restricted to workflow types in $\mathcal{M}$. Every workflow type $g \in \mathcal{M}$ has already been evaluated on the development sample, so its correctness vector $(a_{1g}, \ldots, a_{ng})$ and recurring execution cost c_g are known. The restricted problem (12) is the best net value achievable using only these evaluated workflow types. When $\mathcal{M} = \mathcal{G}$, it coincides with the full deployment benchmark (11); otherwise it is only a finite-pool approximation.

The role of the outer layer is to decide where to search next in the implicit class $\mathcal{G}$. The guiding principle is marginal value. Once a finite-pool linear program (LP) relaxation is solved, the optimization problem assigns dual prices to development tasks and to execution slots. A high task price means that an additional correct candidate on that task would be valuable. A high execution-slot price means that a new workflow must deliver enough marginal value to justify occupying one of the limited run slots. These prices are passed to a workflow-pricing oracle, which searches the implicit class for a workflow with high price-weighted correctness net of recurring cost. Thus, the outer layer repeatedly asks a simple operational question: *is there a workflow, not yet available to the optimizer, whose expected marginal contribution is large enough to matter?*

Workflow generator oracle. We do not answer the above question by searching $\mathcal{G}$ ourselves. Instead, we assume access to a workflow generator, which we call the pricing oracle, and which may be a teacher model, an automated workflow-search procedure, or a human engineering team. The optimizer supplies the current prices, and the oracle proposes a workflow that scores well against them. If the oracle returns such a workflow, the workflow is evaluated on the development sample and becomes available to the finite-pool optimizer. If the oracle can certify that no workflow has sufficiently large price-adjusted value, then the current LP relaxation has no important missing workflow at that query tolerance. We discuss this in more detail in Section 3.3.

The overall information flow is summarized in Figure 4.

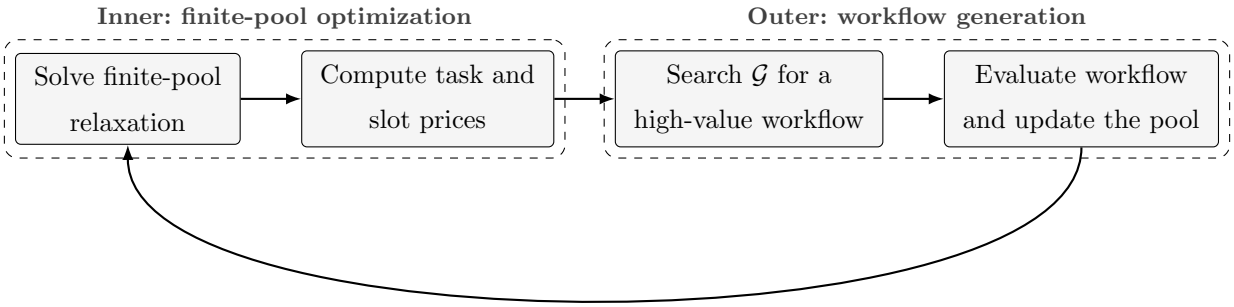


Figure 4 Inner–outer workflow-generation loop. The dashed regions distinguish the finite-pool inner optimization from the outer workflow-generation and pool-update steps.

The next subsection formalizes this workflow-pricing interface. Section 5.4 derives the corresponding task and slot prices from the LP relaxation, and Section 6 uses these prices to optimize over the implicit workflow class through an ellipsoid-based method.

3.3. Cost-aware generation over an implicit workflow class

To search beyond the currently evaluated workflow pool, the optimizer needs a way to identify promising workflows in the implicit class $\mathcal{G}$. As noted above, this search is carried out by the pricing oracle, and it is guided by nonnegative *task prices* w_i supplied by the optimizer. These prices come from the current dual solution: a larger w_i means that an additional correct output on development task i would create greater marginal value in the current LP relaxation. The prices therefore direct the oracle toward the *residual tasks*, meaning those the current workflow pool still handles poorly and on which an additional correct output would create the most value.

Given task prices $w = (w_1, \dots, w_n)$ and the cost price γ, define the global workflow-pricing value

$$P(w,\gamma) = \max_{g\in\mathcal{G}} \left\{ \sum_{i=1}^{n} w_i a_{ig} - \gamma c_g \right\}. \tag{13}$$

The term $\sum_i w_i a_{ig}$ rewards a workflow for being correct on high-price tasks, while γc_g penalizes its recurring execution cost. We refer to the difference

$$s_g(w) = \sum_{i=1}^{n} w_i a_{ig} - \gamma c_g$$

as the *score* of workflow g at prices w, so that $P(w,\gamma) = \max_{g\in\mathcal{G}} s_g(w)$. Accordingly, the pricing problem searches for a workflow with the greatest price-weighted correctness net of compute cost.

Let θ denote the current dual price of occupying an execution slot (i.e., the *slot price*). The largest violation among the workflow-indexed dual constraints is

$$\Phi(w,\theta,\gamma) = P(w,\gamma) - \theta = \max_{g\in\mathcal{G}} \{ s_g(w) - \theta \}. \tag{14}$$

Hence, $\Phi(w,\theta,\gamma) > 0$ means that some workflow creates more price-weighted value than the slot price. The pricing interface asks the oracle to identify workflows that perform well on tasks with high current prices, while screening out candidates whose recurring execution costs outweigh their potential contribution. In the customer-service routing example, high task prices may concentrate on cancellation, refund, or escalation cases for which an additional correct workflow output would be especially valuable.

Because exact maximization over $\mathcal{G}$ may itself be difficult, we allow an approximate stochastic pricing procedure. A (δ,β) *weak global pricing oracle* returns a workflow $\widetilde{g} \in \mathcal{G}$ such that, conditional on the query history, with probability at least $1-\beta$,

$$s_{\widetilde{g}}(w) \geq P(w,\gamma) - \delta. \tag{15}$$

Given a proposed slot price θ, exactly one of two things happens. If the returned workflow has score strictly greater than θ, then the proposed price cannot be right: a workflow in $\mathcal{G}$ is worth more than the slot it would occupy, even after its recurring execution cost is charged, so the optimizer has found a better workflow than it currently has. If, however, the returned workflow has score at most θ, then, on the event that the guarantee in (15) holds, no workflow in the entire class scores more than δ above the proposed price:

$$P(w,\gamma) \leq \theta + \delta.$$

Thus a single call to the oracle either produces a workflow that beats the current slot price or certifies, up to tolerance δ, that no workflow in $\mathcal{G}$ can do so. This is precisely the weak-separation information required by the ellipsoid method, which Section 6 uses to optimize over the implicit class $\mathcal{G}$ without enumerating it.

Recall that the oracle may be implemented by a teacher model, an automated workflow-search procedure, an explicit search over a prespecified workflow grammar, or a human engineering team. The theoretical guarantee developed later requires the oracle to satisfy the global approximation property in (15). In particular, the mere failure of a search procedure to find a workflow with score above θ does not certify that no such workflow exists.

4. Selector Strength and the Value of Workflow Variety

This section studies how selector quality limits the value of running multiple workflows. Even if the firm can generate many diverse candidate workflows, variety is only beneficial when the selector can reliably identify correct answers from the realized candidate set. The results in this section formalize this limitation: they bound how much any multi-workflow portfolio can outperform the best single workflow, as a function of selector strength and workflow cost. These bounds provide pre-optimization screens that can inform the firm whether expanding the portfolio or improving the selector is the more valuable investment.

The remainder of this section develops these bounds in three steps. Section 4.1 defines the odds-lift index Λ, a single scalar that summarizes how much the selector improves the odds of returning a correct output relative to picking a candidate at random. Section 4.2 uses Λ to bound the net

value of any workflow pool relative to the best available singleton workflow, producing a screening rule that can rule out portfolio expansion before solving any optimization problem. Section 4.3 specializes this bound to the Plackett–Luce choice model, in which the odds-lift index reduces to a single discrimination parameter that can be estimated directly from data.

4.1. Recovery envelopes and selector odds lift

The recovery curves $\{\psi_k\}$ introduced in Section 3 may vary with candidate-set size k and need not follow any particular random-utility model. To obtain a single structural bound that applies across all portfolio sizes, we impose the following assumption.

ASSUMPTION 1 **(Common fraction-based recovery envelope)**. *There exists a function* $H : [0,1] \to [0,1]$ *with* $H(0) = 0$ *and* $H(1) = 1$ *such that*

$$\psi_k(r) \le H\left(\frac{r}{k}\right), \qquad k = 1, \dots, K_{\max}, \quad r = 0, \dots, k. \tag{16}$$

Assumption 1 states that selector performance can be bounded by a function of the *fraction* of correct candidates alone, rather than the count r and size k separately. This reflects the premise that a selector's task difficulty is governed primarily by how concentrated the correct answer is among the alternatives it is shown, a premise that holds across many selector implementations. It is worth noting that Assumption 1 is mild: some envelope always exists. Specifically, given any finite collection of curves $\{\psi_k\}_{k=1}^{K_{\max}}$, the choice

$$H(p) := \max\{\psi_k(r) : 1 \le k \le K_{\max},\ 0 \le r \le k,\ r/k \le p\} \tag{17}$$

is a valid envelope, since $\psi_k(0) = 0$ and $\psi_k(k) = 1$ for every k.

We now introduce an index that measures how much the selector improves the odds of a correct final output relative to random selection.

DEFINITION 1 (SELECTOR ODDS-LIFT INDEX). For a recovery envelope H, define

$$\Lambda(H) = \sup_{p \in (0,1)} \frac{H(p)/(1 - H(p))}{p/(1-p)}, \tag{18}$$

with the convention that $\Lambda(H) = +\infty$ if $H(p) = 1$ for some $p \in (0,1)$.

The index $\Lambda(H)$ admits a direct interpretation as an odds ratio. At any correct fraction p, $p/(1-p)$ is the odds of returning a correct output under uniform random selection, and $H(p)/(1 - H(p))$ is the corresponding odds under the selector's recovery envelope. Their ratio measures the selector's odds improvement over random chance at that fraction, and $\Lambda(H)$ takes the largest such improvement, i.e., the supremum, over all fractions $p \in (0,1)$.

Three cases illustrate the range of $\Lambda(H)$. A selector no better than random chance has $H(p) = p$ at every fraction, giving $\Lambda(H) = 1$. A selector that systematically improves on random chance, without ever achieving certainty in returning the correct output unless all candidates are correct, has $\Lambda(H) \in (1, \infty)$. A selector that recovers the correct output with certainty at some interior fraction $p \in (0,1)$, that is, $H(p) = 1$ while $p < 1$, has infinite odds-lift, $\Lambda(H) = +\infty$, by the convention in

Definition 1. This last case is excluded whenever we assume $\Lambda(H)$ is finite, as we do throughout the results that follow.

The next lemma converts the bound $\Lambda(H) \le \Lambda$ into an explicit closed-form envelope for H.

LEMMA 1 **(Odds-lift envelope).** *If $\Lambda(H) \le \Lambda < \infty$ for some $\Lambda \ge 1$, then*

$$H(p) \le h_\Lambda(p) := \frac{\Lambda p}{1 + (\Lambda - 1)p}, \qquad p \in [0,1]. \tag{19}$$

The function h_Λ is the tight recovery envelope implied by an odds-lift bound Λ: no selector with odds lift at most Λ can exceed h_Λ at any correct fraction. We call this a Luce-shaped envelope. (The formal Plackett–Luce specialization, for which this same functional form arises exactly, is introduced in Section 4.3.) The next property, the concavity of h_Λ, is what makes the envelope useful for comparing portfolios of different sizes.

LEMMA 2 **(Shape of the odds-lift envelope).** *For every $\Lambda \ge 1$, the function h_Λ is increasing and concave on $[0,1]$.*

The key here is that the realized recovery curves ψ_k need not themselves be concave. Lemmas 1 and 2 show that a finite odds-lift bound nonetheless forces every such curve to lie below a common concave, Luce-shaped, envelope.

4.2. The value of workflow variety

We now bound $\mathrm{OPT}_{\psi,\gamma}(\mathcal{M})$, the best net value attainable using workflow types from the evaluated pool $\mathcal{M}$. For each workflow type $g \in \mathcal{M}$, define its *standalone accuracy* as its average correctness on the development sample:

$$\bar{a}_g = \frac{1}{n} \sum_{i=1}^{n} a_{ig}. \tag{20}$$

Because repeated assignments are allowed, the extremal accuracy and cost quantities for a size-k portfolio can be defined directly from the workflow types. For each $k \le K_{\max}$, define

$$A_k = \max_{g \in \mathcal{M}} \bar{a}_g, \qquad C_k = k \min_{g \in \mathcal{M}} c_g. \tag{21}$$

Indeed, all k execution slots may be assigned to the same workflow type. Thus, A_k is the largest average standalone accuracy that can be assigned to k execution slots, whereas C_k is the smallest total execution cost of any k slots. Consequently, every execution-count vector $\boldsymbol{m} \in \mathbb{Z}_+^{\mathcal{M}}$ satisfying $k(\boldsymbol{m}) = k$ obeys

$$\frac{1}{k} \sum_{g \in \mathcal{M}} \bar{a}_g m_g \le A_k \qquad \text{and} \qquad C(\boldsymbol{m}) \ge C_k.$$

The two bounds are computed separately and need not be attained by the same workflow type.

Combining these quantities with the concave odds-lift envelope h_Λ from Section 4.1 yields an upper bound on the net value of every size-k portfolio and, consequently, on $\mathrm{OPT}_{\psi,\gamma}(\mathcal{M})$.

THEOREM 1 **(Selector-limited value of workflow variety).** *Suppose Assumption 1 holds and $\Lambda(H) \le \Lambda < \infty$ for some $\Lambda \ge 1$. For a finite evaluated workflow pool $\mathcal{M}$, let $V_1 = \max\{0,$*

$\max_{g\in\mathcal{M}}\{\bar{a}_g-\gamma c_g\}\}$ *denote the best net value achievable by a singleton portfolio or the outside option. Then*

$$V_1 \le \mathrm{OPT}_{\psi,\gamma}(\mathcal{M}) \le U_{\Lambda,\gamma}(\mathcal{M}) := \max\left\{0, \max_{1\le k\le K_{\max}} [h_\Lambda(A_k)-\gamma C_k]\right\}. \tag{22}$$

In particular, when $\gamma=0$*, letting* $a^\star=\max_g \bar{a}_g$ *yields*

$$a^\star \le \mathrm{OPT}_{\psi,0}(\mathcal{M}) \le h_\Lambda(a^\star). \tag{23}$$

We call the upper bound $\mathrm{OPT}_{\psi,0}(\mathcal{M})\le h_\Lambda(a^\star)$ *in* (23) *the no-cost upper bound: it applies when the recurring workflow running costs are ignored, i.e., when* $\gamma=0$*. This no-cost upper bound is tight over the class of selectors with odds lift at most* Λ*.*

The lower bound $V_1 \le \mathrm{OPT}_{\psi,\gamma}(\mathcal{M})$ immediately holds because both the singleton portfolios and the outside option are feasible. Thus, the main content of the theorem is really the upper bound, which gives a quick test for whether any genuinely multi-workflow portfolio can be worth considering. Specifically, if

$$V_1 \ge \max_{2\le k\le K_{\max}} [h_\Lambda(A_k)-\gamma C_k], \tag{24}$$

then no multi-workflow portfolio in the evaluated pool can outperform the best singleton or the outside option. (Note that the maximum in (24) starts at $k=2$ because $k=0$ and $k=1$ are already accounted for by V_1.)

Corollary 1 **(Selector-strength and uniform-cost implications)**. *Let* $a^\star=\max_g \bar{a}_g$ *be as in Theorem 1. The following hold:*

1. *If* $\gamma=0$*, the maximum gain over the best workflow satisfies*

$$\mathrm{OPT}_{\psi,0}(\mathcal{M})-a^\star \le h_\Lambda(a^\star)-a^\star = \frac{(\Lambda-1)a^\star(1-a^\star)}{1+(\Lambda-1)a^\star} \le \frac{\sqrt{\Lambda}-1}{\sqrt{\Lambda}+1}. \tag{25}$$

2. *If every workflow has the same execution cost* c*, and*

$$\gamma c \ge h_\Lambda(a^\star)-a^\star, \tag{26}$$

then the best nonempty portfolio is a singleton. With the outside option, the global optimum is either that singleton or no deployment.

The corollary highlights the limit of adding more workflows. When $\Lambda=1$, the envelope is $h_1(p)=p$, so no combination of workflows can outperform the best individual workflow. As Λ grows, stronger selection can support more value from complementary workflows, but this value is capped by the selector-strength term in (25). The finite pool $\mathcal{M}$ can be arbitrarily large: in the no-cost bound, its size affects the cap only through the best singleton accuracy $a^\star$, not directly through $|\mathcal{M}|$. Thus a larger pool cannot by itself overcome a weak selector.

4.3. Plackett–Luce specialization

The general bound in Section 4.2 does not require a random-utility model. We now specialize to the Plackett–Luce model, a standard model for choice and ranking in which each alternative has an attraction weight and is selected with probability proportional to that weight. Beyond its foundations in choice theory, the model has been used extensively in computer science for Bayesian ranking, label ranking, rank aggregation, and online preference learning (Luce 1959, Plackett 1975, Guiver and Snelson 2009, Cheng et al. 2010, Hajek et al. 2014, Szörényi et al. 2015). The model is useful here because it gives an interpretable one-parameter measure of selector strength, closed-form fixed-size increments, and a tractable marginal-value formula for the generation loop.

Recall from Section 3 that, for a nonzero execution-count vector $\boldsymbol{m}$, the selector $\mathrm{Sel}(X, O_{\boldsymbol{m}})$ observes the task, the candidate outputs, and their execution traces and returns one candidate as the final output. To model how it makes this choice, condition on a task instance and let $Z_j = \mathbf{1}\{A_j = Y^\star\}$ indicate whether candidate j is correct. We posit that the selector assigns each candidate j a latent score

$$S_j = \eta Z_j + \varepsilon_j, \tag{27}$$

and returns the candidate with the highest score. Here $\eta \geq 0$ is the selector's *discrimination advantage*: it measures how much more attractive a correct candidate is to the selector than an incorrect one, on average. When $\eta = 0$, correct and incorrect candidates are equally attractive and the selector behaves like a coin flip; larger η means the selector systematically favors correctness. The terms ε_j are i.i.d. standard Gumbel shocks representing idiosyncratic factors, unrelated to correctness, that also influence the selector's choice. The selector never observes Z_j directly; (27) is a statistical description of the relationship between true correctness and the selector's realized preference, not a claim about the selector's internal reasoning.

Let $\lambda = \exp(\eta) \geq 1$. By the Gumbel-max identity, choosing the candidate with the highest score S_j is equivalent to choosing candidate j with probability proportional to $\exp(\eta Z_j)$: each correct candidate receives attraction weight $\lambda = \exp(\eta)$ and each incorrect candidate receives weight $1 = \exp(0)$, and the selector returns a given candidate with probability equal to its weight divided by the sum of all weights. Summing this probability over all correct candidates, if r of k candidates are correct, the probability that the selector returns a correct answer is

$$\psi^{\mathrm{PL}}_{k,\lambda}(r) = \frac{\lambda r}{\lambda r + k - r} = \frac{\lambda r}{k + (\lambda - 1) r}, \qquad r = 0, 1, \ldots, k. \tag{28}$$

This is the Plackett–Luce recovery curve. Define

$$h_\lambda(p) = \frac{\lambda p}{1 + (\lambda - 1) p}, \qquad p \in [0, 1]. \tag{29}$$

Dividing numerator and denominator of (28) by k shows $\psi^{\mathrm{PL}}_{k,\lambda}(r) = h_\lambda(r/k)$: the Plackett–Luce recovery probability depends on r and k only through the correct fraction $p = r/k$, exactly the fraction-based structure assumed in Assumption 1. Moreover, h_λ has the same functional form as the envelope h_Λ from Lemma 1, and a direct calculation confirms

$$\frac{h_\lambda(p)/(1 - h_\lambda(p))}{p/(1-p)} = \lambda, \qquad p \in (0, 1). \tag{30}$$

Thus the Plackett–Luce discrimination parameter λ is exactly its corresponding selector odds-lift index: a Plackett–Luce selector with strength λ satisfies $\Lambda(H) = \lambda$ with $H = h_\lambda$, so the general bounds of Section 4.2 apply to it with equality at $\Lambda = \lambda$, not merely as an upper bound.

The continuous concavity of h_λ follows immediately from Lemma 2 by setting $\Lambda = \lambda$. The next lemma records the additional result needed for the inner optimization problem in Section 5.

LEMMA 3 **(Fixed-size Plackett–Luce increments)**. *For every $k \geq 1$ and $\lambda \geq 1$, $\psi^{\mathrm{PL}}_{k,\lambda}$ is nondecreasing and discrete concave. Its increments are*

$$d^{\mathrm{PL}}_{\ell,k} = \psi^{\mathrm{PL}}_{k,\lambda}(\ell) - \psi^{\mathrm{PL}}_{k,\lambda}(\ell-1) = \frac{\lambda k}{(k+(\lambda-1)\ell)\,(k+(\lambda-1)(\ell-1))}, \quad \ell = 1,\dots,k. \tag{31}$$

The estimation of λ from validation data is described in Section 8.1. Once λ is computed, the general portfolio objective from Section 3 has a closed-form expression. For every nonzero execution-count vector $\boldsymbol{m}$, define

$$J_\lambda(\boldsymbol{m}) = \frac{1}{n}\sum_{i=1}^{n} h_\lambda\left(\frac{r_i(\boldsymbol{m})}{k(\boldsymbol{m})}\right), \qquad \Pi_{\lambda,\gamma}(\boldsymbol{m}) = J_\lambda(\boldsymbol{m}) - \gamma C(\boldsymbol{m}), \tag{32}$$

with both functions set to zero at $\boldsymbol{m} = \mathbf{0}$. Here $J_\lambda(\boldsymbol{m})$ plays the role of $J_\psi(\boldsymbol{m})$ from Section 3, but with the recovery curve $\psi_{k(\boldsymbol{m})}$ replaced by its Plackett–Luce form h_λ evaluated at the correct fraction $r_i(\boldsymbol{m})/k(\boldsymbol{m})$. This is the objective used in the optimization algorithms of Sections 5 and 6.

To study the structure of J_λ, we represent each execution slot as a distinct labeled copy of its workflow type. Thus, an execution-count vector $\boldsymbol{m}$ can be viewed as an ordinary subset of the ground set $\mathcal{G} \times \{1,\dots,K_{\max}\}$ containing m_g labeled copies of each workflow type g, with the second coordinate serving only to distinguish repeated executions. A natural question is whether J_λ, viewed as a set function under this representation, has the familiar properties of a coverage objective commonly found in submodular optimization.

The next proposition shows that neither property holds in general. The endogenous-size objective can decrease when an execution is added and can exhibit both increasing and decreasing marginal returns. Standard monotone-submodular optimization tools therefore do not apply directly, which motivates our development in Section 5.

PROPOSITION 1 **(More workflows need not be better)**. *For every $\lambda \geq 1$, J_λ is generally nonmonotone with respect to adding executions: there exist a feasible $\boldsymbol{m}$ and workflow type g such that selector-aware accuracy decreases $J_\lambda(\boldsymbol{m} + \boldsymbol{e}_g) < J_\lambda(\boldsymbol{m})$. Moreover, J_λ is neither submodular nor supermodular. Subtracting the modular workflow-cost term $\gamma C(\boldsymbol{m})$ preserves these failures, so $\Pi_{\lambda,\gamma}$ has the same general structure.*

The reason is a selection externality: increasing one component m_g by one changes not only the number of correct candidates $r_i(\boldsymbol{m})$ but also the total number of candidates $k(\boldsymbol{m})$ faced by the selector. If the added execution is incorrect on task i, then $r_i(\boldsymbol{m})$ is unchanged while $k(\boldsymbol{m})$ increases, this lowers the correct fraction $r_i(\boldsymbol{m})/k(\boldsymbol{m})$, and since h_λ is increasing, strictly lowers recovery probability on that task; this is why J_λ can decrease when a workflow is added, ruling

out monotonicity. Conversely, because h_λ is concave, the marginal gain from adding a correct workflow depends on the composition of the existing portfolio. A correct workflow can be more valuable after an incorrect workflow has entered the portfolio and diluted the correct fraction, producing increasing marginal returns and violating submodularity. Supermodularity would require the opposite inequality: the marginal gain from adding a workflow must weakly increase as the base portfolio becomes larger. This property also fails[1]. In Appendix D, we construct explicit examples exhibiting both failures.

5. The Inner Optimization Problem

This section studies the portfolio problem for a given nonempty evaluated workflow-type pool $\mathcal{M} \subseteq \mathcal{G}$, allowing repeated execution of the same workflow type. The resulting finite-pool formulations serve two purposes. They produce exact and approximate deployment plans for the current workflow pool, and they reveal the dual-price structure used to search over the implicit workflow class in Section 6.

The restricted problem decomposes exactly by run size:

$$\mathrm{OPT}_{\psi,\gamma}(\mathcal{M}) = \max\left\{0, \max_{1\le k\le K_{\max}} \mathrm{OPT}_k(\mathcal{M})\right\}, \qquad \mathrm{OPT}_k(\mathcal{M}) = \max_{\substack{\boldsymbol{m}\in\mathbb{Z}_+^{\mathcal{M}}\\ k(\boldsymbol{m})=k}} \Pi_{\psi,\gamma}(\boldsymbol{m}). \tag{33}$$

This decomposition isolates the source of difficulty created by endogenous run size. As shown in Proposition 1, adding an execution changes both the number of correct candidates and the total number of alternatives faced by the selector, so the global objective need not be monotone or submodular. Within a fixed-size problem, however, every feasible execution-count vector satisfies $k(\boldsymbol{m}) = k$. The selector therefore faces the same number of candidates across all portfolios in that slice, and portfolio composition affects recovery only through the correct counts $r_i(\boldsymbol{m})$. Under Assumption 2 below, each fixed-size accuracy objective is a *concave-coverage* function in the sense of Barman et al. (2021). We formulate the fixed-size problem exactly as an integer program, derive an LP relaxation and its dual prices, and develop a randomized-rounding procedure with an explicit performance certificate (Ageev and Sviridenko 2004, Chekuri et al. 2010, Calinescu et al. 2011).

The remainder of this section proceeds in five steps. Section 5.1 shows that a sparse, geometrically spaced cardinality grid certifies a near-optimal run size without solving the fixed-size problem for every k. Section 5.2 then fixes a run size and imposes a concavity assumption on each ψ_k, introducing a curvature parameter that will later control the tightness of the rounding guarantee. Section 5.3 formulates an exact integer program for each fixed size and shows its optimal value coincides with $\mathrm{OPT}_k(\mathcal{M})$. Section 5.4 relaxes this integer program to an LP whose dual prices identify the residual tasks most worth targeting with new workflows. Section 5.5 rounds the LP solution into a feasible portfolio and bounds how far its value can fall short of the true fixed-size optimum, yielding a certificate that holds across the entire cardinality grid.

[1] For example, let w_1 and w_2 be incorrect workflows and let c_1 be correct. Adding c_1 to $\{w_1\}$ produces the marginal gain $h_\lambda(1/2)$, whereas adding it to the larger set $\{w_1, w_2\}$ produces only $h_\lambda(1/3)$. Since h_λ is increasing, $h_\lambda(1/2) > h_\lambda(1/3)$, which violates increasing marginal returns. Thus some configurations exhibit increasing marginal returns and others exhibit decreasing marginal returns, so J_λ is neither submodular nor supermodular.

5.1. Sparse geometric grids for the optimal run size

Proposition 1 rules out simply deploying the largest feasible portfolio because more workflows need not increase net value. Thus, finding the best run size requires comparing across $k = 1, \dots, K_{\max}$, not assuming $k = K_{\max}$ is optimal. Doing this by solving $\mathrm{OPT}_k(\mathcal{M})$ for every candidate k can be expensive, especially since the inner problem must be resolved at every generation round as the pool grows. This subsection shows that when a single concave function generates every recovery curve, checking only a sparse, geometrically spaced set of sizes is enough to certify a near-optimal size, without solving the fixed-size problem at every k. Formally, this is the case when the fraction-based envelope of Assumption 1 holds with equality rather than merely as an upper bound. Recall that Section 4.3 showed that Plackett–Luce recovery has exactly this form.

THEOREM 2 **(Geometric cardinality approximation).** *Suppose the recovery curves are generated by a common increasing concave function $h : [0,1] \to [0,1]$ with $h(0) = 0$, so that $\psi_k(r) = h(r/k)$, $r = 0, \dots, k$. Let $\mathrm{OPT}_k^+(\mathcal{M}) = \max\{0, \mathrm{OPT}_k(\mathcal{M})\}$. Then, for every $1 \le b \le k \le K_{\max}$,*

$$\mathrm{OPT}_b^+(\mathcal{M}) \ge \frac{b}{k} \mathrm{OPT}_k^+(\mathcal{M}). \tag{34}$$

Consequently, if a cardinality grid $\mathcal{K} \subseteq \{1, \dots, K_{\max}\}$ has coverage ratio $\varrho \ge 1$, meaning that for every $k \le K_{\max}$ there exists $b \in \mathcal{K}$ with $b \le k \le \varrho b$, then

$$\max_{b \in \mathcal{K}} \mathrm{OPT}_b^+(\mathcal{M}) \ge \frac{1}{\varrho} \mathrm{OPT}_{\psi,\gamma}(\mathcal{M}). \tag{35}$$

In particular, the dyadic grid gives a $1/2$-approximation, and a grid with ratio $\varrho = 1/(1 - \eta_{\mathrm{grid}})$ gives a $(1 - \eta_{\mathrm{grid}})$-approximation using $O(\eta_{\mathrm{grid}}^{-1} \log K_{\max})$ fixed-size solves.

Inequality (34) says that a portfolio of size $b \le k$ can always secure at least a b/k fraction of the value achievable at size k: shrinking the run size costs at most proportionally, never more, because concavity of h rules out returns to scale that vanish faster than linearly. This is what makes a coarse grid safe: whichever true optimal size $k^\star$ exhaustive search would find, some grid point b within a factor ϱ of $k^\star$ still retains at least a $1/\varrho$ share of the optimal value, which is exactly (35).

Two concrete grids turn this general guarantee into the specific ratios. The *dyadic grid*

$$\mathcal{K}_{\mathrm{dyad}} = \{2^j : j = 0, 1, \dots, \lceil \log_2 K_{\max} \rceil\} \cap \{1, \dots, K_{\max}\}$$

has coverage ratio $\varrho = 2$ and $O(\log K_{\max})$ points, giving the $1/2$-approximation. More generally, for any $\varrho > 1$, the *ratio-ϱ grid*

$$\mathcal{K}_\varrho = \{\lceil \varrho^j \rceil : j = 0, 1, \dots, \lceil \log_\varrho K_{\max} \rceil\} \cap \{1, \dots, K_{\max}\}$$

has coverage ratio ϱ and $O(\log K_{\max})$ points. Taking $\varrho = 1/(1 - \eta_{\mathrm{grid}})$ gives the $(1 - \eta_{\mathrm{grid}})$-approximation using $O(\eta_{\mathrm{grid}}^{-1} \log K_{\max})$ fixed-size solves. The full grid $\mathcal{K} = \{1, \dots, K_{\max}\}$ corresponds to the case $\varrho = 1$, where every size is its own anchor and the guarantee in (35) becomes exact. Small pools can afford this exhaustive search. Large generation loops resolve the inner problem at every round, so a sparse grid, dyadic or otherwise, cuts the number of LP solves and oracle calls.

5.2. Concave recovery and selector curvature at fixed size

We now fix k and build towards solving $\mathrm{OPT}_k(\mathcal{M})$: exactly, through the integer program in Section 5.3, and at scale, through the rounding certificate in Section 5.5. This subsection lays the groundwork with the following condition on the fixed-size recovery curve ψ_k.

ASSUMPTION 2 **(Concave recovery at each fixed size).** *For every $k = 1, \dots, K_{\max}$, ψ_k is nondecreasing and discrete concave:*

$$d_{1,k} \geq d_{2,k} \geq \cdots \geq d_{k,k} \geq 0, \qquad d_{\ell,k} = \psi_k(\ell) - \psi_k(\ell-1). \tag{36}$$

When $d_{1,k} > 0$, define

$$c_{\psi,k} = 1 - \frac{d_{k,k}}{d_{1,k}}. \tag{37}$$

The curvature $c_{\psi,k} \in [0,1]$ measures how much recovery flattens as correct candidates accumulate: it is zero when marginal recovery gains are constant and approaches one when the last correct candidate contributes much less than the first. The requirements used below are nested. The exact integer-program result in Proposition 2 requires only that ψ_k be nondecreasing, equivalently, that $d_{\ell,k} \geq 0$ for every ℓ. This is weaker than Assumption 2, which additionally requires the diminishing-increment inequalities $d_{1,k} \geq \cdots \geq d_{k,k}$. The stronger condition is used only to identify the fixed-size objective as concave coverage and to establish the LP-rounding certificate. Diminishing increments are natural when the first correct candidate provides most of the selector's recoverable signal and additional correct candidates are partly redundant. For example, once the candidate set already contains a clearly correct answer, a second or third correct answer may provide less additional help to the selector than the first. Plackett–Luce recovery satisfies the condition exactly by Lemma 3.

For the rounding analysis in Section 5.5, it is useful to extend the fixed-size curve beyond the deployed range $r = 0, \dots, k$:

$$\widetilde{\psi}_k(r) = d_{k,k} r + \sum_{t=1}^{k-1} (d_{t,k} - d_{t+1,k}) \min\{r, t\}, \qquad r \geq 0. \tag{38}$$

This continuation expresses the fixed-size accuracy function as a modular linear term plus a nonnegative combination of threshold coverage functions $\min\{r,t\}$, the form used in the rounding proof, and it agrees with ψ_k at the deployed integer points $r = 0, \dots, k$, so it changes nothing about the value of any actual size-k portfolio.

5.3. Exact integer programs

For a fixed run size k, the selector-aware contribution of task i under an execution-count vector $\boldsymbol{m}$ is $\psi_k(r_i(\boldsymbol{m}))$, which is generally nonlinear in the execution counts. We linearize this term using binary tier indicators.

Let $z_g \in \mathbb{Z}_+$ denote the number of execution slots assigned to workflow type g. For each task i and tier $\ell = 1, \dots, k$, let $y_{i\ell} \in \{0,1\}$ indicate whether the execution plan produces at least ℓ correct candidates on task i. Because

$$\psi_k(r) = \sum_{\ell=1}^{r} d_{\ell,k}, \qquad d_{\ell,k} = \psi_k(\ell) - \psi_k(\ell-1),$$

an integral allocation z induces the execution-count vector with $m_g = z_g$. Weighting $y_{i\ell}$ by $d_{\ell,k}$ therefore reproduces $\psi_k(r_i(\boldsymbol{m}))$ exactly. The formulation below enforces that the active tiers form a prefix and that their total number cannot exceed the number of correct candidate executions produced on the task. For each k, solve

$$
\begin{aligned}
I_k(\mathcal{M}) = \max_{y,z} \quad & \frac{1}{n}\sum_{i=1}^{n}\sum_{\ell=1}^{k} d_{\ell,k}\, y_{i\ell} - \gamma \sum_{g\in\mathcal{M}} c_g z_g && && (39)\\
\text{s.t.} \quad & \sum_{\ell=1}^{k} y_{i\ell} \le \sum_{g\in\mathcal{M}} a_{ig} z_g, && i = 1,\dots,n, &&\\
& y_{i\ell} \le y_{i,\ell-1}, && i = 1,\dots,n, && \ell = 2,\dots,k,\\
& \sum_{g\in\mathcal{M}} z_g = k, &&&&\\
& z_g \in \mathbb{Z}_+, && g \in \mathcal{M}, &&\\
& y_{i\ell} \in \{0,1\}, && i = 1,\dots,n, && \ell = 1,\dots,k.
\end{aligned}
$$

Solving (39) is not an approximation. The next result shows its optimal value coincides exactly with the fixed-size optimum $\mathrm{OPT}_k(\mathcal{M})$, and it does so under only a nondecreasing recovery curve, without invoking the concavity assumed in Section 5.2.

Proposition 2 **(Exactness of the fixed-size inner problem)**. *If $\mathcal{M}$ is nonempty and ψ_k is nondecreasing, then $I_k(\mathcal{M}) = \mathrm{OPT}_k(\mathcal{M})$. Consequently,*

$$
\mathrm{OPT}_{\psi,\gamma}(\mathcal{M}) = \max\left\{0, \max_{1\le k\le K_{\max}} I_k(\mathcal{M})\right\}. \tag{40}
$$

Equation (40) is the size decomposition (33) with each $\mathrm{OPT}_k(\mathcal{M})$ replaced by its exact integer program value $I_k(\mathcal{M})$, so endogenous run size does not require a monolithic nonlinear formulation: exact enumeration solves (39) for every k, while Theorem 2 permits a sparse grid instead when a controlled approximation is sufficient.

5.4. LP relaxation, dual prices, and reduced costs

The exact integer program in Section 5.3 solves the inner problem for the current pool, but it says nothing about which workflows outside the pool are worth generating next. Extracting that information requires relaxing integrality and reading off dual prices.[2]

Relaxing integrality in (39) gives

$$
\begin{aligned}
L_k(\mathcal{M}) = \max_{y,z} \quad & \frac{1}{n}\sum_{i=1}^{n}\sum_{\ell=1}^{k} d_{\ell,k}\, y_{i\ell} - \gamma \sum_{g\in\mathcal{M}} c_g z_g && && (41)\\
\text{s.t.} \quad & \sum_{\ell=1}^{k} y_{i\ell} \le \sum_{g\in\mathcal{M}} a_{ig} z_g, && i = 1,\dots,n, &&
\end{aligned}
$$

[2] Chen and Chua (2024) likewise use shadow prices to coordinate primal resource allocations, although their focus is joint differential privacy rather than search over an implicit workflow class.

$$
\begin{aligned}
& y_{i\ell} \le y_{i,\ell-1}, && i=1,\dots,n, \quad \ell=2,\dots,k,\\
& \sum_{g\in\mathcal{M}} z_g = k, &&\\
& z_g \ge 0, && g\in\mathcal{M},\\
& 0 \le y_{i\ell} \le 1, && i=1,\dots,n, \quad \ell=1,\dots,k.
\end{aligned}
$$

For fixed z, let $x_i(z)=\sum_g a_{ig}z_g$, now possibly fractional. Under Assumption 2, the marginal weights $d_{\ell,k}$ are already sorted in decreasing order, so filling the y slots in index order up to $x_i(z)$ is optimal; the accuracy component then equals $\widetilde{\psi}_k(x_i(z))$, the same analytical continuation defined in (38), now evaluated at a possibly fractional argument rather than an integer one.

Let $\mu_i \ge 0$ denote the dual variable associated with the task-i coverage constraint, let $\nu_{i\ell}\ge 0$ correspond to the prefix constraint for tier ℓ, let $\sigma_{i\ell}\ge 0$ correspond to the upper bound $y_{i\ell}\le 1$, and let θ be the unrestricted dual variable associated with the fixed-cardinality constraint $\sum_g z_g = k$. Because repeated execution is allowed, the primal variables z_g have no upper bounds; hence, the dual contains no workflow-specific variables corresponding to constraints of the form $z_g \le 1$. Using the boundary convention $\nu_{i1}=\nu_{i,k+1}=0$, a dual formulation of (41) is

$$
\begin{aligned}
D_k(\mathcal{M}) = \min_{\mu,\nu,\theta,\sigma} \quad & k\theta + \sum_{i=1}^{n}\sum_{\ell=1}^{k}\sigma_{i\ell} && && (42)\\
\text{s.t.} \quad & \mu_i + \nu_{i\ell} - \nu_{i,\ell+1} + \sigma_{i\ell} \ge \frac{d_{\ell,k}}{n}, && i=1,\dots,n, \quad \ell=1,\dots,k,\\
& \sum_{i=1}^{n} a_{ig}\mu_i - \gamma c_g \le \theta, && g\in\mathcal{M},\\
& \mu_i, \sigma_{i\ell} \ge 0, && i=1,\dots,n, \quad \ell=1,\dots,k,\\
& \nu_{i\ell} \ge 0, && i=1,\dots,n, \quad \ell=2,\dots,k,\\
& \theta \text{ free.}
\end{aligned}
$$

For a workflow type $g\in\mathcal{G}\setminus\mathcal{M}$, define its reduced-cost score by

$$
\Gamma_k(g;\mu,\theta) = \sum_{i=1}^{n}\mu_i a_{ig} - \gamma c_g - \theta. \tag{43}
$$

The first term is the task-price-weighted value of the workflow's correct outputs, γc_g is its recurring execution cost, and θ is the dual price of one execution slot. Thus, $\Gamma_k(g;\mu,\theta)>0$ means that workflow g violates its full-dual constraint $\sum_{i=1}^{n}\mu_i a_{ig} - \gamma c_g \le \theta$. To see why all workflow-indexed constraints can be checked through one pricing problem, recall the score function $s_g(\mu) = \sum_{i=1}^{n}\mu_i a_{ig} - \gamma c_g$. Because repeated-execution multiplicities are uncapped, we have

$$
\max_{\substack{z_g\ge 0,\ g\in\mathcal{G}\\ \sum_{g\in\mathcal{G}} z_g = k}} \sum_{g\in\mathcal{G}} s_g(\mu) z_g = k \max_{g\in\mathcal{G}} s_g(\mu). \tag{44}
$$

Hence it is enough to solve the single global pricing problem

$$
\max_{g\in\mathcal{G}} s_g(\mu).
$$

If its value exceeds θ, the maximizing workflow identifies a missing workflow that could improve the current LP solution and should therefore be added to the evaluated pool. If its value is at most θ, no workflow in the implicit class has enough price-weighted accuracy, net of execution cost, to improve the current relaxation at these dual prices.

5.5. Cost-preserving randomized rounding and certificates

The integer program in Section 5.3, when exactly solved, already returns a deployable portfolio, with no rounding required. The obstacle is scale, not exactness: this is a concave-coverage problem, and coverage problems are known to be NP-hard in general (Nemhauser et al. 1978, Feige 1998), so solving the integer program to optimality by branch-and-bound can become expensive when $\mathcal{M}$ or n is large. This subsection trades exactness for scalability: it solves the efficient LP relaxation (41), rounds the fractional solution into a feasible portfolio, and certifies how close the result comes to the true fixed-size optimum $\mathrm{OPT}_k(\mathcal{M})$.

For any feasible fractional execution vector $z \geq 0$ satisfying $\sum_{g\in\mathcal{M}} z_g = k$, let $x_i(z) = \sum_{g\in\mathcal{M}} a_{ig} z_g$, be the fractional number of correct executions assigned to task i. We separate the relaxed objective into three components:

$$Q_k(z) = \frac{1}{n}\sum_{i=1}^{n} \widetilde{\psi}_k\big(x_i(z)\big), \qquad R_k(z) = \frac{1}{n}\sum_{i=1}^{n} x_i(z), \qquad C_k(z) = \sum_{g\in\mathcal{M}} c_g z_g. \tag{45}$$

The corresponding relaxed net value is $F_k(z) = Q_k(z) - \gamma C_k(z)$.

Let $z^{k\star}$ be an optimal solution of the size-k LP, and write $Q_k^\star = Q_k(z^{k\star})$, $R_k^\star = R_k(z^{k\star})$, and $C_k^\star = C_k(z^{k\star})$. Then $L_k(\mathcal{M}) = F_k(z^{k\star}) = Q_k^\star - \gamma C_k^\star$. To obtain an integral execution plan, consider any feasible fractional execution vector z satisfying $\sum_{g\in\mathcal{M}} z_g = k$, and define

$$q_g(z) = \frac{z_g}{k}, \qquad g \in \mathcal{M}.$$

Because $\sum_g z_g = k$, the vector $q(z)$ is a probability distribution over workflow types. Draw $G_1, \ldots, G_k \overset{\text{iid}}{\sim} q(z)$, and define the random execution-count vector

$$M_{k,g}^{\mathrm{RR}}(z) = \sum_{j=1}^{k} \mathbf{1}\{G_j = g\}, \qquad \boldsymbol{M}_k^{\mathrm{RR}}(z) = \big(M_{k,g}^{\mathrm{RR}}(z) : g \in \mathcal{M}\big).$$

Thus, each of the k execution slots independently receives a workflow type according to the fractional allocation, and

$$k\big(\boldsymbol{M}_k^{\mathrm{RR}}(z)\big) = k \quad \text{almost surely}, \qquad \mathbb{E}\big[M_{k,g}^{\mathrm{RR}}(z)\big] = z_g.$$

Hence the rounding procedure preserves every workflow multiplicity, and therefore total execution cost, in expectation. Moreover, for task i, the random number of correct candidates satisfies

$$R_i^{\mathrm{RR}}(z) := r_i\big(\boldsymbol{M}_k^{\mathrm{RR}}(z)\big) \sim \mathrm{Binomial}\left(k, \frac{x_i(z)}{k}\right).$$

When $z = z^{k\star}$, abbreviate

$$\boldsymbol{M}_k^{\mathrm{RR}} := \boldsymbol{M}_k^{\mathrm{RR}}(z^{k\star}), \qquad M_{k,g}^{\mathrm{RR}} := M_{k,g}^{\mathrm{RR}}(z^{k\star}), \qquad R_i^{\mathrm{RR}} := R_i^{\mathrm{RR}}(z^{k\star}).$$

The next theorem bounds this rounding scheme for any feasible fractional vector z, not only an exact LP optimizer.

THEOREM 3 **(Fixed-size net-value rounding certificate).** *Suppose Assumption 2 holds for run size k and $d_{1,k} > 0$. Let z be any feasible fractional execution vector for the repeated-execution LP, and let $\boldsymbol{M}_k^{\mathrm{RR}}(z)$ be the size-k execution multiset obtained by the with-replacement rounding procedure described above. Then*

$$\mathbb{E}\left[\Pi_{\psi,\gamma}\left(\boldsymbol{M}_k^{\mathrm{RR}}(z)\right)\right] \geq \left(1-\frac{1}{e}\right) Q_k(z) + \frac{d_{k,k}}{e} R_k(z) - \gamma C_k(z), \tag{46}$$

$$F_k(z) - \mathbb{E}\left[\Pi_{\psi,\gamma}\left(\boldsymbol{M}_k^{\mathrm{RR}}(z)\right)\right] \leq \frac{1}{e}\left\{Q_k(z) - d_{k,k} R_k(z)\right\} \leq \frac{c_{\psi,k}}{e} Q_k(z). \tag{47}$$

In particular, if $z = z^{k\star}$ is an optimal solution of the size-k LP, then

$$\mathrm{OPT}_k(\mathcal{M}) - \mathbb{E}\left[\Pi_{\psi,\gamma}\left(\boldsymbol{M}_k^{\mathrm{RR}}\right)\right] \leq \frac{1}{e}\left(Q_k^\star - d_{k,k} R_k^\star\right) \leq \frac{c_{\psi,k}}{e} Q_k^\star. \tag{48}$$

The final bound has a uniform interpretation. Because $0 \leq y_{i\ell}^{k\star} \leq 1$ and $\sum_{\ell=1}^{k} d_{\ell,k} = \psi_k(k) - \psi_k(0) = 1$, the LP accuracy component satisfies $0 \leq Q_k^\star \leq 1$. Consequently,

$$0 \leq \mathrm{OPT}_k(\mathcal{M}) - \mathbb{E}\left[\Pi_{\psi,\gamma}(\boldsymbol{M}_k^{\mathrm{RR}})\right] \leq \frac{c_{\psi,k}}{e} Q_k^\star \leq \frac{c_{\psi,k}}{e} \leq \frac{1}{e}.$$

Thus $c_{\psi,k}/e$ is a directly interpretable worst-case additive loss in accuracy-equivalent units, while the bound involving $Q_k^\star$ can be strictly sharper for the realized LP solution (see also Remark 2 at the end of this section).

The recurring execution-cost term does not incur any additional rounding loss. Because with-replacement rounding preserves each workflow multiplicity in expectation,

$$\mathbb{E}\left[C\left(\boldsymbol{M}_k^{\mathrm{RR}}\right)\right] = \sum_{g\in\mathcal{M}} c_g \mathbb{E}\left[M_{k,g}^{\mathrm{RR}}\right] = \sum_{g\in\mathcal{M}} c_g z_g^{k\star} = C_k^\star.$$

Thus, all loss in the certificate arises from rounding the nonlinear selector-aware accuracy term, not from execution cost. The with-replacement procedure is the natural rounding scheme for the repeated-execution model, because it permits the same workflow type to be assigned to multiple execution slots while preserving expected multiplicities and execution cost.

Specializing (37) to Plackett–Luce recovery gives a closed form for the curvature:

$$c_{k,\lambda}^{\mathrm{PL}} = 1 - \frac{k+\lambda-1}{\lambda\{k+(\lambda-1)(k-1)\}}. \tag{49}$$

This expression depends only on the run size k and selector strength λ. It satisfies $c_{1,\lambda}^{\mathrm{PL}} = 0, c_{k,1}^{\mathrm{PL}} = 0$. For $k \geq 2$ and $\lambda > 1$, the curvature is nondecreasing in both k and λ. Moreover, for fixed λ, $\lim_{k\to\infty} c_{k,\lambda}^{\mathrm{PL}} = 1 - \frac{1}{\lambda^2}$. Thus the rounding certificate is exact for a singleton portfolio and for a random selector. As the portfolio grows or the selector becomes more discriminating, the recovery curve becomes more curved: the first correct candidate accounts for a larger share of the total recovery gain, and the worst-case additive rounding certificate becomes looser. This does not mean that a

stronger selector reduces portfolio value; it means only that a linear relaxation may approximate the more strongly curved recovery objective less tightly.

For a cardinality grid $\mathcal{K}$, define the grid LP benchmark

$$U_{\mathrm{LP}}(\mathcal{K};\mathcal{M}) := \max\left\{0, \max_{k\in\mathcal{K}} L_k(\mathcal{M})\right\}. \tag{50}$$

Under Plackett–Luce recovery, $c^{\mathrm{PL}}_{k,\lambda}$ is nondecreasing in k. Let $\bar{k} = \max\mathcal{K}$ and define the worst-case rounding loss on the grid by

$$\varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda) := \frac{c^{\mathrm{PL}}_{\bar{k},\lambda}}{e} = \frac{1}{e}\max_{k\in\mathcal{K}} c^{\mathrm{PL}}_{k,\lambda}.$$

Corollary 2 **(Grid-level Plackett–Luce rounding guarantee)**. *Suppose the selector follows the Plackett–Luce recovery curve with strength $\lambda \geq 1$. For each $k \in \mathcal{K}$, let $\boldsymbol{M}^{\mathrm{RR}}_k$ be obtained by applying the with-replacement rounding procedure to an optimal solution of the size-k LP in (41). Assume the firm retains a feasible baseline execution-count vector $\boldsymbol{m}^0 \in \mathbb{Z}^{\mathcal{M}}_+$, such as a validated incumbent deployment, satisfying $\Pi_{\lambda,\gamma}(\boldsymbol{m}^0) \geq \underline{V} > 0$. After rounding, choose the best realized portfolio among the baseline and the rounded grid candidates: $\widehat{\boldsymbol{M}} \in \arg\max_{S\in\{\boldsymbol{m}^0\}\cup\{\boldsymbol{M}^{\mathrm{RR}}_k : k\in\mathcal{K}\}} \Pi_{\lambda,\gamma}(S)$. Then, we have*

$$\mathbb{E}\big[\Pi_{\lambda,\gamma}(\widehat{\boldsymbol{M}})\big] \geq \max\left\{\underline{V},\, U_{\mathrm{LP}}(\mathcal{K};\mathcal{M}) - \varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda)\right\} \geq \frac{\underline{V}}{\underline{V} + \varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda)} U_{\mathrm{LP}}(\mathcal{K};\mathcal{M}). \tag{51}$$

Moreover, if the size-k integer programs are solved exactly instead, the same guarantee holds without the expectation.

Note that because $L_k(\mathcal{M})$ is an LP relaxation of the size-k problem, we have $U_{\mathrm{LP}}(\mathcal{K};\mathcal{M}) \geq \max\left\{0, \max_{k\in\mathcal{K}} \mathrm{OPT}_k(\mathcal{M})\right\}$. Thus,

$$\mathbb{E}\big[\Pi_{\lambda,\gamma}(\widehat{\boldsymbol{M}})\big] \;\geq\; \frac{\underline{V}}{\underline{V} + \varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda)} \max_{k\in\mathcal{K}} \mathrm{OPT}_k(\mathcal{M}).$$

The first inequality in (51) combines two safeguards. The retained baseline guarantees value at least $\underline{V}$, while the rounded grid solution achieves the LP benchmark up to the uniform additive rounding loss $\varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda)$. The second inequality converts these two additive guarantees into a multiplicative bound relative to the grid LP benchmark.

The resulting factor depends only on the baseline value and the largest selector curvature among the grid sizes. Since $0 \leq \varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda) \leq 1/e$, the factor is always strictly positive whenever $\underline{V} > 0$. Moreover, the rounding loss vanishes when the fixed-size recovery curve is linear, in which case the LP solution is preserved in expectation by the rounding procedure.

Remark 2 (On the uniform bound $1/e$). The fact that the uniform bound $1/e$ in Theorem 3 does not vanish as the selector becomes perfect is structural. Under Plackett–Luce recovery, as $\lambda \to \infty$, $\psi^{\mathrm{PL}}_{k,\lambda}(r) \to \mathbf{1}\{r \geq 1\}$. The fixed-size accuracy problem then reduces to maximum coverage: the firm must choose k workflows to maximize the fraction of tasks covered by at least one correct workflow. Thus, even a perfect selector removes selection error but not the combinatorial difficulty of constructing the portfolio. The resulting $1 - 1/e$ rounding factor, or equivalently the uniform

additive loss of at most $1/e$ under normalized accuracy, is consistent with the classical maximum-coverage barrier. This gap concerns the LP-rounding method; it disappears if the fixed-size integer program is solved exactly.

6. Ellipsoid-Based Optimization Over an Implicit Workflow Class

The finite-pool formulations in Section 5 introduce one primal variable for each evaluated workflow type. When the feasible workflow class $\mathcal{G}$ is large and represented only implicitly, the corresponding repeated-execution LP may contain too many variables to enumerate. The dual, however, has only finitely many price variables. Its only implicit component is a family of constraints indexed by workflow types. This is exactly the setting in which the ellipsoid method can optimize through separation rather than explicit enumeration (Grötschel et al. 1981, 1988).

Fix a run size k. An equivalent projected dual of the full LP relaxation is

$$
\begin{aligned}
E_k(\mathcal{G}) = \min_{\mu,\xi,\theta} \quad & \sum_{i=1}^{n} \xi_i + k\theta && && (52)\\
\text{s.t.} \quad & \xi_i + r\mu_i \geq \frac{\psi_k(r)}{n}, && i = 1,\dots,n, \quad r = 0,\dots,k,\\
& 0 \leq \mu_i \leq \frac{d_{1,k}}{n}, \qquad 0 \leq \xi_i \leq \frac{1}{n}, && i = 1,\dots,n,\\
& -\gamma c_{\max} \leq \theta \leq d_{1,k},\\
& \sum_{i=1}^{n} \mu_i a_{ig} - \gamma c_g \leq \theta, && g \in \mathcal{G}.
\end{aligned}
$$

Appendix D.11 derives (52) and proves that $E_k(\mathcal{G}) = L_k(\mathcal{G})$.

6.1. Single-workflow separation

Section 3.3 introduced the global workflow-pricing value

$$
P(w,\gamma) = \max_{g\in\mathcal{G}} \left\{ \sum_{i=1}^{n} w_i a_{ig} - \gamma c_g \right\}
$$

and explained how it compares with the price θ of one execution slot. The dual in (52) now provides the formal origin of these quantities: the task prices w_i are precisely the dual variables μ_i, and the workflow-indexed constraints require $P(\mu,\gamma) \leq \theta$. Thus, the pricing interface from Section 3.3 is exactly the separation oracle needed to optimize the implicit dual. We do not solve $P(\mu,\gamma)$ by explicitly enumerating or optimizing over $\mathcal{G}$. Instead, at each candidate dual solution, we pass the task prices and execution-cost penalty to the weak global pricing oracle, which searches for a high-scoring workflow according to (15).

In particular, if the oracle returns a workflow $\widetilde{g}$ satisfying $\sum_{i=1}^{n} \mu_i a_{i\widetilde{g}} - \gamma c_{\widetilde{g}} > \theta$, then the constraint indexed by $\widetilde{g}$ in (52) is violated. That constraint supplies a separating hyperplane, which the ellipsoid method uses to exclude the current candidate dual point and continue its search. Conversely, suppose the oracle is δ-accurate in the sense of (15) and returns a workflow whose score is at most θ. The guarantee established in Section 3.3 then implies $P(\mu,\gamma) \leq \theta + \delta$. Hence all

workflow-indexed constraints are satisfied after increasing the slot price from θ to $\theta+\delta$. Because θ has coefficient k in the dual objective, this adjustment increases the objective by at most $k\delta$.

The pricing oracle therefore implements weak separation for the implicit dual: it either produces a violated workflow constraint or certifies feasibility of the entire workflow-indexed constraint family up to an objective error of $k\delta$.

6.2. Batched stochastic pricing

The pricing oracle may itself be stochastic. We assume that there exists a constant $p_{\mathrm{orc}} > 0$ such that every primitive oracle call, conditional on the full query history, satisfies the additive weak-pricing guarantee in (15) with probability at least p_{orc}. Thus, when queried at task prices μ with tolerance δ, a primitive call returns a workflow $\widetilde{g}$ satisfying

$$\sum_{i=1}^{n} \mu_i a_{i\widetilde{g}} - \gamma c_{\widetilde{g}} \geq P(\mu,\gamma) - \delta$$

with conditional probability at least p_{orc}.

A single unsuccessful pricing call cannot safely be used to certify that no violated workflow constraint exists. More seriously, an invalid separator can undermine the correctness of the entire ellipsoid routine. We therefore amplify the primitive oracle at each separation query. Specifically, the algorithm makes m pricing calls at the same dual prices and retains the returned workflow with the highest score. The batch is unsuccessful only if none of its m primitive calls satisfies the weak-pricing guarantee. Therefore, conditional on the history before the batch,

$$\mathbb{P}\{\text{batch fails} \,|\, \text{history}\} \leq (1-p_{\mathrm{orc}})^m \leq e^{-p_{\mathrm{orc}} m}. \tag{53}$$

Thus, batching converts a primitive stochastic pricing procedure with constant success probability into a weak-separation oracle whose failure probability decays exponentially in the batch size.

6.3. Ellipsoid algorithm and finite-call guarantee

We now combine the projected dual, the batched pricing oracle, and the fixed-size rounding procedure into an end-to-end algorithm. The method solves the implicit LP separately for each run size on the cardinality grid, recovers a fractional execution allocation supported on workflows discovered during separation, and rounds that allocation into a deployable portfolio. We write $\varepsilon_{\mathrm{ell}} > 0$ for the prescribed ellipsoid-method tolerance, where the subscript "ell" denotes the ellipsoid method.

For each $k \in \mathcal{K}$, let $N_{\mathrm{ell},k}(\varepsilon_{\mathrm{ell}})$ be a deterministic upper bound on the number of iterations of the while-loop in Algorithm 1 before the ellipsoid routine reaches optimization tolerance $\varepsilon_{\mathrm{ell}}/2$. Thus, in the run corresponding to size k, the iteration index takes the values $t = 0, \ldots, T_k - 1$, for some $T_k \leq N_{\mathrm{ell},k}(\varepsilon_{\mathrm{ell}})$. Classical ellipsoid-method complexity bounds imply[3] $N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}}) := \sum_{k\in\mathcal{K}} N_{\mathrm{ell},k}(\varepsilon_{\mathrm{ell}}) = \sum_{k\in\mathcal{K}} \mathrm{poly}\left(n, k, B, \log \frac{1}{\varepsilon_{\mathrm{ell}}}\right)$. Because each iteration invokes at most one batch of m primitive pricing calls, Algorithm 1 makes at most $T := m\, N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})$ primitive pricing calls.

[3] Here B denotes an upper bound on the binary encoding length of the rational problem data; see Grötschel et al. (1981, 1988).

Algorithm 1 Ellipsoid-Based Dual-Guided Workflow Optimization

Input: cardinality grid $\mathcal{K}$; recovery curves $\{\psi_k\}$; cost price γ; implicit-LP tolerance $\varepsilon_{\text{ell}} > 0$; batch size m; primitive stochastic pricing oracle; feasible baseline execution-count vector $\boldsymbol{m}^0$.
for $k \in \mathcal{K}$ **do**
  Set the pricing tolerance $\tau_k \leftarrow \frac{\varepsilon_{\text{ell}}}{2k}$.
  Initialize $\mathcal{G}_k^{\text{rec}} \leftarrow \varnothing$ and initialize the ellipsoid routine for (52) with optimization tolerance $\varepsilon_{\text{ell}}/2$; set $t \leftarrow 0$.
  while the ellipsoid routine has not met its stopping criterion **do**
    Let (μ^t, ξ^t, θ^t) be the current ellipsoid iterate.
    if an explicit constraint in (52) is violated **then**
      Supply one such violated explicit constraint to the ellipsoid routine as a separating hyperplane.
    else
      Call the primitive pricing oracle m times at (μ^t, γ, τ_k), obtaining $\widetilde{g}^{t,1}, \ldots, \widetilde{g}^{t,m}$, and set $g^t \in \arg\max_{j\in[m]} \{\sum_{i=1}^n \mu_i^t a_{i\widetilde{g}^{t,j}} - \gamma c_{\widetilde{g}^{t,j}}\}$.
      Record the returned workflow: $\mathcal{G}_k^{\text{rec}} \leftarrow \mathcal{G}_k^{\text{rec}} \cup \{g^t\}$.
      if $\sum_{i=1}^n \mu_i^t a_{ig^t} - \gamma c_{g^t} > \theta^t$ **then**
        Supply the violated workflow constraint $\sum_{i=1}^n \mu_i a_{ig^t} - \gamma c_{g^t} \le \theta$ to the ellipsoid routine as a separating hyperplane.
      else
        Supply the weak-separation certificate $P(\mu^t, \gamma) \le \theta^t + \tau_k$ to the ellipsoid routine.
      end if
    end if
    Let the ellipsoid routine perform its update and stopping test; set $t \leftarrow t+1$.
  end while
  Perform primal recovery by solving the restricted fixed-size LP relaxation (41) with $\mathcal{M} = \mathcal{G}_k^{\text{rec}}$. Let (y^k, z^k) be the resulting recovered primal solution, extended by $z_g^k = 0$ for $g \notin \mathcal{G}_k^{\text{rec}}$, such that $z_g^k \ge 0, \sum_{g\in\mathcal{G}_k^{\text{rec}}} z_g^k = k, F_k(z^k) \ge L_k(\mathcal{G}) - \varepsilon_{\text{ell}}$.
  Set $q_g^k = z_g^k/k$, draw $G_1^k, \ldots, G_k^k \overset{\text{iid}}{\sim} q^k$, and define $M_{k,g}^{\text{RR}} = \sum_{j=1}^k \mathbf{1}\{G_j^k = g\}, \boldsymbol{M}_k^{\text{RR}} = \big(M_{k,g}^{\text{RR}} : g \in \mathcal{G}_k^{\text{rec}}\big)$.
end for
Return the execution-count vector $\widehat{\boldsymbol{M}}$ with the largest realized net value among the outside option $\mathbf{0}$, the baseline $\boldsymbol{m}^0$, and the rounded vectors $\{\boldsymbol{M}_k^{\text{RR}} : k \in \mathcal{K}\}$.

THEOREM 4 **(High-probability implicit-class guarantee)**. *Suppose $\mathcal{G}$ is a finite, implicitly represented workflow-type class whose rational data have encoding length at most B, and suppose $c_g \le c_{\max}$ for every $g \in \mathcal{G}$. Suppose the selector follows the Plackett–Luce recovery curve with strength $\lambda \ge 1$, and let the cardinality grid $\mathcal{K}$ have coverage ratio $\varrho \ge 1$. Retain a feasible baseline portfolio $\boldsymbol{m}^0$ satisfying $\Pi_{\lambda,\gamma}(\boldsymbol{m}^0) \ge \underline{V} > 0$. Suppose further that every primitive pricing call satisfies (15), conditional on the query history, with probability at least $p_{\text{orc}} > 0$. Run Algorithm 1 with batch size*

m. Then, with probability at least

$$1 - N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})e^{-p_{\mathrm{orc}}m} = 1 - \exp\left\{-\frac{p_{\mathrm{orc}}T}{N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})} + \log N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})\right\}, \tag{54}$$

all batched separation calls satisfy their weak-pricing guarantees, and the returned portfolio obeys

$$\mathbb{E}_{\mathrm{rnd}}\left[\Pi_{\lambda,\gamma}(\widehat{\boldsymbol{M}})\right] \geq \max\left\{\underline{V},\, \frac{1}{\varrho}\mathrm{OPT}_{\lambda,\gamma}(\mathcal{G}) - \varepsilon_{\mathrm{ell}} - \varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda)\right\}. \tag{55}$$

Consequently,

$$\mathbb{E}_{\mathrm{rnd}}[\Pi_{\lambda,\gamma}(\widehat{\boldsymbol{M}})] \geq \frac{1}{\varrho}\frac{\underline{V}}{\underline{V} + \varepsilon_{\mathrm{ell}} + \varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda)}\mathrm{OPT}_{\lambda,\gamma}(\mathcal{G}). \tag{56}$$

Here, the expectation $\mathbb{E}_{\mathrm{rnd}}$ is taken over the final randomized-rounding step, conditional on the successful batched-separation event.

To make the probability of any failed batch at most β, it is sufficient to choose

$$m \geq \frac{1}{p_{\mathrm{orc}}}\left[\log N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}}) + \log\frac{1}{\beta}\right]. \tag{57}$$

The resulting primitive-call complexity is

$$T = O\left(\frac{N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})}{p_{\mathrm{orc}}}\log\frac{N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})}{\beta}\right). \tag{58}$$

The guarantee separates three distinct sources of loss. The factor $1/\varrho$ comes from replacing exhaustive search over all run sizes by the cardinality grid. The term $\varepsilon_{\mathrm{ell}}$ is the error from solving the implicit LP only approximately, and $\varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda)$ is the loss from converting the recovered fractional allocations into integral execution portfolios. The stochastic pricing oracle affects the confidence level, but not the value bound conditional on successful separation.

Most importantly, the benchmark in (55)–(56) is the best portfolio over the full implicit workflow class $\mathcal{G}$, not merely over the workflows explicitly generated during the algorithm. The method therefore does not require enumerating $\mathcal{G}$ or discovering every workflow with positive reduced cost; it requires only enough global pricing calls to optimize the implicit dual to the prescribed tolerance.

7. Stochastic Workflow Outcomes

Sections 3 through 6 treat a_{ig} as deterministic: workflow g either solves development task i or it does not, and repeated executions reproduce the same outcome. In practice, an execution of the same workflow on the same task need not return the same answer twice. In this section, we reinterpret $a_{ig} \in [0,1]$ as the probability that a single execution of workflow g is correct on task i, with the deterministic model recovered when every a_{ig} is zero or one.

The deployment decision is unchanged. It remains the execution-count vector $\boldsymbol{m} = (m_g)_{g\in\mathcal{G}} \in \mathbb{Z}_+^{\mathcal{G}}$, with finite support. What changes is that the number of correct candidate answers on a task is now random rather than determined by $\boldsymbol{m}$.

The stochastic model requires the following independent-sampling assumption.

ASSUMPTION 3 **(IID workflow execution)**. *For every development task i and workflow type g, there is a parameter $a_{ig} \in [0,1]$. Each execution of workflow g on task i has a correctness indicator distributed as* Bernoulli(a_{ig}). *Correctness indicators are mutually independent across tasks, workflow types, execution copies, and sampling stages. Thus, for each fixed pair (i,g), repeated executions are i.i.d. Bernoulli draws with mean a_{ig}.*

For a feasible $\boldsymbol{m}$, let $Z_{igq} \sim \text{Bernoulli}(a_{ig})$ denote the correctness indicator of copy q of workflow g on task i. Under Assumption 3, the random number of correct outputs on task i is

$$R_i(\boldsymbol{m}) = \sum_{g\in\mathcal{G}} \sum_{q=1}^{m_g} Z_{igq} \in \{0,\dots,k(\boldsymbol{m})\}, \qquad \mathbb{E}[R_i(\boldsymbol{m})] = \sum_{g\in\mathcal{G}} m_g a_{ig}. \tag{59}$$

The stochastic net value is

$$\Pi^{\text{stoch}}_{\psi,\gamma}(\boldsymbol{m}) = \mathbb{E}\left[\frac{1}{n}\sum_{i=1}^{n} \psi_{k(\boldsymbol{m})}\big(R_i(\boldsymbol{m})\big)\right] - \gamma C(\boldsymbol{m}), \qquad \Pi^{\text{stoch}}_{\psi,\gamma}(\mathbf{0}) = 0. \tag{60}$$

Recovery is evaluated at the realized correct count and only then averaged. In general, $\mathbb{E}[\psi_k(R_i(\boldsymbol{m}))] \neq \psi_k(\mathbb{E}[R_i(\boldsymbol{m})])$, so the deterministic objective is not recovered by substituting expected correctness for realized correctness. For a finite workflow pool $\mathcal{M} \subseteq \mathcal{G}$, define

$$\text{OPT}^{\text{stoch}}_{\psi,\gamma}(\mathcal{M}) = \max_{\substack{\boldsymbol{m}\in\mathbb{Z}_+^{\mathcal{M}} \\ k(\boldsymbol{m})\le K_{\max}}} \Pi^{\text{stoch}}_{\psi,\gamma}(\boldsymbol{m}).$$

Selector-strength bound. The deterministic selector-strength bound of Theorem 1 extends to stochastic execution. Only the reading of $\bar{a}_g$ changes: under (20) it was the fraction of development tasks workflow g solves, whereas it is now the average probability that one execution of g is correct. Recall that $h_\Lambda(p) = \Lambda p/\{1+(\Lambda-1)p\}$ from Lemma 1.

PROPOSITION 3 **(Selector odds lift under stochastic execution)**. *Suppose Assumption 1 holds and $\Lambda(H) \le \Lambda < \infty$. Every feasible nonzero $\boldsymbol{m}$ satisfies*

$$\frac{1}{n}\sum_{i=1}^{n} \mathbb{E}\big[\psi_{k(\boldsymbol{m})}\big(R_i(\boldsymbol{m})\big)\big] \le h_\Lambda\left(\frac{1}{k(\boldsymbol{m})}\sum_{g\in\mathcal{G}} m_g \bar{a}_g\right). \tag{61}$$

For a finite pool $\mathcal{M}$, let $a^\star = \max_{g\in\mathcal{M}} \bar{a}_g$, $c_{\min} = \min_{g\in\mathcal{M}} c_g$, and $V_1^{\text{stoch}} = \max\{0, \max_{g\in\mathcal{M}}(\bar{a}_g - \gamma c_g)\}$. Then

$$V_1^{\text{stoch}} \le \text{OPT}^{\text{stoch}}_{\psi,\gamma}(\mathcal{M}) \le \max\left\{0, \max_{1\le k\le K_{\max}} [h_\Lambda(a^\star) - \gamma k c_{\min}]\right\}. \tag{62}$$

If $\gamma = 0$, then

$$a^\star \le \text{OPT}^{\text{stoch}}_{\psi,0}(\mathcal{M}) \le h_\Lambda(a^\star), \qquad \text{OPT}^{\text{stoch}}_{\psi,0}(\mathcal{M}) - a^\star \le \frac{\sqrt{\Lambda}-1}{\sqrt{\Lambda}+1}. \tag{63}$$

The conclusion is the same as in the deterministic model. Under the iid law, $\mathbb{E}[R_i(\boldsymbol{m})] = \sum_g m_g a_{ig}$, and Jensen's inequality reduces the bound to the same average success probabilities $\bar{a}_g$. Thus stochastic execution adds no new term to the selector-strength cap; the additional statistical issue is that the probabilities a_{ig} must now be estimated.

Implicit workflow generation. The probabilities a_{ig} are unknown. Whenever a workflow g is first made available to the optimizer, we execute it independently L_1 times on every development task and define

$$\widehat{a}_{ig}^{(1)} = \frac{1}{L_1}\sum_{\ell=1}^{L_1} Z_{ig}^{(1,\ell)}, \qquad Z_{ig}^{(1,\ell)} \overset{\text{iid}}{\sim} \text{Bernoulli}(a_{ig}). \tag{64}$$

The superscript (1) marks the Stage-1 estimation sample. For the analysis, an independent L_1-sample panel may be associated with every workflow in the finite class $\mathcal{G}$ and revealed only when that workflow is queried. This lazy-sampling interpretation does not require the algorithm to enumerate $\mathcal{G}$. For every $\delta_1 \in (0,1)$, Hoeffding's inequality and a union bound give

$$\mathbb{P}\left\{\max_{\substack{i=1,\dots,n\\ g\in\mathcal{G}}} \left|\widehat{a}_{ig}^{(1)} - a_{ig}\right| \le \varepsilon_a(L_1,\delta_1)\right\} \ge 1-\delta_1, \qquad \varepsilon_a(L_1,\delta_1) := \sqrt{\frac{\log(2n|\mathcal{G}|/\delta_1)}{2L_1}}. \tag{65}$$

Let $\widehat{\mathbb{P}}_1$ be the product-Bernoulli execution law obtained by replacing a_{ig} with $\widehat{a}_{ig}^{(1)}$, and let $\widehat{\mathbb{E}}_1$ denote expectation under this plug-in law. Define

$$\widehat{\Pi}_{\psi,\gamma}^{(1)}(\boldsymbol{m}) = \widehat{\mathbb{E}}_1\left[\frac{1}{n}\sum_{i=1}^{n}\psi_{k(\boldsymbol{m})}\big(R_i(\boldsymbol{m})\big)\right] - \gamma C(\boldsymbol{m}), \qquad \widehat{\Pi}_{\psi,\gamma}^{(1)}(\mathbf{0}) = 0. \tag{66}$$

On the concentration event in (65), a common-uniform coupling of the true and plug-in Bernoulli executions gives

$$\sup_{\substack{\boldsymbol{m}\in\mathbb{Z}_+^{\mathcal{G}}\\ k(\boldsymbol{m})\le K_{\max}}} \left|\widehat{\Pi}_{\psi,\gamma}^{(1)}(\boldsymbol{m}) - \Pi_{\psi,\gamma}^{\text{stoch}}(\boldsymbol{m})\right| \le \varepsilon_{\text{est}}(L_1,\delta_1) := \min\left\{1, K_{\max}\varepsilon_a(L_1,\delta_1)\right\}. \tag{67}$$

Algorithm 2 in Appendix F applies the grid, ellipsoid, batched-pricing, primal-recovery, and rounding arguments to the plug-in iid law. Its workflow-indexed constraints are separated using the plug-in stochastic pricing score in (131). Although the iid execution law is determined by the marginal probabilities, this stochastic reduced cost is not generally the deterministic mean-column score $\sum_i \mu_i \widehat{a}_{ig}^{(1)} - \gamma c_g$: the marginal task price depends on the realized number of other correct candidates in the same execution scenario. Every newly proposed workflow is therefore evaluated using its own fresh L_1-sample panel before its plug-in stochastic score is used.

Fresh-sample evaluation. Let $\mathcal{M}_T$ be the workflow pool produced by the generation stage. Because $\mathcal{M}_T$ was assembled using the Stage-1 estimates, those same executions are not reused to select the final portfolio. We freeze $\mathcal{M}_T$ and collect a second, independent sample. Specifically, for every replication $\ell = 1,\dots,L_2$, task i, workflow $g \in \mathcal{M}_T$, and potential execution copy $q = 1,\dots,K_{\max}$, let $Z_{igq}^{(2,\ell)} \sim \text{Bernoulli}(a_{ig})$ be mutually independent Stage-2 draws, independent of the complete generation history. For a candidate execution-count vector $\boldsymbol{m}$, define

$$R_i^{(2,\ell)}(\boldsymbol{m}) = \sum_{g\in\mathcal{M}_T}\sum_{q=1}^{m_g} Z_{igq}^{(2,\ell)}. \tag{68}$$

Thus one Stage-2 replication supplies a common fresh execution table from which every candidate portfolio over $\mathcal{M}_T$ can be evaluated. Set

$$\widehat{\Pi}^{\mathrm{final}}_{L_2}(\boldsymbol{m}) = \frac{1}{L_2}\sum_{\ell=1}^{L_2}\left[\frac{1}{n}\sum_{i=1}^{n}\psi_{k(\boldsymbol{m})}\big(R_i^{(2,\ell)}(\boldsymbol{m})\big)\right] - \gamma C(\boldsymbol{m}), \qquad \widehat{\Pi}^{\mathrm{final}}_{L_2}(\mathbf{0}) = 0. \tag{69}$$

For a cardinality grid $\mathcal{K}$, define $\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T) = \{\mathbf{0}\} \cup \left\{\boldsymbol{m} \in \mathbb{Z}_+^{\mathcal{M}_T} : k(\boldsymbol{m}) \in \mathcal{K}\right\}$. Let N_T denote the size of this class. Since repeated execution is allowed,

$$N_T = 1 + \sum_{k\in\mathcal{K}} \binom{|\mathcal{M}_T| + k - 1}{k}, \qquad \varepsilon_{\mathrm{samp}}(L_2, \delta_2) := \sqrt{\frac{\log(2N_T/\delta_2)}{2nL_2}}. \tag{70}$$

Conditional on the frozen generated pool, Hoeffding's inequality and a union bound imply that, with probability at least $1-\delta_2$, every portfolio in $\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)$ is evaluated within $\varepsilon_{\mathrm{samp}}(L_2, \delta_2)$ of its true stochastic value.

Let

$$\widetilde{\boldsymbol{m}} \in \operatorname*{arg\,max}_{\boldsymbol{m}\in\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)} \widehat{\Pi}^{\mathrm{final}}_{L_2}(\boldsymbol{m})$$

be the Stage-2 empirical maximizer. Retain a feasible baseline $\boldsymbol{m}^0 \in \mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)$ whose true stochastic value is known to satisfy $\Pi^{\mathrm{stoch}}_{\psi,\gamma}(\boldsymbol{m}^0) \geq \underline{V} > 0$, and return

$$\widehat{\boldsymbol{m}} = \begin{cases} \widetilde{\boldsymbol{m}}, & \widehat{\Pi}^{\mathrm{final}}_{L_2}(\widetilde{\boldsymbol{m}}) - \varepsilon_{\mathrm{samp}}(L_2, \delta_2) \geq \underline{V}, \\ \boldsymbol{m}^0, & \text{otherwise.} \end{cases} \tag{71}$$

The rule switches away from the baseline only when the fresh Stage-2 value clears $\underline{V}$ by more than the uniform sampling deviation.

The complete two-stage procedure is stated as Algorithm 2 in the appendix. The following theorem records its end-to-end guarantee.

THEOREM 5 **(High-probability stochastic implicit-class guarantee)**. *Suppose Assumption 3 holds, $\mathcal{G}$ is a finite implicitly represented workflow-type class with $c_g \leq c_{\max}$. Suppose $\psi_k(r) = h(r/k)$ for a common increasing concave function $h : [0,1] \to [0,1]$ with $h(0) = 0$ and $h(1) = 1$, and let $\mathcal{K}$ have coverage ratio $\varrho \geq 1$. Retain a baseline $\boldsymbol{m}^0$ satisfying $\Pi^{\mathrm{stoch}}_{\psi,\gamma}(\boldsymbol{m}^0) \geq \underline{V} > 0$.*

Run Algorithm 2 with sample sizes L_1, L_2, confidence levels $\delta_1, \delta_2 \in (0,1)$, ellipsoid tolerance $\varepsilon_{\mathrm{ell}} > 0$, and pricing-batch size m. Suppose every primitive pricing call satisfies (134), conditional on the query history, with probability at least $p_{\mathrm{orc}} > 0$, and let $N^{\mathrm{stoch}}_{\mathrm{price}}(\varepsilon_{\mathrm{ell}})$ bound the total number of pricing batches. Define $\varepsilon_{\mathrm{rnd}} := \max_{k\in\mathcal{K}} \frac{c_{\psi,k}}{e}$. Then, with probability at least $1 - \delta_1 - \delta_2 - N^{\mathrm{stoch}}_{\mathrm{price}}(\varepsilon_{\mathrm{ell}})e^{-p_{\mathrm{orc}}m}$, the returned execution-count vector satisfies

$$\Pi^{\mathrm{stoch}}_{\psi,\gamma}(\widehat{\boldsymbol{m}}) \geq \max\left\{\underline{V}, \frac{1}{\varrho}\mathrm{OPT}^{\mathrm{stoch}}_{\psi,\gamma}(\mathcal{G}) - \varepsilon_{\mathrm{ell}} - \varepsilon_{\mathrm{rnd}} - 2\varepsilon_{\mathrm{est}}(L_1, \delta_1) - 2\varepsilon_{\mathrm{samp}}(L_2, \delta_2)\right\}. \tag{72}$$

Consequently,

$$\Pi^{\mathrm{stoch}}_{\psi,\gamma}(\widehat{\boldsymbol{m}}) \geq \frac{1}{\varrho}\frac{\underline{V}}{\underline{V} + \varepsilon_{\mathrm{ell}} + \varepsilon_{\mathrm{rnd}} + 2\varepsilon_{\mathrm{est}}(L_1, \delta_1) + 2\varepsilon_{\mathrm{samp}}(L_2, \delta_2)}\mathrm{OPT}^{\mathrm{stoch}}_{\psi,\gamma}(\mathcal{G}). \tag{73}$$

8. Numerical Experiments

The numerical study addresses two questions. First, how reliably can a post-output selector identify a correct answer from a mixed candidate set, and how does this ability vary across applications and selector models? We study selector recovery on three domains (ABCD, Schema-Guided Dialogue (SGD), and HotpotQA) using three selector model families. Second, does selector-aware workflow portfolio design improve deployment performance? We answer these questions using each data set by evaluating the complete pipeline: initial workflow evaluation, selector calibration, cost-aware portfolio optimization, dual-guided workflow generation, selector recalibration after the workflow bank expands, and fresh held-out deployment.

The three domains differ in the type of decision the system must make and the information available for making it. ABCD (Chen et al. 2021a) is a customer-service routing task with a common operational taxonomy: after observing the first few customer turns, the system must identify the appropriate service subflow. SGD (Rastogi et al. 2020) contains task-oriented conversations between users and virtual assistants across many services and domains. Each service is accompanied by a schema describing the intents it supports and the information fields relevant to those intents. The system must therefore interpret a conversation relative to a service-specific set of possible intents and slots, rather than a single taxonomy shared across all conversations. HotpotQA (Yang et al. 2018) is an open-domain question-answering task in which answering a question typically requires combining information from multiple passages. It therefore provides a substantively different setting in which candidate workflows must perform multistep evidence-based reasoning. Together, the three domains span fixed-taxonomy service routing, schema-dependent dialogue understanding, and multistep question answering.

Common experimental protocol. Candidate-generating workflows are stochastic. For every task–workflow pair, we execute the workflow independently $L = 5$ times using `do_sample=True`, temperature one, and distinct random seeds. We retain each execution's output, correctness indicator, seed, and parse status and estimate

$$\widehat{a}_{ig} = \frac{1}{5} \sum_{\ell=1}^{5} Z_{ig\ell}.$$

Selectors are evaluated deterministically (`do_sample=False`). Candidate order is randomized to reduce position effects. When one task contributes multiple selector panels or orderings, confidence intervals treat the task, rather than each individual selector decision, as the independent sampling unit.

In all three end-to-end experiments, we set $K_{\max} = 6$ and search $K \in \{1, \ldots, 6\}$. The primary formulation permits repeated execution of a workflow type, and we also report a no-repeat benchmark imposing $m_g \leq 1$. We optimize the LP relaxation using a 16-scenario sample-average approximation (SAA), where each scenario is drawn from the plug-in independent Bernoulli execution law defined by the estimated probabilities $\widehat{a}_{ig}$. We then apply 512 randomized-rounding trials. For every returned integral portfolio, we compute the distribution of the number of correct outputs exactly: because this count is a sum of independent Bernoulli variables with potentially different

probabilities, it follows a Poisson–binomial distribution. We average the selector recovery curve over this exact distribution rather than reporting the finite 16-scenario average.

A correct final decision is valued at \$1, recurring compute cost is expressed using the small-model reference-call normalization described below, and we set $\gamma = 1$. To conserve space, we report ABCD in detail, summarize the SGD and HotpotQA results in Section 8.3, and provide their complete experiments in Sections G.1 and G.2 in the Appendix.

8.1. ABCD customer-service routing

Task and data. ABCD is a customer-service dialogue dataset organized around operational flows and policy-constrained service actions (Chen et al. 2021a). We study an early routing task: after observing at most the customer's first three messages and the available dialogue context, the system must identify the appropriate service subflow from 96 possible labels, including routes associated with order cancellation, refund status, subscription changes, troubleshooting, and escalation.We preserve the official data splits and the naturally occurring distribution of labels within each split. The experiment uses 800 development tasks to evaluate workflows and optimize the portfolio, all 1,004 available tasks from the development split to calibrate the selector, and 800 tasks from the official test split for final held-out evaluation. No single label accounts for more than 4.2% of the observations in any split. Thus, performance cannot be driven by repeatedly predicting a small number of dominant classes; the system must distinguish among a large set of possible routing decisions.

Initial workflow bank and stochastic execution. We begin with a deliberately simple bank of 18 single-call workflows, formed by crossing three generator models (Qwen2.5-3B-Instruct, Mistral-7B-Instruct-v0.3, and Granite-3.3-8B-Instruct) with six prompting strategies. The strategies differ in the reasoning process the model is asked to follow before making the routing decision. The *direct* strategy asks for the routing label immediately. *Evidence first* asks the model to identify the parts of the conversation that are most informative for the decision before choosing a label. *Decomposition* asks it to consider the customer's intent, the current service state, and the routing logic separately before combining them into a final decision. *Verify and revise* asks the model to make an initial judgment, check it for possible errors, and revise it if needed within the same response. *Alternatives* asks the model to compare several plausible routing labels before selecting one, while *limited context* deliberately withholds part of the available dialogue information to create a less informed workflow. Importantly, these differences do not change the workflow topology: every initial workflow consists of a single model call, with no separate verifier, fallback call, or multistep agent interaction. This provides a controlled starting bank against which we later evaluate whether richer multistep agentic workflows add value.

We set the reference cost of one 7B-model call to 0.001 and scale execution cost linearly with model size and the number of model calls.[4] The generated workflows instead consist of executable

[4] As of August 22, 2026, Together AI prices Qwen2.5-7B-Instruct-Turbo, a 7B-parameter model, at 0.30 per million input tokens and 0.30 per million output tokens. At these rates, a call to this 7B model using a total of 3,333.33 input and output tokens costs $3{,}333.33 \times \frac{0.30}{10^6} = 0.001$. Thus, the 0.001 reference cost corresponds directly to the market price of a 3,333.33-token call to a hosted 7B model.

Table 1 ABCD design and selector calibration. The primary selector uses execution slots rather than answer deduplication and aggregates cyclic candidate-order rotations by vote. Confidence intervals for λ are clustered at the task level.

Quantity	setting or estimate
Development / calibration / held-out tasks	800/1,004/800
Allowed routing labels	96 ABCD subflows
Initial workflow bank	18 workflows: 3 generator models $\times$ 6 reasoning strategies
Stochastic execution	$L = 5$ independent draws per task–workflow cell; temperature one
Run-size search	$K \in \{1, 2, 3, 4, 5, 6\}$; repeats allowed in the primary analysis
Workflow generation	one-scenario common-uniform ellipsoid pricing; 20 distinct-candidate evaluation budget
Final portfolio optimization	16 SAA scenarios and 512 randomized-rounding trials per K
Selector	Qwen2.5-7B-Instruct; greedy decoding; up to six cyclic rotations
Initial-bank calibration sample	1,500 balanced panels from 586 distinct tasks
Initial-bank selector strength	$\widehat{\lambda} = 2.9848$, 95% CI $[2.4930, 3.5844]$
Initial-bank implied pairwise recovery	$\widehat{\lambda}/(1+\widehat{\lambda}) = 74.90\%$
Average selector advantage over uniform choice	21.33 percentage points on the initial-bank calibration design
Expanded-bank selector strength	$\widehat{\lambda} = 2.5229$, 95% CI $[2.0918, 3.0670]$
Expanded-bank implied pairwise recovery	$\widehat{\lambda}/(1+\widehat{\lambda}) = 71.61\%$
Selector parse rate	100% on both initial- and expanded-bank calibration panels

compositions of GPT-4o-mini calls connected through Python control flow. Depending on the generated program, calls may be sequenced or run in parallel and combined through voting, critique, verification, clarification, or conditional fallback. OpenAI describes GPT-4o-mini as a small model and, at its release, characterized it as belonging to roughly the same small-model tier as models such as Llama 3 8B. For consistency in experimental cost accounting, we assign each GPT-4o-mini invocation one reference small-model call and the same normalized cost of \$0.001.

Across the 800 development tasks, the Mistral decomposition workflow has the highest estimated one-execution accuracy, at 46.625%. However, when each of the 18 initial workflows is executed once, the probability that at least one of them produces the correct answer rises to 69.896%. This difference shows that the workflows are complementary: many tasks missed by the best individual workflow are solved by another workflow in the pool.

Precision and stability of the $L = 5$ execution design. With only five executions for each task-workflow pair, the individual $\widehat{a}_{ig}$ estimates can be noisy.[5] Our goal, however, is not to estimate every task-workflow success probability precisely. Rather, we need sufficiently stable estimates of each workflow's average performance across the 800 development tasks to support portfolio construction. We therefore examine whether the workflow-level results are sensitive to using only a few executions.

[5] For reference, estimating a single Bernoulli success probability with a worst-case 95% Wilson interval requires $L = 93$ for a ± 0.10 margin and $L = 381$ for a ± 0.05 margin.

For each workflow, we recompute its average accuracy using only the first $L' = 1, 2, 3$ executions and compare the resulting workflow ranking with the ranking based on all $L = 5$ executions. We measure agreement using Spearman's rank correlation (Spearman 1961), where values close to one indicate similar rankings. The correlations are 0.945, 0.971, and 0.958 for $L' = 1, 2, 3$, respectively. Relative to the $L = 5$ estimates, the largest absolute change in any workflow's average accuracy across these comparisons is 1.25 percentage points, approximately 2.7% of the best workflow's 46.625% average accuracy, and four of the five highest-ranked workflows remain in the top five. These results suggest that, although individual task-workflow estimates remain noisy, $L = 5$ provides reasonably stable workflow-level information for portfolio construction.

Bank-specific selector calibration. We next estimate how well the selector can distinguish correct from incorrect outputs produced by the initial ABCD workflow bank. For each $K \in \{2, \dots, 6\}$ and each possible number of correct candidates $r \in \{1, \dots, K-1\}$, we construct 100 candidate panels containing exactly r correct execution outputs and $K - r$ incorrect outputs. The selector observes the task, dialogue context, and candidate labels, but does not observe workflow identity, generator identity, execution cost, or which candidates are correct. The resulting calibration sample contains 1,500 panels drawn from 586 distinct tasks.

We estimate selector strength by maximum likelihood under the Plackett–Luce recovery model. Let $S_j \in \{0, 1\}$ indicate whether the selector chooses a correct candidate on calibration panel j, where r_j of the K_j candidates are correct. Under the Plackett–Luce specification,

$$\operatorname{logit} \Pr(S_j = 1 \mid r_j, K_j) = \alpha + \log\left(\frac{r_j}{K_j - r_j}\right), \qquad \lambda = \exp(\alpha).$$

Thus, λ measures how strongly the selector favors correct candidates relative to incorrect ones, and we estimate it by fitting the logistic model above. The resulting estimate is $\widehat{\lambda} = 2.9848$, with a task-clustered 95% confidence interval of $[2.4930, 3.5844]$. Under the fitted model, when exactly one of two candidates is correct, the selector chooses the correct candidate with probability 74.90%. Averaged across all panel compositions in the calibration design, its fitted probability of selecting a correct candidate is 21.33 percentage points higher than under uniform random selection.

Figure 5 compares the observed selector recovery rates with two benchmarks: uniform random selection and the fitted Plackett–Luce curve. The fitted curve captures the main pattern that selector performance improves as the fraction of correct candidates increases, although the empirical recovery rates do not lie exactly on the one-parameter model. We thus use Plackett–Luce as a simple operational approximation for portfolio optimization rather than as an exact behavioral description of the selector. To check whether this approximation leads to useful deployment decisions, we subsequently evaluate the optimized portfolios using the actual selector on fresh held-out tasks. The final deployment results therefore do not depend solely on the fitted parametric recovery model.

Initial-pool optimization. We next test whether the complementarity in the initial bank can be converted into deployed value after accounting for selector confusion and compute cost. We set $K_{\max} = 6$, matching the largest candidate set used in selector calibration. For every $K \in \{1, \dots, 6\}$, we solve the repeat-allowed stochastic LP relaxation using 16 independent execution-outcome scenarios drawn from the plug-in Bernoulli law. We then apply randomized rounding 512 times and

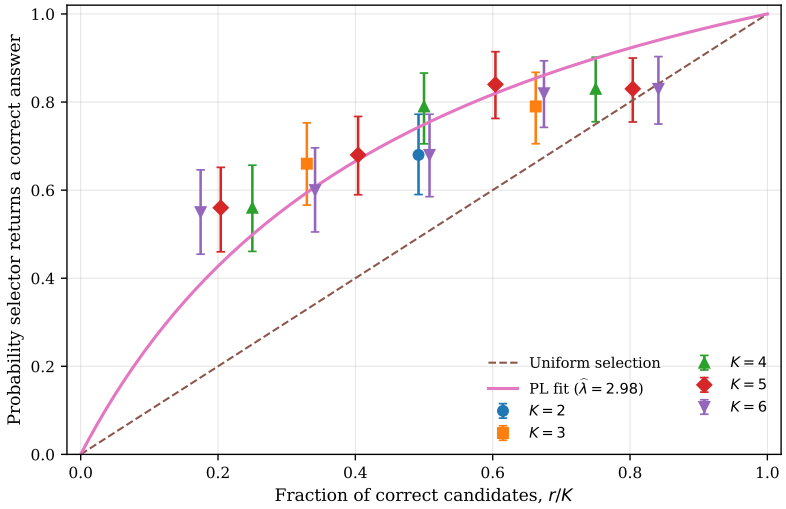


Figure 5 ABCD selector recovery. Points show task-clustered estimates for controlled (K, r) cells, vertical bars show 95% task-clustered bootstrap intervals, the dashed line is uniform selection, and the solid curve is the fitted Plackett–Luce model. Small horizontal offsets separate cells with the same correct fraction.

evaluate each distinct integral portfolio using the exact Poisson–binomial calculation described above. The best portfolio from the initial workflow bank has $K = 6$ and contains one Granite verify-and-revise execution, one Mistral decomposition execution, two Mistral verify-and-revise executions, one Qwen evidence-first execution, and one Qwen verify-and-revise execution. On the development sample, its calibrated selector accuracy is 52.523%, and its workflow-cost-adjusted value is 0.520074 per task. Thus, the optimized portfolio assigns two execution slots to the Mistral verify-and-revise workflow. When repeated execution is prohibited, the best no-repeat portfolio has a slightly lower development value of 0.518755.

Dual-guided workflow generation. We use ADAS Meta Agent Search (Hu et al. 2025) to generate new workflows, with OpenAI's GPT-4o-mini serving both as the meta-agent that proposes workflow structures and as the execution model within the generated workflows. The search is guided by task prices from the stochastic dual. These prices identify development tasks on which an additional correct workflow output would be most valuable, allowing ADAS to focus its search on weaknesses of the current workflow bank. To implement this idea, for each K we run the ellipsoid separation procedure associated with Algorithm 2, using a one-scenario sample-average approximation (SAA) of the stochastic projected dual. Because the stochastic dual assigns a separate slot price θ_q to each labeled execution copy $q = 1, \ldots, K$, workflow pricing is checked separately for each copy. At a given ellipsoid iterate, we first check the explicit dual constraints and the workflow constraints corresponding to workflows already evaluated. ADAS is invoked only if these constraints do not separate the current iterate. Whenever ADAS proposes a new workflow, we freeze its design and evaluate it independently using the same $L = 5$ execution protocol before using its estimated performance in subsequent pricing calculations.

We impose a prespecified budget of 20 distinct new workflow evaluations during generation. After the generation stage, the final finite-pool portfolio optimization uses 16 SAA scenarios and 512

randomized-rounding trials. Across the fixed-K searches, the procedure makes 26 ADAS pricing queries and incorporates four generated workflows, expanding the evaluated bank from 18 to 22 workflow types. Because each fixed-K search is subject to its prespecified workflow-evaluation budget, the resulting expanded bank should be viewed as the output of a budgeted dual-guided search rather than an exhaustive search over the full workflow class.

Expanded-bank recalibration and reoptimization. After adding the generated workflows, we recalibrate the same Qwen selector using a new controlled panel constructed from outputs in the expanded workflow bank. This step allows the estimated selector model to reflect the new kinds of candidate outputs it will encounter after workflow generation. The estimated selector strength is $\widehat{\lambda} = 2.5229$, with a 95% confidence interval of $[2.0918, 3.0670]$. This interval overlaps the initial-bank confidence interval, although the point estimate is lower. The comparison illustrates why we recalibrate after expanding the workflow bank: λ summarizes selector performance for the candidate distribution it faces and should not be treated as a fixed intrinsic property of the selector.

We then reoptimize the portfolio over all 22 workflow types. The best repeat-allowed portfolio has $K = 4$, where K counts complete workflow executions, or equivalently the four candidate outputs ultimately presented to the selector. Two of these four slots are assigned to generated workflow G4 (`FinalABCDRouterWithLabel`) in Table 2. A single execution of G4 makes three independently seeded GPT-4o-mini routing calls on the same ABCD task and returns the label receiving the most votes. Assigning G4 to two slots therefore means running this entire three-call workflow twice independently, producing two candidate outputs. The remaining two slots contain one initial-bank Mistral verify-and-revise execution and one Qwen evidence-first execution. Thus, the $K = 4$ portfolio presents four candidate outputs to the final selector but uses eight underlying model calls per task: six from the two G4 executions and one from each of the two initial-bank workflows.

On the development sample, this portfolio has calibrated selector accuracy of 54.190% and a workflow-cost-adjusted value of 0.534410. The no-repeat benchmark also selects $K = 4$, but replaces the second copy of G4 with G3 (`Final_Single_Routing_Agent`), a generated router with critique and fallback. Its workflow-cost-adjusted value is 0.533396.

Composition of the generated workflows. Table 2 summarizes the four workflows generated by ADAS and added to the evaluated workflow bank. Every model call within these workflows uses GPT-4o-mini. Thus, the generated workflows differ not in the underlying model, but in how they organize multiple calls and combine their outputs, using mechanisms such as parallel sampling, voting, critique, clarification, and conditional fallback. The repeat-allowed optimal portfolio assigns two execution slots to `FinalABCDRouterWithLabel`. Each deployed task therefore receives two independent runs of this three-call voting workflow, together with one Mistral and one Qwen workflow from the initial bank. In the no-repeat benchmark, the second copy is replaced by one execution of `Final_Single_Routing_Agent`.

Fresh held-out deployment. We freeze all reported portfolios before examining the 800 held-out tasks. For each workflow type appearing in a reported portfolio, we collect five new stochastic executions per task. We first evaluate each portfolio under the plug-in Plackett–Luce model, using the estimated workflow success probabilities and selector strength. We then evaluate actual deploy-

Table 2 ADAS-generated workflows incorporated into the expanded ABCD bank. All generated model calls use GPT-4o-mini through the ADAS `LLMAgentBase` interface. Each GPT-4o-mini invocation is assigned one $0.001 reference-call unit; mean calls and normalized cost are measured during independent workflow evaluation.

Workflow	Executable structure	Mean calls	Cost
G1	Five independent routing calls followed by a plurality vote. If the highest vote is tied, three additional routing calls are made and a second plurality vote resolves the tie.	8.0	0.008
G2	Five independent routing calls followed by a plurality vote. If the leading vote is tied, one additional clarification routing call determines the returned label.	6.0	0.006
G3	One initial routing call followed by a critique/validation call. If the proposed label is invalid, ambiguous, or unsure, an additional fallback routing call is made.	2.0	0.002
G4	Three independent routing calls on the same task, followed by a vote; the workflow returns the label with the largest vote count.	3.0	0.003

Notes. G1–G4 correspond respectively to the frozen ADAS programs `FinalEnhancedRoutingAgent`, `FinalRoutingAgent`, `Final_Single_Routing_Agent`, and `FinalABCDRouterWithLabel`. Cost is recurring normalized execution cost per workflow run.

ment performance by using Qwen2.5-7B-Instruct as the final selector. The selector does not observe workflow identity or correctness. To reduce sensitivity to candidate position, we present each candidate set under up to six cyclic rotations of the candidate order and use the candidate receiving the most selection votes across rotations.

8.2. Results on ABCD

Table 3 and figure 6 report deployment performance on 800 fresh held-out ABCD tasks. Actual selector accuracy increases from 43.625% for the best initial singleton to 46.750% for the optimized initial-bank portfolio and to 50.250% for the portfolio obtained after ellipsoid-guided workflow generation. Thus, optimizing the initial workflow bank contributes 3.125 percentage points, and expanding the bank through workflow generation contributes an additional 3.500 percentage points. Relative to the initial singleton, the complete procedure gains 6.625 percentage points, or 15.2%. The same ordering appears under the calibrated Plackett–Luce model, where predicted selector accuracy rises from 43.550% to 47.269% and then to 50.648%. Notably, the final portfolio uses only four execution slots, compared with six for the optimized initial-bank portfolio. The generated workflows therefore allow the system to use fewer workflow executions while achieving higher held-out accuracy. The no-repeat benchmark reaches 50.125%, only 0.125 percentage points below the primary repeat-allowed solution.

The improvement does not come simply from increasing the probability that at least one candidate is correct. That probability, which we refer to as oracle coverage, decreases modestly from 59.517% for the optimized initial-bank portfolio to 58.264% for the expanded-bank portfolio. At the same time, the accuracy obtained by selecting uniformly at random from the realized candidate outputs rises from 40.533% to 45.681%. Thus, the generated workflows produce candidate sets in which correct outputs are more prevalent, even though the chance of having at least one correct

Table 3 Fresh held-out ABCD deployment on 800 tasks. PL accuracy is the plug-in Plackett–Luce prediction; actual accuracy uses the blind deterministic Qwen selector. Brackets report 95% Wilson intervals. Workflow and selector costs are per task, and actual total-system net subtracts both.

Method	K	PL accuracy	Actual accuracy (95% CI)	Random accuracy	Oracle coverage	Workflow cost	Selector cost	Actual total-system net
Best initial singleton	1	0.4355	0.4363 [0.4023, 0.4708]	0.4355	0.4355	0.001043	0.000000	0.435207
Optimized initial bank	6	0.4727	0.4675 [0.4332, 0.5021]	0.4053	0.5952	0.005157	0.006514	0.455829
ADAS-expanded bank	4	0.5065	0.5025 [0.4679, 0.5371]	0.4568	0.5826	0.007486	0.004343	0.490671
ADAS-expanded, no-repeat	4	0.5069	0.5013 [0.4667, 0.5358]	0.4572	0.5829	0.006486	0.004343	0.490421

output is slightly lower. After subtracting both workflow execution cost and the realized cost of the selector, actual total-system net value increases from 0.435207 for the singleton to 0.455829 for the optimized initial-bank portfolio and 0.490671 for the final ADAS-expanded portfolio.

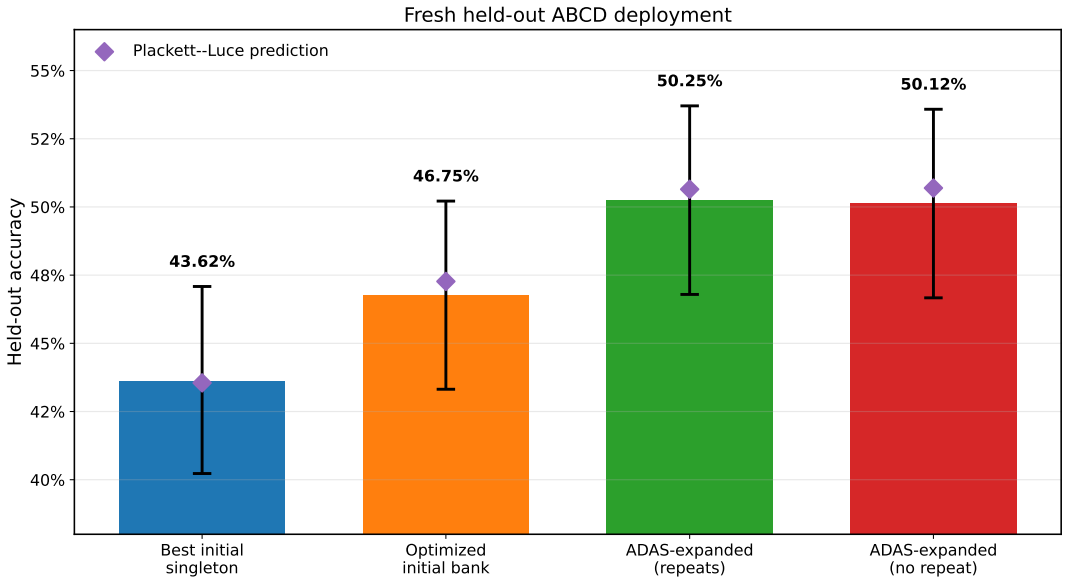


Figure 6 Fresh held-out ABCD accuracy. Bars report actual deterministic selector accuracy, error bars show 95% Wilson intervals over 800 tasks, and diamonds report the corresponding plug-in Plackett–Luce predictions.

8.3. Results on SGD and HotpotQA

The same end-to-end pipeline produces different sources of value across the other two domains. On SGD, the initial workflow bank is already strong and complementary: optimizing it raises actual held-out selector accuracy from 85.250% for the best singleton to 92.750%. None of the 20 evaluated ADAS candidates is incorporated at the queried dual prices, so the final portfolio coincides with the optimized initial-bank portfolio. Thus, SGD demonstrates the value of portfolio optimization without requiring workflow generation.

On HotpotQA, optimizing the initial bank raises actual held-out accuracy only from 30.250% to 31.125%, but dual-guided generation incorporates a three-call evidence-extraction, verification, and multihop-solving workflow. Two independent executions of this workflow attain 55.250% actual accuracy; one execution attains 54.875% and has slightly higher total-system net value after selector cost is included. Together with ABCD, these results show that portfolio optimization, workflow

generation, and multiplicity are distinct sources of value whose importance varies across applications. Complete experimental designs, calibration results, portfolio compositions, and held-out evaluations are reported in Sections G.1 and G.2.

9. Conclusion

Agentic AI systems often approach the same task through multiple workflows and then rely on a selector to choose the final answer. This paper shows that workflow variety creates both opportunity and risk. While additional executions may solve cases that the current portfolio misses, they also consume compute and may introduce plausible wrong answers that make final selection more difficult. We develop a framework for managing this tradeoff. Our selector strength bound places a sharp limit on the value of workflow variety and identifies conditions under which a single workflow or no deployment is optimal. We then develop optimization methods for choosing the run size and allocating execution slots across workflow types. For large implicit workflow classes, dual prices guide a workflow generation oracle toward residual tasks on which an additional correct output would be most valuable, while the ellipsoid method provides performance guarantees without requiring the workflow class to be enumerated.

The numerical results illustrate both the value and the limits of workflow portfolios. Relative to the best initial singleton, the final portfolio raises actual held-out selector accuracy by 6.625 percentage points on ABCD, 7.500 points on SGD, and 25.000 points on HotpotQA. Workflow generation adds value on ABCD and HotpotQA but not on SGD, where optimizing the already strong initial bank is sufficient. Multiplicity likewise has heterogeneous value: it binds in ABCD and HotpotQA but not in SGD, and its incremental accuracy benefit need not justify its additional compute and selection cost. The central message is that more inference time variety is not automatically better. Effective deployment requires firms to coordinate workflow generation, run size, execution allocation, and compute expenditure with the recovery capabilities of the selector.

References

Abdolmaleki M, Duenyas I (2026) Hold back, team up, or take over? optimal timing of AI access. URL http://dx.doi.org/10.2139/ssrn.6953340, available at SSRN 6953340.

Abdolmaleki M, Duenyas I, Kapuscinski R (2026) Pricing delayed agentic AI services: When high-value jobs wait longer. URL http://dx.doi.org/10.2139/ssrn.6721378, available at SSRN 6721378.

Ageev AA, Sviridenko MI (2004) Pipage Rounding: A New Method of Constructing Algorithms with Proven Performance Guarantee. *Journal of Combinatorial Optimization* 8(3):307–328.

Agrawal S, Wang Z, Ye Y (2014) A dynamic near-optimal algorithm for online linear programming. *Operations Research* 62(4):876–890.

Alaei S, Makhdoumi A, Malekian A (2010) Maximizing sequence-submodular functions and its application to online advertising. *arXiv preprint arXiv:1009.4153* .

Baek J, Fu Y, Ma W, Peng T (2026) Ai agents for inventory control: Human-llm-or complementarity. *arXiv preprint arXiv:2602.12631* .

Barman S, Fawzi O, Fermé P (2021) Tight Approximation Guarantees for Concave Coverage Problems. *38th International Symposium on Theoretical Aspects of Computer Science (STACS 2021)*, volume 187 of *Leibniz International Proceedings in Informatics (LIPIcs)*, 9:1–9:17.

Bertsimas D, Mišić VV (2019) Exact first-choice product line optimization. *Operations Research* 67(3):651–670.

Calinescu G, Chekuri C, Pál M, Vondrák J (2011) Maximizing a Monotone Submodular Function Subject to a Matroid Constraint. *SIAM Journal on Computing* 40(6):1740–1766.

Chekuri C, Vondrák J, Zenklusen R (2010) Dependent Randomized Rounding via Exchange Properties of Combinatorial Structures. *Proceedings of the 51st Annual IEEE Symposium on Foundations of Computer Science*, 575–584.

Chen D, Chen H, Yang Y, Lin A, Yu Z (2021a) Action-Based Conversations Dataset: A Corpus for Building More In-Depth Task-Oriented Dialogue Systems. URL https://arxiv.org/abs/2104.00783.

Chen D, Chua GA (2024) Noisy dual mirror descent: A near optimal algorithm for jointly-dp convex resource allocation. *Advances in Neural Information Processing Systems* 37:91723–91755.

Chen L, Davis JQ, Hanin B, Bailis P, Zaharia M, Zou J, Stoica I (2025) Optimizing Model Selection for Compound AI Systems. URL https://arxiv.org/abs/2502.14815.

Chen L, Zaharia M, Zou J (2024) FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research* .

Chen Q, Jasin S, Duenyas I (2019) Nonparametric Self-Adjusting Control for Joint Learning and Optimization of Multiproduct Pricing with Finite Resource Capacity. *Mathematics of Operations Research* 44(2):601–631.

Chen Q, Jasin S, Duenyas I (2021b) Technical Note—Joint Learning and Optimization of Multi-Product Pricing with Finite Resource Capacity and Unknown Demand Parameters. *Operations Research* 69(2):560–573.

Cheng W, Dembczyński K, Hüllermeier E (2010) Label Ranking Methods Based on the Plackett–Luce Model. *Proceedings of the 27th International Conference on Machine Learning*, 215–222.

Dantzig GB, Wolfe P (1960) Decomposition Principle for Linear Programs. *Operations Research* 8(1):101–111.

Dean A, Zhang Z, Jasin S, Liu Y (2026) Multi-LLM Query Optimization. URL https://arxiv.org/abs/2603.24617.

Desaulniers G, Desrosiers J, Solomon MM, eds. (2005) *Column Generation* (Springer).

Dong J, Mo W, Qi Z, Shi C, Fang EX, Tarokh V (2023) Pasta: pessimistic assortment optimization. *International Conference on Machine Learning*, 8276–8295 (PMLR).

Echenique F, Fallah A, Jordan MI (2025) A general framework for estimating preferences using response time data. *arXiv preprint arXiv:2507.20403* .

Elmachtoub AN, Grigas P (2022) Smart "predict, then optimize". *Management science* 68(1):9–26.

Faradonbeh MKS, Faradonbeh MSS (2023) Online reinforcement learning in stochastic continuous-time systems. *The Thirty Sixth Annual Conference on Learning Theory*, 612–656 (PMLR).

Feige U (1998) A Threshold of $\ln n$ for Approximating Set Cover. *Journal of the ACM* 45(4):634–652.

Gartner (2025a) Gartner Predicts 40% of Enterprise Apps Will Feature Task-Specific AI Agents by 2026, Up from Less Than 5% in 2025. Gartner Newsroom, URL https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agents-by-2026-up-fr accessed: 2026-06-26.

Gartner (2025b) Gartner Predicts Over 40% of Agentic AI Projects Will Be Canceled by End of 2027. Gartner Newsroom, URL https://www.gartner.com/en/newsroom/press-releases/2025-06-25-gartner-predicts-over-40-percent-of-agentic-ai-projects-will-be-canceled-by-end-of-2027, accessed: 2026-06-26.

Grötschel M, Lovász L, Schrijver A (1981) The ellipsoid method and its consequences in combinatorial optimization. *Combinatorica* 1(2):169–197, URL http://dx.doi.org/10.1007/BF02579273.

Grötschel M, Lovász L, Schrijver A (1988) *Geometric Algorithms and Combinatorial Optimization*, volume 2 of *Algorithms and Combinatorics* (Berlin: Springer-Verlag), URL http://dx.doi.org/10.1007/978-3-642-97881-4.

Guiver J, Snelson E (2009) Bayesian Inference for Plackett–Luce Ranking Models. *Proceedings of the 26th Annual International Conference on Machine Learning*, 377–384, URL http://dx.doi.org/10.1145/1553374.1553423.

Guo Y, Jasin S, Lin CA (2026) Congestion-Aware Static LLM Cascades: Analysis of a Steady-State Framework. Available at SSRN, sSRN 6759238.

Hajek B, Oh S, Xu J (2014) Minimax-Optimal Inference from Partial Rankings. *Advances in Neural Information Processing Systems*, volume 27, 1475–1483.

Harvard Business Review Analytic Services (2026) From the Edge to the Core: Bringing Agentic AI to the Heart of the Enterprise. Harvard Business Review Sponsored Content, URL https://hbr.org/sponsored/2026/01/from-the-edge-to-the-core, accessed: 2026-06-26.

Hu QJ, Bieker J, Li X, Jiang N, Keigwin B, Ranganath G, Keutzer K, Upadhyay S (2024) RouterBench: A Benchmark for Multi-LLM Routing System. URL https://arxiv.org/abs/2403.12031.

Hu S, Lu C, Clune J (2025) Automated Design of Agentic Systems. *International Conference on Learning Representations*.

Huang K, Shi Y, Ding D, Li Y, Fei Y, Lakshmanan LVS, Xiao X (2025) ThriftLLM: On Cost-Effective Selection of Large Language Models for Classification Queries. *Proceedings of the VLDB Endowment* 18(11):4410–4423.

Jasin S (2014) Reoptimization and Self-Adjusting Price Control for Network Revenue Management. *Operations Research* 62(5):1168–1178.

Jiang D, Ren X, Lin BY (2023) LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.

Keskin NB, Zeevi A (2014) Dynamic pricing with an unknown demand model: Asymptotically optimal semi-myopic policies. *Operations research* 62(5):1142–1167.

Kleywegt AJ, Shapiro A, Homem-de Mello T (2002) The Sample Average Approximation Method for Stochastic Discrete Optimization. *SIAM Journal on Optimization* 12(2):479–502.

Li G, Liang J, Liu M, Lei Y, Jasin S, Yang F, Baxi P (2026) Asymptotically Optimal Sequential Testing with Heterogeneous LLMs. URL https://arxiv.org/abs/2604.01086.

Liang Y, Wu C, Song T, Wu W, Xia Y, Liu Y, Ou Y, Lu S, Ji L, Mao S, Wang Y, Shou L, Gong M, Duan N (2024) TaskMatrix.AI: Completing tasks by connecting foundation models with millions of APIs. *Intelligent Computing* 3:0063, URL http://dx.doi.org/10.34133/icomputing.0063.

Liu F, Liu Y, Zhang Q, Tong X, Yuan M (2026a) EoH-S: Evolution of Heuristic Set Using LLMs for Automated Heuristic Design. *Proceedings of the AAAI Conference on Artificial Intelligence* 40(43):37090–37098.

Liu F, Yao Y, Guo P, Yang Z, Zhao Z, Lin X, Tong X, Yuan M, Lu Z, Wang Z, Zhang Q (2026b) A Systematic Survey on Large Language Models for Algorithm Design. *ACM Computing Surveys* .

Lu P, Peng B, Cheng H, Galley M, Chang KW, Wu YN, Zhu SC, Gao J (2023) Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems*, volume 36, 43447–43478.

Luce RD (1959) *Individual Choice Behavior: A Theoretical Analysis* (New York: Wiley).

McFadden D (1974) Conditional Logit Analysis of Qualitative Choice Behavior. Zarembka P, ed., *Frontiers in Econometrics*, 105–142 (New York: Academic Press).

McKinsey & Company (2025) The State of AI: Global Survey 2025. McKinsey QuantumBlack, URL https://www.mckinsey.com/capabilities/quantumblack/our-insights/the-state-of-ai, accessed: 2026-06-26.

McKinsey & Company (2026) Building the Foundations for Agentic AI at Scale. McKinsey Technology, URL https://www.mckinsey.com/capabilities/mckinsey-technology/our-insights/building-the-foundations-for-agentic-ai-at-scale, accessed: 2026-06-26.

Nemhauser GL, Wolsey LA, Fisher ML (1978) An Analysis of Approximations for Maximizing Submodular Set Functions—I. *Mathematical Programming* 14:265–294.

Novikov A, Vũ N, Eisenberger M, Dupont E, Huang PS, Wagner AZ, Shirobokov S, Kozlovskii B, Ruiz FJR, Mehrabian A, Kumar MP, See A, Chaudhuri S, Holland G, Davies A, Nowozin S, Kohli P, Balog M (2025) AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. URL https://arxiv.org/abs/2506.13131.

Ong I, Almahairi A, Wu V, Chiang WL, Wu T, Gonzalez JE, Kadous MW, Stoica I (2025) RouteLLM: Learning to Route LLMs with Preference Data. *International Conference on Learning Representations*.

Plackett RL (1975) The Analysis of Permutations. *Journal of the Royal Statistical Society: Series C* 24(2):193–202.

Radvand T, Abdolmaleki M, Mostagir M, Tewari A (2026) A training-free method for LLM text attribution. URL http://dx.doi.org/10.2139/ssrn.6169527, available at SSRN 6169527; also available as arXiv:2501.02406.

Ransbotham S, Kiron D, Khodabandeh S, Iyer S, Das A (2025) The Emerging Agentic Enterprise: How Leaders Must Navigate a New Age of AI. MIT Sloan Management Review and Boston Consulting Group, URL https://shop.sloanreview.mit.edu/the-emerging-agentic-enterprise-how-leaders-must-navigate-a-new-age-of-ai, accessed: 2026-06-26.

Rastogi A, Zang X, Sunkara S, Gupta R, Khaitan P (2020) Towards Scalable Multi-Domain Conversational Agents: The Schema-Guided Dialogue Dataset. *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, 8689–8696.

Saad-Falcon J, Lafuente AG, Natarajan S, Maru N, Todorov H, Guha E, Buchanan EK, Chen M, Guha N, Ré C, Mirhoseini A (2025) Archon: An Architecture Search Framework for Inference-Time Techniques. *International Conference on Learning Representations*.

Shen Y, Song K, Tan X, Li D, Lu W, Zhuang Y (2023) HuggingGPT: Solving AI tasks with ChatGPT and its friends in Hugging Face. *Advances in Neural Information Processing Systems*, volume 36, 38154–38180.

Spearman C (1961) The proof and measurement of association between two things. .

Szörényi B, Busa-Fekete R, Paul A, Hüllermeier E (2015) Online Rank Elicitation for Plackett–Luce: A Dueling Bandits Approach. *Advances in Neural Information Processing Systems*, volume 28, 604–612.

Talluri KT, van Ryzin GJ (2004) *The Theory and Practice of Revenue Management* (Springer).

van Ryzin G, Mahajan S (1999) On the Relationship Between Inventory Costs and Variety Benefits in Retail Assortments. *Management Science* 45(11):1496–1509.

Wang J, Wang J, Athiwaratkun B, Zhang C, Zou J (2025) Mixture-of-Agents Enhances Large Language Model Capabilities. *International Conference on Learning Representations*.

Wang X, Wei J, Schuurmans D, Le Q, Chi EH, Narang S, Chowdhery A, Zhou D (2023) Self-Consistency Improves Chain of Thought Reasoning in Language Models. *International Conference on Learning Representations*.

Wang Z, Peura H, Wiesemann W (2024) Randomized assortment optimization. *Operations Research* 72(5):2042–2060.

Yang Z, Qi P, Zhang S, Bengio Y, Cohen WW, Salakhutdinov R, Manning CD (2018) HotpotQA: A Dataset for Diverse, Explainable Multi-Hop Question Answering. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, 2369–2380.

Zhang G, Niu L, Fang J, Wang K, Bai L, Wang X (2025a) Multi-Agent Architecture Search via Agentic Supernet. URL https://arxiv.org/abs/2502.04180.

Zhang H, Jasin S (2022) Online Learning and Optimization of (Some) Cyclic Pricing Policies in the Presence of Patient Customers. *Manufacturing & Service Operations Management* 24(2):1165–1182.

Zhang J, Xiang J, Yu Z, Teng F, Chen X, Chen J, Zhuge M, Cheng X, Hong S, Wang J, Zheng B, Liu B, Luo Y, Wu C (2025b) AFlow: Automating Agentic Workflow Generation. *International Conference on Learning Representations*.

Zhuge M, Wang W, Kirsch L, Faccio F, Khizbullin D, Schmidhuber J (2024) GPTSwarm: Language Agents as Optimizable Graphs. *International Conference on Machine Learning*.

Appendices for "Managing Agentic AI Workflow Portfolios under Imperfect Selection and Compute Cost."

Appendix A: A Graph-Based Formalization of Workflow Classes

This appendix gives one formal representation of the implicit workflow class $\mathcal{G}$. The main analysis requires only that each workflow g can be executed on a task, produces a candidate answer and trace, and has an evaluated correctness column and recurring cost. A graph grammar is useful for making the feasible class explicit, but the portfolio optimization results do not depend on this particular representation.

Let $\mathcal{U} = \{u_1, \dots, u_Q\}$ denote a library of primitive executable modules. A module may be a prompt-based agent, retriever, solver, verifier, critic, tool call, aggregation rule, learned classifier, neural-network block, or answer formatter. A workflow g is a finite directed computation graph

$$g = (V_g, E_g, \ell_g, s_g, t_g),$$

where V_g is a finite node set, E_g represents information flow, $\ell_g : V_g \to \mathcal{U}$ assigns a module to each node, and s_g, t_g are input and output nodes. We impose $|V_g| \le L_{\max}$ and any application-specific restrictions on retrieval, tools, model calls, memory, context length, or termination. The feasible workflow class $\mathcal{G}$ consists of all graphs satisfying these restrictions.

Appendix B: Other selector technologies

The endogenous framework allows a different recovery curve ψ_k for each candidate-set size k. The selector-strength bound in Section 4 applies whenever these curves admit a common fraction-based envelope with finite odds-lift. The fixed-size IP remains valid for any nondecreasing recovery curve, while the randomized-rounding certificate additionally requires diminishing increments. A selector that reaches recovery probability one at an interior correct fraction has infinite odds-lift, so the uniform selector-strength cap becomes vacuous even though the exact size-conditioned IP remains applicable.

Conservative clean recovery. Suppose each correct candidate is rejected with probability β, each incorrect candidate is falsely certified with probability α, and verifier outputs are conditionally independent. The event that at least one correct candidate is certified and no incorrect candidate is certified has probability

$$\psi^{\text{clean}}_{\alpha,\beta,k}(r) = \mathbf{1}\{r \ge 1\}(1-\beta^r)(1-\alpha)^{k-r}. \tag{74}$$

Uniform selection among certified candidates. If the selector chooses uniformly among certified candidates, with $U \sim \text{Binomial}(r, 1-\beta)$ and $W \sim \text{Binomial}(k-r, \alpha)$,

$$\psi^{\text{unif}}_{\alpha,\beta,k}(r) = \sum_{u=1}^{r} \sum_{w=0}^{k-r} \frac{u}{u+w} \binom{r}{u} (1-\beta)^u \beta^{r-u} \binom{k-r}{w} \alpha^w (1-\alpha)^{k-r-w}. \tag{75}$$

Voting. Under a conservative model in which accepted wrong workflows coordinate on the same wrong answer, let $C_r \sim \text{Binomial}(r, 1-\beta)$ and $W_{k-r} \sim \text{Binomial}(k-r, \alpha)$. A sufficient recovery event is $C_r > W_{k-r}$, giving

$$\psi^{\text{vote}}_{\alpha,\beta,k}(r) = \mathbb{P}\{C_r > W_{k-r}\}. \tag{76}$$

Voting is typically threshold-like and need not be concave. The size-conditioned IP remains exact, but the concave-coverage rounding certificate does not apply unless the estimated curve satisfies diminishing increments.

Appendix C: A Numerical Illustration of Workflow Multiplicity

This example illustrates why it can be optimal to assign multiple execution slots to the same workflow type under an imperfect selector. Suppose there are two deterministic workflow types, A and B, and three task types with the correctness patterns and population shares shown in Table 4.

Table 4 Task types and workflow correctness.

Task type	Workflow A	Workflow B	Population share
Type 1	Correct	Incorrect	0.40
Type 2	Incorrect	Correct	0.20
Type 3	Incorrect	Incorrect	0.40

For compactness, write execution-count vectors in the coordinate order (A, B), and define

$$\boldsymbol{m}^{A} = (1,0), \qquad \boldsymbol{m}^{AB} = (1,1), \qquad \boldsymbol{m}^{AAB} = (2,1).$$

Workflow A has standalone accuracy 0.40, whereas workflow B has standalone accuracy 0.20. Workflow B is nevertheless complementary because it solves Type 2 tasks, which workflow A misses.

Suppose the selector follows the Plackett–Luce recovery curve with $\lambda = 2$:

$$\psi^{\text{PL}}_{k,2}(r) = \frac{2r}{2r+k-r}. \tag{77}$$

For the portfolio $\boldsymbol{m}^{AB}$, exactly one of the two candidates is correct on Types 1 and 2, so

$$J_2(\boldsymbol{m}^{AB}) = 0.40\left(\frac{2}{3}\right) + 0.20\left(\frac{2}{3}\right) = 0.40. \tag{78}$$

Now consider the portfolio $\boldsymbol{m}^{AAB}$. On Type 1 tasks, two of the three candidates are correct, giving recovery probability 4/5. On Type 2 tasks, only workflow B is correct, giving recovery probability 1/2. Therefore,

$$J_2(\boldsymbol{m}^{AAB}) = 0.40\left(\frac{4}{5}\right) + 0.20\left(\frac{1}{2}\right) = 0.42. \tag{79}$$

Thus,

$$J_2(\boldsymbol{m}^{AAB}) = 0.42 > J_2(\boldsymbol{m}^{AB}) = J_2(\boldsymbol{m}^{A}) = 0.40. \tag{80}$$

The additional execution of A creates no new task coverage. Instead, it increases the representation of the stronger workflow on the more common Type 1 tasks, raising recovery from 2/3 to 4/5, while retaining workflow B to cover Type 2 tasks. The gain on Type 1 tasks exceeds the corresponding loss in selector recovery on Type 2 tasks.

The result also survives a small execution cost. If every execution has accuracy-equivalent cost $\kappa = 0.005$, then

$$\begin{aligned} \Pi_2(\boldsymbol{m}^{A}) &= 0.400 - 0.005 = 0.395, \\ \Pi_2(\boldsymbol{m}^{AB}) &= 0.400 - 0.010 = 0.390, \\ \Pi_2(\boldsymbol{m}^{AAB}) &= 0.420 - 0.015 = 0.405. \end{aligned} \tag{81}$$

Hence, $\boldsymbol{m}^{AAB}$ remains optimal among these portfolios after accounting for recurring execution cost. By contrast, repeating A without including another workflow type would leave its selector-aware accuracy unchanged at 0.40 and would only add cost. Multiplicity is valuable here because it adjusts the relative representation of complementary workflow types, not because pure repetition expands coverage.

Appendix D: Proofs

D.1. Proof of Lemma 1

For $p \in (0,1)$, the condition $\Lambda(H) \le \Lambda$ gives

$$\frac{H(p)}{1-H(p)} \le \Lambda \frac{p}{1-p}. \tag{82}$$

Finite odds-lift implies $H(p) < 1$ at every interior p, so all denominators are positive. Multiplying through and collecting terms yields

$$H(p)\{1 - p + \Lambda p\} \le \Lambda p. \tag{83}$$

Because $1 - p + \Lambda p = 1 + (\Lambda - 1)p > 0$,

$$H(p) \le \frac{\Lambda p}{1 + (\Lambda - 1)p} = h_\Lambda(p). \tag{84}$$

At $p = 0$, both sides are zero under $H(0) = 0$; at $p = 1$, the right-hand side is one and the inequality follows from $H(1) \le 1$. $\square$

D.2. Proof of Lemma 2

For $p \in [0,1]$,

$$h'_\Lambda(p) = \frac{\Lambda}{\{1+(\Lambda-1)p\}^2} \geq 0, \qquad h''_\Lambda(p) = -\frac{2\Lambda(\Lambda-1)}{\{1+(\Lambda-1)p\}^3} \leq 0. \tag{85}$$

Thus h_Λ is increasing and concave. □

D.3. Proof of Lemma 3

For continuous r,

$$\frac{d}{dr}\frac{\lambda r}{k+(\lambda-1)r} = \frac{\lambda k}{\{k+(\lambda-1)r\}^2} \geq 0, \tag{86}$$

and

$$\frac{d^2}{dr^2}\frac{\lambda r}{k+(\lambda-1)r} = -\frac{2\lambda k(\lambda-1)}{\{k+(\lambda-1)r\}^3} \leq 0. \tag{87}$$

The discrete increment follows by subtracting adjacent values and simplifying, which yields (31); its denominator increases in ℓ. Finally,

$$h'_\lambda(p) = \frac{\lambda}{\{1+(\lambda-1)p\}^2} > 0, \qquad h''_\lambda(p) = -\frac{2\lambda(\lambda-1)}{\{1+(\lambda-1)p\}^3} \leq 0. \tag{88}$$

□

D.4. Proof of Proposition 1

Consider one task and zero compute costs. Let c_1, c_2 be workflows that are correct on the task and let w_1, w_2 be wrong. Nonmonotonicity follows from

$$J_\lambda(\{c_1\}) = 1, \qquad J_\lambda(\{c_1, w_1\}) = h_\lambda(1/2) < 1. \tag{89}$$

For submodularity, take $A = \{c_1\}$, $B = \{c_1, w_1\}$, and $e = c_2$. Then

$$\Delta(e \mid A) = 0, \qquad \Delta(e \mid B) = h_\lambda(2/3) - h_\lambda(1/2) > 0, \tag{90}$$

which violates diminishing returns. For supermodularity, take $A = \{w_1\}$, $B = \{w_1, w_2\}$, and $e = c_1$. Then

$$\Delta(e \mid A) = h_\lambda(1/2) > h_\lambda(1/3) = \Delta(e \mid B), \tag{91}$$

which violates increasing returns. Adding or subtracting a modular function does not change the submodularity or supermodularity inequalities. □

D.5. Proof of Theorem 2

Fix an execution-count vector $\boldsymbol{m} \in \mathbb{Z}_+^{\mathcal{M}}$ satisfying $k(\boldsymbol{m}) = k$, label its k execution slots, and sample b labeled slots uniformly without replacement. Let $\boldsymbol{M}^{(b)}$ be the resulting random execution-count vector and let $\alpha = b/k$. For task i, write $r_i = r_i(\boldsymbol{m})$ and let

$$R_i = r_i\big(\boldsymbol{M}^{(b)}\big).$$

Then

$$R_i \sim \mathrm{Hypergeom}(k, r_i, b), \qquad \mathbb{E}[R_i] = \alpha r_i.$$

If $r_i = 0$, then both the size-k task value and the expected size-b task value are zero. Suppose $r_i > 0$ and define $p_i = r_i/k$ and $X_i = R_i/b$. Since h is increasing, concave, and satisfies $h(0) = 0$,

$$h(x) \geq h(p_i)\min\{x/p_i, 1\}, \qquad x \in [0,1]. \tag{92}$$

Indeed, for $x \leq p_i$ the inequality follows from concavity between 0 and p_i, and for $x \geq p_i$ it follows from monotonicity. Now

$$\frac{X_i}{p_i} = \frac{R_i}{\alpha r_i}. \tag{93}$$

Let $Z_i = R_i/r_i \in [0,1]$. Then $\mathbb{E}[Z_i] = \alpha$ and, for every $z \in [0,1]$, $\min\{z, \alpha\} \geq \alpha z$. Hence

$$\mathbb{E}\left[\min\left\{\frac{X_i}{p_i}, 1\right\}\right] = \frac{1}{\alpha}\mathbb{E}[\min\{Z_i, \alpha\}] \geq \frac{1}{\alpha}\alpha\mathbb{E}[Z_i] = \alpha. \tag{94}$$

Combining this with (92) gives

$$\mathbb{E}[h(R_i/b)] \geq \alpha h(r_i/k). \tag{95}$$

Averaging across tasks yields

$$\mathbb{E}\big[J_\psi\big(\boldsymbol{M}^{(b)}\big)\big] \geq \alpha J_\psi(\boldsymbol{m}), \tag{96}$$

Because costs are additive,

$$\mathbb{E}\big[C\big(\boldsymbol{M}^{(b)}\big)\big] = \alpha C(\boldsymbol{m}), \tag{97}$$

Therefore

$$\mathbb{E}\big[\Pi_{\psi,\gamma}\big(\boldsymbol{M}^{(b)}\big)\big] \geq \alpha \Pi_{\psi,\gamma}(\boldsymbol{m}). \tag{98}$$

Some realization of $\boldsymbol{M}^{(b)}$ attains at least this expected value. If $\boldsymbol{m}$ is an optimal size-k execution-count vector with positive value, then

$$\mathrm{OPT}_b(\mathcal{M}) \geq \alpha \mathrm{OPT}_k(\mathcal{M}), \tag{99}$$

which proves (34); if $\mathrm{OPT}_k(\mathcal{M}) \leq 0$, the statement with positive parts is immediate. For the grid result, choose for each k an anchor $b \in \mathcal{K}$ with $b \leq k \leq \varrho b$. Then

$$\mathrm{OPT}_b^+(\mathcal{M}) \geq \frac{b}{k}\mathrm{OPT}_k^+(\mathcal{M}) \geq \frac{1}{\varrho}\mathrm{OPT}_k^+(\mathcal{M}). \tag{100}$$

Maximizing over k proves (35). The dyadic and ratio-$1/(1-\eta_{\mathrm{grid}})$ claims follow from the definition of the grid coverage ratio. □

D.6. Proof of Theorem 1

The lower bound follows from the outside option and singleton feasibility. With one candidate, the selector must return that candidate, so the net value of the singleton execution-count vector $\boldsymbol{e}_g$ is $\bar{a}_g - \gamma c_g$.

Fix a nonzero execution-count vector $\boldsymbol{m} \in \mathbb{Z}_+^{\mathcal{M}}$ satisfying $k(\boldsymbol{m}) = k$, and define

$$p_i(\boldsymbol{m}) = \frac{r_i(\boldsymbol{m})}{k}.$$

By Assumption 1 and lemma 1,

$$J_\psi(\boldsymbol{m}) = \frac{1}{n}\sum_{i=1}^{n} \psi_k\big(r_i(\boldsymbol{m})\big) \tag{101}$$

$$\leq \frac{1}{n}\sum_{i=1}^{n} H\big(p_i(\boldsymbol{m})\big) \tag{102}$$

$$\leq \frac{1}{n}\sum_{i=1}^{n} h_\Lambda\big(p_i(\boldsymbol{m})\big). \tag{103}$$

By Lemma 2 and Jensen's inequality,

$$\frac{1}{n}\sum_{i=1}^{n} h_\Lambda\big(p_i(\boldsymbol{m})\big) \leq h_\Lambda\left(\frac{1}{n}\sum_{i=1}^{n} p_i(\boldsymbol{m})\right) \tag{104}$$

$$= h_\Lambda\left(\frac{1}{k}\sum_{g\in\mathcal{M}} \bar{a}_g m_g\right) \tag{105}$$

$$\leq h_\Lambda(A_k). \tag{106}$$

Also $C(\boldsymbol{m}) \geq C_k$. Hence

$$\Pi_{\psi,\gamma}(\boldsymbol{m}) \leq h_\Lambda(A_k) - \gamma C_k.$$

Maximizing over k and including the outside option proves (22). When $\gamma = 0$, $A_k \leq a^\star$ for all k, giving (23).

For tightness over the selector class, take recovery $H = h_\Lambda$. Suppose $a^\star = r/k$ and $k \leq K_{\max}$. Construct k tasks and k workflows using a cyclic incidence matrix in which each workflow solves exactly r tasks and each task is solved by exactly r workflows. Let $\boldsymbol{m}^\star$ assign one execution slot to each of these k workflows. Then $p_i(\boldsymbol{m}^\star) = a^\star$ for every task, and therefore $J_\psi(\boldsymbol{m}^\star) = h_\Lambda(a^\star)$. Rational values can be represented exactly after replication, and arbitrary values can be approximated.

□

D.7. Proof of Corollary 1

The first inequality follows from (23). Let

$$G_\Lambda(a) = h_\Lambda(a) - a = \frac{(\Lambda-1)a(1-a)}{1+(\Lambda-1)a}. \tag{107}$$

For $\Lambda > 1$, differentiation gives the unique maximizer $a = 1/(1+\sqrt{\Lambda})$ and maximum $(\sqrt{\Lambda}-1)/(\sqrt{\Lambda}+1)$. At $\Lambda = 1$, both sides are zero.

If all workflows cost c, any execution-count vector $\boldsymbol{m}$ satisfying $k(\boldsymbol{m}) = k \geq 2$ obeys

$$\Pi_{\psi,\gamma}(\boldsymbol{m}) \leq h_\Lambda(a^\star) - k\gamma c.$$

Under (26),

$$h_\Lambda(a^\star) - k\gamma c \leq a^\star - \gamma c, \tag{108}$$

so no multi-workflow portfolio beats the best singleton. The outside option dominates only when the singleton net value is negative. □

D.8. Proof of Proposition 2

Fix an execution-count vector $\boldsymbol{m} \in \mathbb{Z}_+^{\mathcal{M}}$ satisfying $k(\boldsymbol{m}) = k$, and set

$$z_g = m_g, \qquad g \in \mathcal{M}.$$

Then

$$\sum_{g\in\mathcal{M}} a_{ig} z_g = r_i(\boldsymbol{m}), \qquad i = 1,\dots,n.$$

For this fixed z, the prefix constraints imply that the active tier variables for task i form a prefix, while the capacity constraint implies that the length of this prefix is at most $r_i(\boldsymbol{m})$. Since $d_{\ell,k} \geq 0$, there exists an optimal choice

$$y_{i\ell} = \mathbf{1}\{\ell \leq r_i(\boldsymbol{m})\}, \qquad \ell = 1,\dots,k.$$

Its task-i contribution is

$$\sum_{\ell=1}^{k} d_{\ell,k} y_{i\ell} = \sum_{\ell=1}^{r_i(\boldsymbol{m})} d_{\ell,k} = \psi_k\big(r_i(\boldsymbol{m})\big),$$

where the last equality uses $\psi_k(0) = 0$. Moreover, the cost term is

$$-\gamma \sum_{g\in\mathcal{M}} c_g z_g = -\gamma C(\boldsymbol{m}).$$

Thus every feasible size-k execution-count vector induces an integer-program solution with the same objective value, and hence

$$I_k(\mathcal{M}) \geq \mathrm{OPT}_k(\mathcal{M}).$$

Conversely, let (y,z) be any integral feasible solution of (39), and define the execution-count vector $\boldsymbol{m} \in \mathbb{Z}_+^{\mathcal{M}}$ by $m_g = z_g$. Then $k(\boldsymbol{m}) = k$. For each task i, let

$$t_i = \sum_{\ell=1}^{k} y_{i\ell}.$$

Because y is binary and satisfies the prefix constraints, its active entries form a prefix of length t_i. Feasibility gives

$$t_i \le \sum_{g\in\mathcal{M}} a_{ig} z_g = r_i(\boldsymbol{m}).$$

Therefore, using the nonnegativity of the increments,

$$\sum_{\ell=1}^{k} d_{\ell,k} y_{i\ell} = \sum_{\ell=1}^{t_i} d_{\ell,k} = \psi_k(t_i) \le \psi_k\big(r_i(\boldsymbol{m})\big).$$

The integer-program objective is consequently at most $\Pi_{\psi,\gamma}(\boldsymbol{m})$, and hence at most $\mathrm{OPT}_k(\mathcal{M})$. Thus

$$I_k(\mathcal{M}) \le \mathrm{OPT}_k(\mathcal{M}).$$

Combining the two inequalities gives $I_k(\mathcal{M}) = \mathrm{OPT}_k(\mathcal{M})$. Finally, combining this equality with the size decomposition (33) proves (40). □

D.9. Proof of Theorem 3

Write $d_\ell = d_{\ell,k}$. For a feasible repeated-execution LP vector z, define

$$x_i = \sum_g a_{ig} z_g, \qquad q_g = z_g/k.$$

Draw $G_1,\dots,G_k$ independently from q, and let $R_i = \sum_{j=1}^{k} a_{iG_j}$. Then

$$R_i \sim \mathrm{Binomial}(k, x_i/k), \qquad \mathbb{E}[R_i] = x_i.$$

For every integer $t \ge 1$ and $r \ge 0$,

$$\min\{r,t\} \ge t\left\{1-(1-1/t)^r\right\}.$$

Therefore

$$\begin{aligned}
\mathbb{E}[\min\{R_i,t\}] &\ge t\left[1-\mathbb{E}\{(1-1/t)^{R_i}\}\right] \\
&= t\left[1-\left(1-\frac{x_i}{kt}\right)^k\right] \\
&\ge t\left(1-e^{-x_i/t}\right) \\
&\ge \left(1-\frac{1}{e}\right)\min\{x_i,t\}.
\end{aligned}$$

The last inequality follows from $1-e^{-u} \ge (1-1/e)\min\{u,1\}$ for $u \ge 0$. Using the decomposition

$$\widetilde{\psi}_k(r) = d_{k,k} r + \sum_{t=1}^{k-1} (d_{t,k} - d_{t+1,k}) \min\{r,t\},$$

and averaging across tasks gives

$$\mathbb{E}\big[J_\psi\big(\boldsymbol{M}_k^{\mathrm{RR}}(z)\big)\big] \geq \left(1-\frac{1}{e}\right) Q_k(z) + \frac{d_{k,k}}{e} R_k(z).$$

Moreover, if N_g is the sampled multiplicity of type g, then

$$\mathbb{E}\big[C\big(\boldsymbol{M}_k^{\mathrm{RR}}(z)\big)\big] = \sum_g c_g \mathbb{E}\big[M_{k,g}^{\mathrm{RR}}(z)\big] = \sum_g c_g z_g = C_k(z).$$

Subtracting cost proves (46) and (47). If $z = z^{k\star}$ is LP-optimal, then $\mathrm{OPT}_k(\mathcal{M}) \leq L_k(\mathcal{M}) = F_k(z^{k\star})$, which gives the first inequality in (48). Finally, concavity implies $Q_k(z) \leq d_{1,k} R_k(z)$, so

$$d_{k,k} R_k(z) = (1 - c_{\psi,k}) d_{1,k} R_k(z) \geq (1 - c_{\psi,k}) Q_k(z).$$

This proves the curvature bound. □

D.10. Proof of Corollary 2

By Theorem 3,

$$\mathbb{E}\big[\Pi_{\lambda,\gamma}(\boldsymbol{M}_k^{\mathrm{RR}})\big] \geq L_k(\mathcal{M}) - \frac{c_{k,\lambda}^{\mathrm{PL}}}{e} Q_k^\star.$$

Since the LP relaxation has $0 \leq y_{i\ell} \leq 1$ and $\sum_{\ell=1}^{k} d_{\ell,k} = 1$, we have $0 \leq Q_k^\star \leq 1$. Hence

$$\mathbb{E}\big[\Pi_{\lambda,\gamma}(\boldsymbol{M}_k^{\mathrm{RR}})\big] \geq L_k(\mathcal{M}) - \frac{c_{k,\lambda}^{\mathrm{PL}}}{e},$$

This establishes the fixed-size additive net-value bound used below.

For the accuracy-only bound, Theorem 3 gives

$$\mathbb{E}\big[J_\lambda(\boldsymbol{M}_k^{\mathrm{RR}})\big] \geq \left(1-\frac{1}{e}\right) Q_k^\star + \frac{d_{k,k}}{e} R_k^\star.$$

By the definition of curvature,

$$d_{k,k} = \big(1 - c_{k,\lambda}^{\mathrm{PL}}\big)\, d_{1,k}.$$

Concavity implies $Q_k^\star \leq d_{1,k} R_k^\star$, so

$$d_{k,k} R_k^\star \geq \big(1 - c_{k,\lambda}^{\mathrm{PL}}\big)\, Q_k^\star.$$

Substituting this inequality yields

$$\mathbb{E}\big[J_\lambda(\boldsymbol{M}_k^{\mathrm{RR}})\big] \geq \left(1-\frac{1}{e}\right) Q_k^\star + \frac{1 - c_{k,\lambda}^{\mathrm{PL}}}{e} Q_k^\star = \left(1 - \frac{c_{k,\lambda}^{\mathrm{PL}}}{e}\right) Q_k^\star,$$

This establishes an auxiliary accuracy-only bound.

It remains to prove the grid-level multiplicative bound. Let

$$U = U_{\mathrm{LP}}(\mathcal{K}; \mathcal{M}), \qquad a = \frac{1}{e} \max_{k \in \mathcal{K}} c_{k,\lambda}^{\mathrm{PL}}.$$

If $U=0$, the result is immediate. Otherwise, choose $k^\star \in \arg\max_{k\in\mathcal{K}} L_k(\mathcal{M})$, so that $L_{k^\star}(\mathcal{M})=U$. By the fixed-size additive net-value bound derived above,

$$\mathbb{E}\big[\Pi_{\lambda,\gamma}(S^{\mathrm{RR}}_{k^\star})\big] \ge U-a.$$

Because $\widehat{\boldsymbol{M}}$ is chosen after comparing the rounded portfolios with the baseline $\boldsymbol{m}^0$ and the outside option, Jensen's inequality for the convex maximum function gives

$$\mathbb{E}\Big[\Pi_{\lambda,\gamma}(\widehat{\boldsymbol{M}})\Big] \ge \max\{\underline{V}, U-a\}.$$

For every $U\ge 0$, $a\ge 0$, and $\underline{V}>0$,

$$\max\{\underline{V}, U-a\} \ge \frac{\underline{V}}{\underline{V}+a}U.$$

Indeed, if $U\le \underline{V}+a$, then

$$\max\{\underline{V}, U-a\} \ge \underline{V} \ge \frac{\underline{V}}{\underline{V}+a}U.$$

If $U>\underline{V}+a$, then

$$\max\{\underline{V}, U-a\} \ge U-a \ge \frac{\underline{V}}{\underline{V}+a}U.$$

Combining the last two displays proves (51). Finally, exact integer-program solves over $\mathcal{M}$ dominate the rounded feasible portfolios, so the same lower bound also applies when the final restricted problems are solved exactly. □

D.11. Projected dual, weak separation, and proof details

For $x\in[0,k]$, let

$$\varphi_k(x)=\frac{1}{n}\widetilde{\psi}_k(x),$$

and define

$$\chi_k(u)=\max_{r=0,\dots,k}\left\{\frac{\psi_k(r)}{n}-ur\right\}, \qquad 0\le u\le \frac{d_{1,k}}{n}. \tag{109}$$

Because φ_k is concave and piecewise linear with slopes in $[0,d_{1,k}/n]$,

$$\varphi_k(x)=\min_{0\le u\le d_{1,k}/n}\{ux+\chi_k(u)\}. \tag{110}$$

Lemma 4 **(Projected dual for repeated executions)**. *For every nonempty finite workflow-type class $\mathcal{G}$ and fixed size k,*

$$L_k(\mathcal{G})=\min_{0\le\mu_i\le d_{1,k}/n}\left\{\sum_{i=1}^{n}\chi_k(\mu_i)+k\max_{g\in\mathcal{G}}\left(\sum_{i=1}^{n}\mu_i a_{ig}-\gamma c_g\right)\right\}. \tag{111}$$

Equivalently, introducing θ and epigraph variables ξ_i yields (52).

A fter optimizing the tier variables, the repeated-execution LP is

$$\max_{z\geq 0:\,\sum_g z_g=k}\left\{\sum_{i=1}^{n}\varphi_k\left(\sum_g a_{ig}z_g\right)-\gamma\sum_g c_g z_g\right\}.$$

Substitute (110) and apply finite- dimensional minimax to interchange the minimization over μ and the maximization over the scaled simplex. For fixed μ, linear optimization over that simplex gives

$$\max_{z\geq 0:\,\sum_g z_g=k}\sum_g z_g\left(\sum_i \mu_i a_{ig}-\gamma c_g\right)=k\max_{g\in\mathcal{G}}\left(\sum_i \mu_i a_{ig}-\gamma c_g\right).$$

This proves (111). The epigraph representation of χ_k is

$$\xi_i+r\mu_i\geq\frac{\psi_k(r)}{n},\qquad r=0,\ldots,k,$$

which gives (52). □

LEMMA 5 **(Batched pricing gives high-confidence weak separation)**. *Fix a candidate dual point (μ,ξ,θ) and a tolerance $\delta>0$. Suppose each primitive pricing call made at task prices μ returns a δ-accurate workflow, meaning a workflow $\widetilde{g}$ satisfying*

$$s_{\widetilde{g}}(\mu)\geq\max_{g\in\mathcal{G}}s_g(\mu)-\delta,\qquad s_g(\mu)=\sum_i\mu_i a_{ig}-\gamma c_g,$$

with conditional probability at least $p_{\mathrm{orc}}>0$. Run m primitive calls at the same prices and keep the highest-scoring returned workflow. Then, conditional on the history before the batch, the batch is δ-accurate with probability at least

$$1-e^{-p_{\mathrm{orc}}m}.$$

On this event, if the best returned workflow has score above θ, its workflow inequality is a valid violated constraint. If its score is at most θ, then all workflow inequalities are satisfied after replacing θ by $\theta+\delta$, increasing the dual objective by at most $k\delta$.

L et Y_j be the indicator that primitive call j in the batch is δ-accurate. The calls may be adaptive to previous failed attempts inside the batch, but by assumption

$$\mathbb{P}\{Y_j=1\,|\,\text{past within the batch}\}\geq p_{\mathrm{orc}}.$$

Therefore

$$\mathbb{P}\{Y_1=\cdots=Y_m=0\,|\,\text{history}\}\leq(1-p_{\mathrm{orc}})^m\leq e^{-p_{\mathrm{orc}}m}.$$

Thus, with probability at least $1-e^{-p_{\mathrm{orc}}m}$, at least one primitive call is δ-accurate. Since the batch keeps the highest-scoring returned workflow, the batch output is also δ-accurate.

If the batch output satisfies $s_{\tilde{g}}(\mu) > \theta$, then the constraint

$$s_{\tilde{g}}(\mu) \le \theta$$

is violated and supplies a valid separating hyperplane. If instead $s_{\tilde{g}}(\mu) \le \theta$, then δ-accuracy gives

$$\max_{g \in \mathcal{G}} s_g(\mu) \le s_{\tilde{g}}(\mu) + \delta \le \theta + \delta.$$

Thus $s_g(\mu) \le \theta + \delta$ for every $g \in \mathcal{G}$. Since the coefficient of θ in (52) is k, this relaxation increases the dual objective by at most $k\delta$. □

D.12. Proof of Theorem 4

For each fixed k, (52) is a rational LP in $2n+1$ variables, with polynomially many explicit recovery and box constraints and an implicit family of workflow constraints. By Lemma 5, a successful batch supplies weak separation for the implicit workflow constraints. The weak optimization–separation equivalence for the ellipsoid method therefore returns an $\varepsilon_{\text{ell}}/2$-optimal dual solution in a number of separation queries polynomial in

$$n, \quad k, \quad B, \quad \log(1/\varepsilon_{\text{ell}}).$$

With $\tau_k = \varepsilon_{\text{ell}}/(2k)$, the possible slot-price relaxation in Lemma 5 contributes at most $k\tau_k = \varepsilon_{\text{ell}}/2$ to the dual objective. Hence, on the event that all batches used by the ellipsoid routine are successful, standard primal recovery from the workflow inequalities generated by the routine gives, for every $k \in \mathcal{K}$, a feasible fractional execution vector z^k supported only on oracle-returned workflows and satisfying

$$F_k(z^k) \ge L_k(\mathcal{G}) - \varepsilon_{\text{ell}}. \tag{112}$$

It remains to bound the probability that all batches are successful. Let $N_{\text{ell}}(\varepsilon_{\text{ell}})$ be the deterministic upper bound on the total number of ellipsoid separation queries across all grid sizes. Index these separation queries by $q = 1, \dots, N_{\text{ell}}$. Let $\mathcal{B}_q$ be the event that batch q contains at least one τ_k-accurate primitive pricing call, where k is the grid size being solved at that query. By Lemma 5,

$$\mathbb{P}(\mathcal{B}_q^c \,|\, \text{history before batch } q) \le e^{-p_{\text{orc}} m}.$$

Therefore,

$$\mathbb{P}\left(\bigcup_{q=1}^{N_{\text{ell}}} \mathcal{B}_q^c\right) \le \sum_{q=1}^{N_{\text{ell}}} \mathbb{E}\big[\mathbb{P}(\mathcal{B}_q^c \,|\, \text{history before batch } q)\big] \tag{113}$$

$$\le N_{\text{ell}}(\varepsilon_{\text{ell}}) e^{-p_{\text{orc}} m}. \tag{114}$$

Equivalently, since $T = mN_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})$,

$$N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})e^{-p_{\mathrm{orc}}m} = \exp\left\{-\frac{p_{\mathrm{orc}}T}{N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})} + \log N_{\mathrm{ell}}(\varepsilon_{\mathrm{ell}})\right\}.$$

This proves the probability statement in (54).

On the event that all batches are successful, apply Theorem 3 to each recovered fractional solution z^k. Since $Q_k(z^k) \le 1$, for every $k \in \mathcal{K}$,

$$\mathbb{E}_{\mathrm{rnd}}\left[\Pi_{\lambda,\gamma}(\boldsymbol{M}_k^{\mathrm{RR}})\right] \ge F_k(z^k) - \frac{c_{k,\lambda}^{\mathrm{PL}}}{e} \ge L_k(\mathcal{G}) - \varepsilon_{\mathrm{ell}} - \frac{c_{k,\lambda}^{\mathrm{PL}}}{e}.$$

Taking the best grid point gives

$$\max_{k\in\mathcal{K}} \mathbb{E}_{\mathrm{rnd}}\left[\Pi_{\lambda,\gamma}(\boldsymbol{M}_k^{\mathrm{RR}})\right] \ge U_{\mathrm{LP}}(\mathcal{K};\mathcal{G}) - \varepsilon_{\mathrm{ell}} - \varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda).$$

By Theorem 2,

$$U_{\mathrm{LP}}(\mathcal{K};\mathcal{G}) \ge \frac{1}{\varrho}\mathrm{OPT}_{\lambda,\gamma}(\mathcal{G}).$$

Comparing the rounded grid candidates with the retained baseline $\boldsymbol{m}^0$ proves (55).

Finally, for every $A \ge 0$, $e \ge 0$, and $\underline{V} > 0$,

$$\max\{\underline{V}, A - e\} \ge \frac{\underline{V}}{\underline{V} + e} A.$$

Apply this inequality with

$$A = \frac{1}{\varrho}\mathrm{OPT}_{\lambda,\gamma}(\mathcal{G}), \qquad e = \varepsilon_{\mathrm{ell}} + \varepsilon_{\mathrm{rnd}}(\mathcal{K},\lambda),$$

to obtain (56). □

Appendix E: Managerial Implications and Extensions

Accuracy value and the price of compute. The parameter γ is not an arbitrary regularizer. If a correct decision is worth v dollars relative to an incorrect one, maximizing $vJ_\psi(\boldsymbol{m}) - C(\boldsymbol{m})$ is equivalent to maximizing $J_\psi(\boldsymbol{m}) - \gamma C(\boldsymbol{m})$ with $\gamma = 1/v$. Varying γ traces an accuracy–compute frontier. The screening bound in Theorem 1 can eliminate run sizes that cannot be efficient before solving any IP.

Selector investment versus workflow investment. The selector-strength cap (25) maps an odds-lift bound Λ into the largest possible gross return from workflow complementarity. When Λ is close to one, spending on additional workflows has little possible upside; improving the selector may dominate expanding the portfolio. When Λ is large, workflow variety can become more valuable, subject to cost. Under Plackett–Luce, $\Lambda = \lambda$, so selector-model investment can be evaluated through the fitted discrimination parameter.

Shared modules, latency, and nonadditive costs. The additive cost $C(\boldsymbol{m}) = \sum_{g \in \mathcal{G}} c_g m_g$ is appropriate for token or dollar expenditure when all assigned workflow executions are run. Workflow graphs may share retrieval results, cached model calls, or verification modules, and parallel execution makes latency depend on a critical path rather than a sum. These features can be represented by fixed-charge module variables or a set-dependent cost $C(S)$. The structural accuracy bound remains valid, while the finite-pool optimization becomes a richer mixed-integer problem.

Stochastic workflow outputs. If correctness is stochastic, the exact selector-aware contribution of task i under execution-count vector $\boldsymbol{m}$ is

$$\mathbb{E}\left[\psi_{k(\boldsymbol{m})}\big(R_i(\boldsymbol{m})\big)\right].$$

where the expectation is over the joint distribution of workflow outcomes. In general this is not equal to applying ψ to the sum of marginal success probabilities. A practical approach is scenario-based sample-average approximation: repeated workflow runs create deterministic correctness scenarios, and the portfolio is optimized against their average selector value (Kleywegt et al. 2002).

Task-dependent run sets. The paper chooses one persistent portfolio for a task population. A pre-output router could select a task-specific subset before execution, yielding a joint routing and post-output selection problem. The current model provides the value and cost of each candidate run set; an outer policy can then allocate run sets by task features or congestion state.

Voting and nonconcave selectors. Majority voting and threshold verification can have increasing marginal value near a decision threshold. The general selector-strength bound still applies if a finite common odds-lift envelope exists, even when the raw recovery curve is nonconcave. If recovery reaches one at an interior correct fraction, however, the odds-lift index is infinite and that uniform cap is vacuous. For each fixed size, the exact IP remains valid with arbitrary nondecreasing increments, but the concave-coverage rounding certificate may fail. Endogenous size still matters because adding a wrong vote can move the system away from the threshold and incurs compute. Selector-specific approximation or robust envelope methods are natural extensions.

Creation cost versus execution cost. Execution cost c_g recurs on every deployed task. Workflow creation and evaluation costs are one-time investments. The generation algorithm can impose a separate budget on oracle calls, candidate evaluation, or human engineering. A longer planning horizon increases the relative importance of recurring execution cost, while a short-lived application may place more weight on creation cost.

Robust selector calibration. The fitted λ may vary across task types or drift when portfolios are optimized. For the structural cap, a robust analysis can construct an upper confidence envelope H^+ for recovery and use $\Lambda(H^+)$, or use an upper confidence bound for λ under Plackett–Luce. For

portfolio optimization, one can optimize worst-case net value over a family of recovery curves or impose a distribution over task-specific selector strengths. The cardinality-grid and IP architecture remains unchanged; only the recovery coefficients $d_{\ell,k}$ vary. If the recovery curves do not share a common concave fraction-based representation, the exact fixed-size IPs remain valid but the geometric cardinality approximation should be treated as a Plackett–Luce or common-fraction specialization rather than a universal guarantee.

Appendix F: Proofs for Stochastic Workflow Outcomes

This appendix gives the complete two-stage stochastic procedure and its proof. Stage 1 uses L_1 independent executions to estimate each task–workflow success probability and constructs a plug-in iid execution law. Workflow generation, stochastic pricing, and primal recovery are performed under that plug-in law. After the generated workflow pool is frozen, Stage 2 discards the Stage-1 outcomes for selection purposes and uses L_2 fresh execution panels to choose the final portfolio.

For the analysis, associate with every task–workflow pair (i,g) a potential Stage-1 pilot panel

$$\left(Z_{ig}^{(1,\ell)} : \ell = 1,\dots,L_1\right), \qquad Z_{ig}^{(1,\ell)} \overset{\text{iid}}{\sim} \text{Bernoulli}(a_{ig}), \tag{115}$$

with all panels mutually independent. The algorithm reveals a panel only when the corresponding workflow is first evaluated. This lazy revelation is equivalent to drawing the entire finite table in advance and does not require enumerating $\mathcal{G}$ computationally.

LEMMA 6 **(Uniform concentration of the L_1 workflow estimates).** *Under Assumption 3, for every $\delta_1 \in (0,1)$,*

$$\mathbb{P}\left\{ \max_{\substack{i=1,\dots,n \\ g\in\mathcal{G}}} \left|\widehat{a}_{ig}^{(1)} - a_{ig}\right| \le \varepsilon_a(L_1,\delta_1) \right\} \ge 1-\delta_1, \tag{116}$$

where $\varepsilon_a(L_1,\delta_1)$ is defined in (65).

F or each fixed (i,g), Hoeffding's inequality gives

$$\mathbb{P}\left\{ \left|\widehat{a}_{ig}^{(1)} - a_{ig}\right| > t \right\} \le 2e^{-2L_1 t^2}.$$

A union bound over the $n|\mathcal{G}|$ potential pilot panels and the choice $t = \varepsilon_a(L_1,\delta_1)$ prove the result. Since the complete pilot table can be regarded as drawn before the generation process begins, revealing its entries adaptively does not alter the bound. □

LEMMA 7 **(Perturbation of iid stochastic portfolio values).** *Let $b = (b_{ig})$ and $b' = (b'_{ig})$ be two success-probability matrices such that $\max_{i,g} |b_{ig} - b'_{ig}| \le \eta$. Let $\Pi^{b}_{\psi,\gamma}$ and $\Pi^{b'}_{\psi,\gamma}$ denote portfolio values under the corresponding product-Bernoulli execution laws. Then*

$$\sup_{\substack{\boldsymbol{m}\in\mathbb{Z}_+^{\mathcal{G}} \\ k(\boldsymbol{m})\le K_{\max}}} \left|\Pi^{b}_{\psi,\gamma}(\boldsymbol{m}) - \Pi^{b'}_{\psi,\gamma}(\boldsymbol{m})\right| \le \min\{1, K_{\max}\eta\}. \tag{117}$$

Consequently, on the event in (116),

$$\sup_{\substack{\boldsymbol{m}\in\mathbb{Z}_+^{\mathcal{G}}\\ k(\boldsymbol{m})\le K_{\max}}} \left|\widehat{\Pi}^{(1)}_{\psi,\gamma}(\boldsymbol{m}) - \Pi^{\mathrm{stoch}}_{\psi,\gamma}(\boldsymbol{m})\right| \le \varepsilon_{\mathrm{est}}(L_1,\delta_1). \tag{118}$$

F ix $\boldsymbol{m}$ and couple every Bernoulli execution under b and b' using the same independent uniform random variable. On task i, one execution of workflow g differs under the two laws with probability $|b_{ig} - b'_{ig}|$. Hence the probability that any of the $k(\boldsymbol{m})$ coupled execution outcomes differs is at most

$$\sum_{g\in\mathcal{G}} m_g |b_{ig} - b'_{ig}| \le k(\boldsymbol{m})\eta.$$

If no coupled outcome differs, the two correct counts and therefore the two recovery values coincide. Since recovery lies in $[0,1]$, the difference in expected task value is at most $\min\{1, k(\boldsymbol{m})\eta\}$. Averaging over tasks and observing that the cost term is identical under the two laws prove (117). The second claim follows from Lemma 6. □

For a fixed run size k, label the execution copies by $q = 1,\dots,k$. The Stage-1 estimates are integer multiples of $1/L_1$. This permits a common finite scenario representation of the entire plug-in law that does not change when a new workflow is revealed. Let

$$\Omega_k = \{1,\dots,L_1\}^{n\times k}, \qquad p_\omega = L_1^{-nk}, \tag{119}$$

and write $U_{iq}(\omega) \in \{1,\dots,L_1\}$ for coordinate (i,q) of scenario ω. For every workflow g, define its potential plug-in correctness outcome in scenario ω by

$$\widehat{A}^{(1)}_{igq}(\omega) = \mathbf{1}\left\{U_{iq}(\omega) \le L_1 \widehat{a}^{(1)}_{ig}\right\}. \tag{120}$$

For each fixed integral assignment of workflows to execution copies, the variables in (120) are independent across tasks and copies and have the required Bernoulli means $\widehat{a}^{(1)}_{ig}$. The same uniforms may be used for different workflow choices within one copy because an integral assignment selects only one workflow for that copy.

For a finite workflow class $\mathcal{M}$, define

$$\mathcal{C}_k(\mathcal{M}) = \left\{\boldsymbol{m}\in\mathbb{Z}_+^{\mathcal{M}} : \sum_{g\in\mathcal{M}} m_g = k\right\}, \qquad |\mathcal{C}_k(\mathcal{M})| = \binom{|\mathcal{M}|+k-1}{k}, \tag{121}$$

and let

$$\widehat{\mathrm{OPT}}^{(1)}_k(\mathcal{M}) = \max_{\boldsymbol{m}\in\mathcal{C}_k(\mathcal{M})} \widehat{\Pi}^{(1)}_{\psi,\gamma}(\boldsymbol{m}). \tag{122}$$

LEMMA 8 **(Plug-in stochastic LP relaxation and projected dual)**. *Fix k and suppose $d_{1,k} \geq \cdots \geq d_{k,k} \geq 0$. Let $\widetilde{\psi}_k$ be defined by (38), and set*

$$\mathcal{X}_k(\mathcal{G}) = \left\{ x \geq 0 : \sum_{g \in \mathcal{G}} x_{gq} = 1,\ q = 1, \ldots, k \right\}. \tag{123}$$

For $x \in \mathcal{X}_k(\mathcal{G})$, define

$$\widehat{s}^{(1)}_{i\omega}(x) := \sum_{q=1}^{k} \sum_{g \in \mathcal{G}} \widehat{A}^{(1)}_{igq}(\omega) x_{gq}, \tag{124}$$

$$\widehat{\mathcal{F}}^{(1)}_k(x) := \sum_{\omega \in \Omega_k} p_\omega \frac{1}{n} \sum_{i=1}^{n} \widetilde{\psi}_k\big(\widehat{s}^{(1)}_{i\omega}(x)\big) - \gamma \sum_{q=1}^{k} \sum_{g \in \mathcal{G}} c_g x_{gq}, \tag{125}$$

$$\widehat{\mathcal{L}}^{(1)}_k(\mathcal{G}) := \max_{x \in \mathcal{X}_k(\mathcal{G})} \widehat{\mathcal{F}}^{(1)}_k(x). \tag{126}$$

Then

$$\widehat{\mathcal{L}}^{(1)}_k(\mathcal{G}) \geq \widehat{\mathrm{OPT}}^{(1)}_k(\mathcal{G}). \tag{127}$$

Define

$$\chi_k(u) = \max_{r=0,\ldots,k} \left\{ \frac{\psi_k(r)}{n} - ur \right\}, \qquad 0 \leq u \leq \frac{d_{1,k}}{n}. \tag{128}$$

Strong duality gives

$$\begin{aligned} \widehat{\mathcal{L}}^{(1)}_k(\mathcal{G}) = \min_{\mu,\theta} \quad & \sum_{\omega \in \Omega_k} p_\omega \sum_{i=1}^{n} \chi_k(\mu_{i\omega}) + \sum_{q=1}^{k} \theta_q \\ \text{s.t.} \quad & \sum_{\omega \in \Omega_k} p_\omega \sum_{i=1}^{n} \mu_{i\omega} \widehat{A}^{(1)}_{igq}(\omega) - \gamma c_g \leq \theta_q, && g \in \mathcal{G},\ q = 1, \ldots, k, \\ & 0 \leq \mu_{i\omega} \leq \frac{d_{1,k}}{n}, && i = 1, \ldots, n,\ \omega \in \Omega_k. \end{aligned} \tag{129}$$

Equivalently, introducing epigraph variables $\xi_{i\omega}$ gives the rational LP

$$\begin{aligned} \widehat{\mathcal{L}}^{(1)}_k(\mathcal{G}) = \min_{\mu,\xi,\theta} \quad & \sum_{\omega \in \Omega_k} p_\omega \sum_{i=1}^{n} \xi_{i\omega} + \sum_{q=1}^{k} \theta_q \\ \text{s.t.} \quad & \xi_{i\omega} + r\mu_{i\omega} \geq \frac{\psi_k(r)}{n}, && i = 1, \ldots, n,\ \omega \in \Omega_k,\ r = 0, \ldots, k, \\ & \sum_{\omega \in \Omega_k} p_\omega \sum_{i=1}^{n} \mu_{i\omega} \widehat{A}^{(1)}_{igq}(\omega) - \gamma c_g \leq \theta_q, && g \in \mathcal{G},\ q = 1, \ldots, k, \\ & 0 \leq \mu_{i\omega} \leq \frac{d_{1,k}}{n}, \quad 0 \leq \xi_{i\omega} \leq \frac{1}{n}, && i = 1, \ldots, n,\ \omega \in \Omega_k, \\ & -\gamma c_{\max} \leq \theta_q \leq d_{1,k}, && q = 1, \ldots, k. \end{aligned} \tag{130}$$

The plug-in stochastic pricing score is therefore

$$\widehat{\mathcal{S}}^{(1)}_{kq}(g; \mu) = \sum_{\omega \in \Omega_k} p_\omega \sum_{i=1}^{n} \mu_{i\omega} \widehat{A}^{(1)}_{igq}(\omega) - \gamma c_g. \tag{131}$$

A n integral x assigns one workflow to every execution copy q and induces counts $m_g = \sum_q x_{gq}$. Its scenario-wise correct counts are integral, so $\widetilde{\psi}_k = \psi_k$ at those counts. By the construction in (120), its scenario distribution is exactly the Stage-1 plug-in iid law. Thus its objective is $\widehat{\Pi}^{(1)}_{\psi,\gamma}(\boldsymbol{m})$, and relaxing integrality proves (127).

Let $\varphi_k(t) = \widetilde{\psi}_k(t)/n$. Concavity gives

$$\varphi_k(t) = \min_{0 \le u \le d_{1,k}/n} \{ut + \chi_k(u)\}, \qquad 0 \le t \le k. \tag{132}$$

Substitute (132) into (125) and apply finite-dimensional minimax to interchange maximization over x and minimization over μ. For fixed μ, maximization separates by execution copy:

$$\begin{aligned} \max_{x \in \mathcal{X}_k(\mathcal{G})} \sum_{q=1}^{k} \sum_{g \in \mathcal{G}} x_{gq} \widehat{\mathcal{S}}^{(1)}_{kq}(g;\mu) \\ = \sum_{q=1}^{k} \max_{g \in \mathcal{G}} \widehat{\mathcal{S}}^{(1)}_{kq}(g;\mu). \end{aligned} \tag{133}$$

Introducing the variables θ_q yields (129); replacing each χ_k by its finite epigraph representation yields (130). □

Even under iid execution, the score in (131) is not generally equal to the deterministic mean-column score $\sum_i \bar{\mu}_i \widehat{a}^{(1)}_{ig} - \gamma c_g$. The scenario-dependent marginal price $\mu_{i\omega}$ and the candidate's correctness indicator are evaluated in the same execution scenario, so averaging them separately need not preserve their product.

At a query (k, q, μ), a primitive plug-in pricing call is τ_k-accurate if it returns $\widetilde{g}$ satisfying

$$\widehat{\mathcal{S}}^{(1)}_{kq}(\widetilde{g};\mu) \ge \max_{g \in \mathcal{G}} \widehat{\mathcal{S}}^{(1)}_{kq}(g;\mu) - \tau_k. \tag{134}$$

The guarantee is defined relative to the complete potential Stage-1 pilot table in (115); only the panels of workflows actually proposed by the oracle need to be materialized.

LEMMA 9 **(Batched plug-in stochastic pricing)**. *Suppose each primitive pricing call satisfies (134), conditional on the complete query history, with probability at least $p_{\mathrm{orc}} > 0$. If each pricing query uses m primitive calls and retains the highest-scoring returned workflow, let $\mathcal{E}_{\mathrm{orc}}$ be the event that every resulting batch is τ_k-accurate. If at most $N^{\mathrm{stoch}}_{\mathrm{price}}$ batches are used, then*

$$\mathbb{P}(\mathcal{E}_{\mathrm{orc}}) \ge 1 - N^{\mathrm{stoch}}_{\mathrm{price}} e^{-p_{\mathrm{orc}} m}. \tag{135}$$

A batch fails only if all m primitive calls fail, which has conditional probability at most $(1 - p_{\mathrm{orc}})^m \le e^{-p_{\mathrm{orc}} m}$. A union bound over the pricing batches proves the claim. □

Algorithm 2 IID Stochastic Dual-Guided Workflow Optimization

Input: initial workflow pool $\mathcal{M}_0$; cardinality grid $\mathcal{K}$; recovery curves $\{\psi_k\}$; cost price γ; Stage-1 sample size L_1 and confidence level δ_1; Stage-2 sample size L_2 and confidence level δ_2; ellipsoid tolerance $\varepsilon_{\text{ell}} > 0$; pricing-batch size m; primitive stochastic pricing oracle; baseline $\boldsymbol{m}^0$ with certified value $\underline{V} > 0$ and $\text{supp}(\boldsymbol{m}^0) \subseteq \mathcal{M}_0$.
For every $g \in \mathcal{M}_0$ and task i, collect L_1 independent executions and compute $\widehat{a}_{ig}^{(1)}$ using (64). Set $\mathcal{M}_{\text{rec}} \leftarrow \mathcal{M}_0$.
for $k \in \mathcal{K}$ **do**
  Set $\tau_k \leftarrow \varepsilon_{\text{ell}}/(2k)$.
  Run the ellipsoid method on the plug-in epigraph dual (130) with optimization tolerance $\varepsilon_{\text{ell}}/2$.
  while the ellipsoid routine has not terminated **do**
    At the current point (μ^t, ξ^t, θ^t), separate violated explicit recovery and box constraints directly.
    if no explicit constraint is violated **then**
      for $q = 1, \ldots, k$ **do**
        Make m primitive pricing calls at (k, q, μ^t, τ_k). Whenever a call proposes a workflow g that has not been evaluated, collect its independent L_1-sample panel, compute $(\widehat{a}_{1g}^{(1)}, \ldots, \widehat{a}_{ng}^{(1)})$, and set $\mathcal{M}_{\text{rec}} \leftarrow \mathcal{M}_{\text{rec}} \cup \{g\}$ before scoring it.
        Retain the proposed workflow g^{tq} with the largest score $\widehat{\mathcal{S}}_{kq}^{(1)}(g; \mu^t)$.
        if $\widehat{\mathcal{S}}_{kq}^{(1)}(g^{tq}; \mu^t) > \theta_q^t$ **then**
          Add its violated workflow constraint to the ellipsoid system.
        else
          Use the weak-separation certificate $\max_{g \in \mathcal{G}} \widehat{\mathcal{S}}_{kq}^{(1)}(g; \mu^t) \le \theta_q^t + \tau_k$.
        end if
      end for
    end if
  end while
  By primal recovery, construct $x^k \in \mathcal{X}_k(\mathcal{G})$ supported on $\mathcal{M}_{\text{rec}}$ such that $\widehat{\mathcal{F}}_k^{(1)}(x^k) \ge \widehat{\mathcal{L}}_k^{(1)}(\mathcal{G}) - \varepsilon_{\text{ell}}$.
end for
Freeze the generated pool $\mathcal{M}_T \leftarrow \mathcal{M}_{\text{rec}}$. Do not reuse Stage-1 execution outcomes for final portfolio selection.
Draw the L_2 independent Stage-2 execution panels $\{Z_{igq}^{(2,\ell)}\}$, compute $\widehat{\Pi}_{L_2}^{\text{final}}$ for every $\boldsymbol{m} \in \mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)$, and choose the empirical maximizer $\widetilde{\boldsymbol{m}}$.
Return $\widehat{\boldsymbol{m}}$ according to the conservative rule (71).

LEMMA 10 **(Ellipsoid recovery under plug-in iid pricing)**. *For each $k \in \mathcal{K}$, run the plug-in stochastic dual with ellipsoid optimization tolerance $\varepsilon_{\text{ell},k}/2$ and pricing tolerance $\tau_k = \varepsilon_{\text{ell},k}/(2k)$. On $\mathcal{E}_{\text{orc}}$, primal recovery returns an $x^k \in \mathcal{X}_k(\mathcal{G})$ supported on the returned workflow pool $\mathcal{M}_T$ such*

that

$$\widehat{\mathcal{F}}_k^{(1)}(x^k) \geq \widehat{\mathcal{L}}_k^{(1)}(\mathcal{G}) - \varepsilon_{\mathrm{ell},k}. \tag{136}$$

Successful plug-in pricing supplies weak separation. If the best returned score for copy q is at most θ_q, then (134) implies that all copy-q workflow constraints hold after replacing θ_q by $\theta_q + \tau_k$. Across all k copies, this increases the dual objective by at most $k\tau_k = \varepsilon_{\mathrm{ell},k}/2$. Ellipsoid optimization contributes the remaining half. Standard optimization–separation and primal recovery give (136). □

LEMMA 11 **(Rounding the plug-in stochastic LP).** *Fix k and $x \in \mathcal{X}_k(\mathcal{G})$. Independently for every copy q, draw G_q with $\mathbb{P}\{G_q = g\} = x_{gq}$ and define*

$$M_{k,g}^{\mathrm{RR}}(x) = \sum_{q=1}^{k} \mathbf{1}\{G_q = g\}, \qquad \boldsymbol{M}_k^{\mathrm{RR}}(x) = \big(M_{k,g}^{\mathrm{RR}}(x) : g \in \mathcal{G}\big).$$

Let

$$\widehat{Q}_k^{(1)}(x) := \sum_{\omega \in \Omega_k} p_\omega \frac{1}{n} \sum_{i=1}^{n} \widetilde{\psi}_k\big(\widehat{s}_{i\omega}^{(1)}(x)\big), \tag{137}$$

$$\widehat{R}_k^{(1)}(x) := \sum_{\omega \in \Omega_k} p_\omega \frac{1}{n} \sum_{i=1}^{n} \widehat{s}_{i\omega}^{(1)}(x), \qquad C_k(x) := \sum_{q=1}^{k} \sum_{g \in \mathcal{G}} c_g x_{gq}. \tag{138}$$

Then

$$\mathbb{E}_{\mathrm{rnd}}\Big[\widehat{\Pi}_{\psi,\gamma}^{(1)}\big(\boldsymbol{M}_k^{\mathrm{RR}}(x)\big)\Big] \geq \left(1 - \frac{1}{e}\right) \widehat{Q}_k^{(1)}(x) + \frac{d_{k,k}}{e} \widehat{R}_k^{(1)}(x) - \gamma C_k(x), \tag{139}$$

$$\widehat{\mathcal{F}}_k^{(1)}(x) - \mathbb{E}_{\mathrm{rnd}}\Big[\widehat{\Pi}_{\psi,\gamma}^{(1)}\big(\boldsymbol{M}_k^{\mathrm{RR}}(x)\big)\Big] \leq \frac{1}{e}\Big\{\widehat{Q}_k^{(1)}(x) - d_{k,k}\widehat{R}_k^{(1)}(x)\Big\} \leq \frac{c_{\psi,k}}{e} \widehat{Q}_k^{(1)}(x). \tag{140}$$

Since $\widehat{Q}_k^{(1)}(x) \leq 1$, a uniform rounding loss is

$$\varepsilon_{\mathrm{rnd},k} = \frac{c_{\psi,k}}{e}. \tag{141}$$

Fix (i,ω) and set $B_q = \widehat{A}_{i,G_q,q}^{(1)}(\omega)$ and $p_q = \sum_g x_{gq} \widehat{A}_{igq}^{(1)}(\omega)$. Conditional on ω, the B_q are independent under the rounding and $N = \sum_q B_q$ has mean $\widehat{s}_{i\omega}^{(1)}(x)$. For every integer $t \geq 1$,

$$\min\{r,t\} \geq t\{1 - (1 - 1/t)^r\}. \tag{142}$$

Hence

$$\begin{aligned} \mathbb{E}_{\mathrm{rnd}}[\min\{N,t\} \mid \omega] &\geq t\left[1 - \prod_{q=1}^{k}\left(1 - \frac{p_q}{t}\right)\right] \\ &\geq t\left(1 - e^{-\widehat{s}_{i\omega}^{(1)}(x)/t}\right) \\ &\geq \left(1 - \frac{1}{e}\right) \min\{\widehat{s}_{i\omega}^{(1)}(x), t\}. \end{aligned} \tag{143}$$

Using

$$\widetilde{\psi}_k(r) = d_{k,k} r + \sum_{t=1}^{k-1} (d_{t,k} - d_{t+1,k}) \min\{r, t\}, \tag{144}$$

and averaging over tasks and scenarios prove (139). Rounding also preserves execution cost in expectation. Subtraction gives the first inequality in (140). Finally, $\widetilde{\psi}_k(r) \le d_{1,k} r$ and $d_{k,k} = (1 - c_{\psi,k}) d_{1,k}$ give the curvature bound, while $0 \le \widetilde{\psi}_k(r) \le 1$ gives $\widehat{Q}_k^{(1)}(x) \le 1$. □

LEMMA 12 **(Stochastic cardinality grid under the plug-in iid law).** *Suppose* $\psi_k(r) = h(r/k)$ *for a common increasing concave function* $h : [0,1] \to [0,1]$ *with* $h(0) = 0$. *For every workflow class* $\mathcal{M}$ *and* $1 \le b \le k \le K_{\max}$,

$$\left\{ \widehat{\mathrm{OPT}}_b^{(1)}(\mathcal{M}) \right\}^+ \ge \frac{b}{k} \left\{ \widehat{\mathrm{OPT}}_k^{(1)}(\mathcal{M}) \right\}^+. \tag{145}$$

Consequently, if $\mathcal{K}$ *has coverage ratio* ϱ,

$$\max\left\{ 0, \max_{b \in \mathcal{K}} \widehat{\mathrm{OPT}}_b^{(1)}(\mathcal{M}) \right\} \ge \frac{1}{\varrho} \widehat{\mathrm{OPT}}_{\psi,\gamma}^{(1)}(\mathcal{M}), \tag{146}$$

where

$$\widehat{\mathrm{OPT}}_{\psi,\gamma}^{(1)}(\mathcal{M}) = \max_{\substack{\boldsymbol{m} \in \mathbb{Z}_+^{\mathcal{M}} \\ k(\boldsymbol{m}) \le K_{\max}}} \widehat{\Pi}_{\psi,\gamma}^{(1)}(\boldsymbol{m}).$$

F ix $\boldsymbol{m} \in \mathcal{C}_k(\mathcal{M})$, label its k execution copies, retain b of them uniformly without replacement, and let $\boldsymbol{M}^{(b)}$ be the retained count vector. Set $\alpha = b/k$. Conditional on a realized plug-in size-k execution, if task i has r_i correct copies and R_i of them are retained, then $R_i \sim \mathrm{Hypergeom}(k, r_i, b)$. For $r_i > 0$, set $p_i = r_i/k$ and $X_i = R_i/b$. Concavity and monotonicity give

$$h(x) \ge h(p_i) \min\{x/p_i, 1\}, \qquad x \in [0,1]. \tag{147}$$

Writing $Z_i = R_i/r_i$, we have $X_i/p_i = Z_i/\alpha$, $\mathbb{E}[Z_i] = \alpha$, and $\min\{z, \alpha\} \ge \alpha z$ on $[0,1]$. Hence $\mathbb{E}[h(R_i/b)] \ge \alpha h(r_i/k)$. The case $r_i = 0$ is immediate.

By the iid plug-in execution law, any retained set of labeled copies has the same distribution as a direct run of the retained execution-count vector, and $\mathbb{E}[C(\boldsymbol{M}^{(b)})] = \alpha C(\boldsymbol{m})$. Therefore

$$\mathbb{E}\left[\widehat{\Pi}_{\psi,\gamma}^{(1)}(\boldsymbol{M}^{(b)}) \right] \ge \alpha \widehat{\Pi}_{\psi,\gamma}^{(1)}(\boldsymbol{m}).$$

Some retained multiset attains at least this expected value. Applying the result to an optimal positive-valued size-k vector proves (145); choosing a grid anchor $b \le k \le \varrho b$ proves (146). □

LEMMA 13 **(Plug-in generated-pool certificate).** *On* $\mathcal{E}_{\mathrm{orc}}$, *define*

$$\varepsilon_{\mathrm{ell}} = \max_{k \in \mathcal{K}} \varepsilon_{\mathrm{ell},k}, \qquad \varepsilon_{\mathrm{rnd}} = \max_{k \in \mathcal{K}} \varepsilon_{\mathrm{rnd},k},$$

and let

$$\widehat{V}_T^{(1)} = \max_{\boldsymbol{m}\in\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)} \widehat{\Pi}_{\psi,\gamma}^{(1)}(\boldsymbol{m}).$$

Then

$$\widehat{V}_T^{(1)} \geq \frac{1}{\varrho}\widehat{\mathrm{OPT}}_{\psi,\gamma}^{(1)}(\mathcal{G}) - \varepsilon_{\mathrm{ell}} - \varepsilon_{\mathrm{rnd}}. \tag{148}$$

F or every $k\in\mathcal{K}$, the recovered x^k is supported on $\mathcal{M}_T$. By Lemmas 10 and 11,

$$\begin{aligned}\widehat{\mathrm{OPT}}_k^{(1)}(\mathcal{M}_T) &\geq \mathbb{E}_{\mathrm{rnd}}\left[\widehat{\Pi}_{\psi,\gamma}^{(1)}\left(\boldsymbol{M}_k^{\mathrm{RR}}(x^k)\right)\right]\\ &\geq \widehat{\mathcal{L}}_k^{(1)}(\mathcal{G}) - \varepsilon_{\mathrm{ell},k} - \varepsilon_{\mathrm{rnd},k}\\ &\geq \widehat{\mathrm{OPT}}_k^{(1)}(\mathcal{G}) - \varepsilon_{\mathrm{ell},k} - \varepsilon_{\mathrm{rnd},k}.\end{aligned} \tag{149}$$

By Lemma 12, some $k\in\mathcal{K}$ satisfies

$$\widehat{\mathrm{OPT}}_k^{(1)}(\mathcal{G}) \geq \frac{1}{\varrho}\widehat{\mathrm{OPT}}_{\psi,\gamma}^{(1)}(\mathcal{G}).$$

Combining the two displays proves (148). □

Lemma 14 **(Transfer of the generated-pool certificate to the true iid law)**. *On the intersection of the Stage-1 concentration event* (116) *and* $\mathcal{E}_{\mathrm{orc}}$,

$$\max_{\boldsymbol{m}\in\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)} \Pi_{\psi,\gamma}^{\mathrm{stoch}}(\boldsymbol{m}) \geq \frac{1}{\varrho}\mathrm{OPT}_{\psi,\gamma}^{\mathrm{stoch}}(\mathcal{G}) - \varepsilon_{\mathrm{ell}} - \varepsilon_{\mathrm{rnd}} - \left(1+\frac{1}{\varrho}\right)\varepsilon_{\mathrm{est}}(L_1,\delta_1). \tag{150}$$

In particular, the final term is at most $2\varepsilon_{\mathrm{est}}(L_1,\delta_1)$.

B y Lemma 7, the true and plug-in values of every feasible portfolio differ by at most $\varepsilon_{\mathrm{est}}(L_1,\delta_1)$. Therefore

$$\begin{aligned}\max_{\boldsymbol{m}\in\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)} \Pi_{\psi,\gamma}^{\mathrm{stoch}}(\boldsymbol{m}) &\geq \widehat{V}_T^{(1)} - \varepsilon_{\mathrm{est}}(L_1,\delta_1)\\ &\geq \frac{1}{\varrho}\widehat{\mathrm{OPT}}_{\psi,\gamma}^{(1)}(\mathcal{G}) - \varepsilon_{\mathrm{ell}} - \varepsilon_{\mathrm{rnd}} - \varepsilon_{\mathrm{est}}(L_1,\delta_1)\\ &\geq \frac{1}{\varrho}\mathrm{OPT}_{\psi,\gamma}^{\mathrm{stoch}}(\mathcal{G}) - \varepsilon_{\mathrm{ell}} - \varepsilon_{\mathrm{rnd}} - \left(1+\frac{1}{\varrho}\right)\varepsilon_{\mathrm{est}}(L_1,\delta_1),\end{aligned}$$

which proves the claim. □

Let $\mathcal{H}_T$ denote the sigma-field generated by the completed workflow generation stage, including all Stage-1 pilot outcomes and oracle randomness.

Lemma 15 **(Conditional fresh-L_2 deviation)**. *Let $\mathcal{A}_T$ be a finite, $\mathcal{H}_T$-measurable class of execution-count vectors. Conditional on $\mathcal{H}_T$, with probability at least $1-\delta_2$,*

$$\sup_{\boldsymbol{m}\in\mathcal{A}_T}\left|\widehat{\Pi}_{L_2}^{\mathrm{final}}(\boldsymbol{m}) - \Pi_{\psi,\gamma}^{\mathrm{stoch}}(\boldsymbol{m})\right| \leq \sqrt{\frac{\log(2|\mathcal{A}_T|/\delta_2)}{2nL_2}}. \tag{151}$$

Consequently, an exact empirical maximizer over $\mathcal{A}_T$ has true value at least the best true value in $\mathcal{A}_T$ minus twice the right-hand side.

F or fixed $\boldsymbol{m}$, the nL_2 random variables

$$\psi_{k(\boldsymbol{m})}\big(R_i^{(2,\ell)}(\boldsymbol{m})\big), \qquad i=1,\dots,n, \quad \ell=1,\dots,L_2,$$

are conditionally independent under Assumption 3, lie in $[0,1]$, and have average mean equal to the stochastic accuracy component of $\boldsymbol{m}$. Hoeffding's inequality and a union bound over $\mathcal{A}_T$ prove (151). Applying the uniform bound once to the empirical maximizer and once to a true maximizer gives the final statement. □

Proof of Proposition 3. Fix a nonzero $\boldsymbol{m}$ and write $k=k(\boldsymbol{m})$. For every task i, Assumption 1 and lemma 1 and concavity of h_Λ give

$$\mathbb{E}[\psi_k(R_i(\boldsymbol{m}))] \le \mathbb{E}\left[h_\Lambda\left(\frac{R_i(\boldsymbol{m})}{k}\right)\right] \le h_\Lambda\left(\frac{\mathbb{E}[R_i(\boldsymbol{m})]}{k}\right). \tag{152}$$

Using (59), averaging over tasks, and applying Jensen once more yield

$$\frac{1}{n}\sum_i \mathbb{E}[\psi_k(R_i(\boldsymbol{m}))] \le h_\Lambda\left(\frac{1}{k}\sum_g m_g \bar{a}_g\right),$$

which proves (61). For a size-k vector over $\mathcal{M}$, the argument of h_Λ is at most $a^\star$ and $C(\boldsymbol{m}) \ge kc_{\min}$, proving (62). Singleton feasibility gives the lower bound. Maximizing $h_\Lambda(a)-a$ over $a\in[0,1]$ proves (63). □

Proof of Theorem 5. Let $\mathcal{E}_{\text{est}}$ be the Stage-1 concentration event in (116), let $\mathcal{E}_{\text{orc}}$ be the event that every pricing batch is τ_k-accurate, and let $\mathcal{E}_{\text{samp}}$ be the Stage-2 uniform-deviation event in (151) for $\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)$. By Lemmas 6, 9 and 15,

$$\mathbb{P}\left(\mathcal{E}_{\text{est}} \cap \mathcal{E}_{\text{orc}} \cap \mathcal{E}_{\text{samp}}\right) \ge 1-\delta_1-\delta_2-N^{\text{stoch}}_{\text{price}}(\varepsilon_{\text{ell}})e^{-p_{\text{orc}}m}. \tag{153}$$

Work on this intersection and write

$$V_T^\star = \max_{\boldsymbol{m}\in\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)} \Pi^{\text{stoch}}_{\psi,\gamma}(\boldsymbol{m}).$$

By Lemma 14,

$$V_T^\star \ge \frac{1}{\varrho}\text{OPT}^{\text{stoch}}_{\psi,\gamma}(\mathcal{G}) - \varepsilon_{\text{ell}} - \varepsilon_{\text{rnd}} - 2\varepsilon_{\text{est}}(L_1,\delta_1). \tag{154}$$

Let $\varepsilon_s = \varepsilon_{\text{samp}}(L_2,\delta_2)$ and let $\boldsymbol{m}_T^\star$ maximize true value over $\mathcal{C}_{\mathcal{K}}(\mathcal{M}_T)$.

Suppose first that the conservative rule returns $\widetilde{\boldsymbol{m}}$. By the rule and uniform deviation,

$$\Pi^{\text{stoch}}_{\psi,\gamma}(\widetilde{\boldsymbol{m}}) \ge \widehat{\Pi}^{\text{final}}_{L_2}(\widetilde{\boldsymbol{m}}) - \varepsilon_s \ge \underline{V}.$$

Empirical optimality and uniform deviation also give

$$\begin{aligned}\Pi^{\text{stoch}}_{\psi,\gamma}(\widetilde{\boldsymbol{m}}) &\ge \widehat{\Pi}^{\text{final}}_{L_2}(\widetilde{\boldsymbol{m}}) - \varepsilon_s \\ &\ge \widehat{\Pi}^{\text{final}}_{L_2}(\boldsymbol{m}_T^\star) - \varepsilon_s \\ &\ge V_T^\star - 2\varepsilon_s.\end{aligned}$$

Hence this case yields

$$\Pi^{\mathrm{stoch}}_{\psi,\gamma}(\widehat{\boldsymbol{m}}) \geq \max\{\underline{V}, V_T^\star - 2\varepsilon_s\}.$$

Suppose instead that the rule returns the baseline $\boldsymbol{m}^0$. Then

$$\widehat{\Pi}^{\mathrm{final}}_{L_2}(\widetilde{\boldsymbol{m}}) - \varepsilon_s < \underline{V}.$$

Empirical optimality and uniform deviation imply

$$\widehat{\Pi}^{\mathrm{final}}_{L_2}(\widetilde{\boldsymbol{m}}) - \varepsilon_s \geq V_T^\star - 2\varepsilon_s,$$

so $V_T^\star - 2\varepsilon_s < \underline{V}$. Since the baseline has true value at least $\underline{V}$, this case gives the same bound. Combining the two cases with (154) proves (72).

Finally, for every $A, e \geq 0$ and $\underline{V} > 0$,

$$\max\{\underline{V}, A - e\} \geq \frac{\underline{V}}{\underline{V} + e} A.$$

Apply this inequality with

$$A = \frac{1}{\varrho}\mathrm{OPT}^{\mathrm{stoch}}_{\psi,\gamma}(\mathcal{G}), \qquad e = \varepsilon_{\mathrm{ell}} + \varepsilon_{\mathrm{rnd}} + 2\varepsilon_{\mathrm{est}}(L_1, \delta_1) + 2\varepsilon_{\mathrm{samp}}(L_2, \delta_2),$$

to obtain (73). □

Appendix G: Additional End-to-End Experiments

The main text reports the ABCD experiment in detail. This appendix applies the same stochastic workflow-evaluation, selector-calibration, portfolio- optimization, and dual-guided generation pipeline to the two other domains in the study. We organize each domain in the same order: task construction and data, stochastic performance of the initial workflow bank, selector calibration, finite-pool optimization, budgeted workflow generation, and fresh held-out deployment. The SGD and HotpotQA experiments are reported below.

G.1. Schema-Guided Dialogue

Task construction and data. Schema-Guided Dialogue (SGD) contains task-oriented conversations spanning multiple services, each accompanied by a natural-language schema that defines the service, its available intents, and its slots (Rastogi et al. 2020). We form a schema-conditioned active-intent classification task. At a user turn, the workflow observes the service description, the descriptions of all intents available for that service, required and optional slots, slot descriptions, and the dialogue prefix through the current user message. It must return exactly one intent from that service's allowed intent set.

To reduce nearly repeated observations within a dialogue, we retain the first user turn at which each distinct service–active-intent pair appears. We exclude frames whose active intent is `NONE`

and services with fewer than two available intents. We preserve the official train, development, and test split structure and sample 800 development tasks for workflow evaluation and portfolio construction, 1,004 development-split tasks for selector calibration, and 800 official test tasks for held-out deployment. The sampled development, calibration, and held-out sets contain 33, 24, and 30 observed active-intent labels, respectively; no single intent accounts for more than 6.13%, 9.27%, and 7.38% of the corresponding samples.

Initial workflow bank and stochastic execution. We use the same initial bank as in the ABCD experiment: 18 single-call workflows obtained by crossing Qwen2.5-3B-Instruct, Mistral-7B-Instruct-v0.3, and Granite-3.3-8B-Instruct with the direct, evidence-first, decomposition, verify-and-revise, alternatives, and limited-context prompting strategies. Every workflow–task pair is executed independently $L = 5$ times at temperature one. The best estimated standalone workflow on the development sample is Granite alternatives, with one-execution accuracy 82.175%. If every workflow in the initial bank is run once independently, expected oracle coverage is 97.467%, indicating that the bank retains meaningful complementarity despite its strong best singleton.

The execution parser succeeds on 95.207% of development executions, and the five draws produce 1.433 distinct intent predictions per task–workflow cell on average, confirming that repeated executions are not simply identical copies of a deterministic output. Workflow-level estimates are also stable at small L. Relative to the full $L = 5$ estimates, the workflow rankings based on the first one, two, and three draws have Spearman correlations 0.897, 0.989, and 0.994; the largest corresponding absolute change in a workflow's average accuracy is 1.975, 0.700, and 0.492 percentage points.

Selector calibration. We construct 100 feasible candidate panels for every $K \in \{2, \dots, 6\}$ and every interior correct count $r \in \{1, \dots, K-1\}$. The selector observes the service schema, dialogue prefix, and candidate intent labels but not workflow identity, cost, or correctness. The primary rotation-vote calibration contains 1,500 panels from 702 distinct tasks and has a 100% parse rate. Maximum-likelihood estimation of the Plackett–Luce recovery model gives $\widehat{\lambda} = 7.2145$ with a 95% confidence interval of $[5.7747, 9.0901]$, obtained by resampling tasks. This estimate corresponds to an 87.83% fitted probability of selecting the correct candidate when exactly one of two candidates is correct and an average 34.33 percentage-point advantage over uniform selection on the calibration design.

Initial-pool optimization. For each $K \in \{1, \dots, 6\}$, we apply the common 16-scenario SAA and 512-trial randomized-rounding procedure described in the main text and then evaluate each distinct integral portfolio using the corresponding exact plug-in calculation. Net development value increases throughout the searched range, and the selected portfolio has $K = 6$. It contains Granite alternatives, Granite direct, Granite limited context, Mistral evidence first, Qwen decomposition,

Table 5 SGD design, selector calibration, and workflow-generation summary.

Quantity	Setting or estimate
Development / calibration / held-out tasks	800/1,004/800
Observed active-intent labels	33/24/30; service-specific allowed intent sets
Initial workflow bank	18 workflows: 3 generator models × 6 prompting strategies
Stochastic workflow execution	$L = 5$ independent draws per task–workflow cell; temperature one
Run-size search	$K \in \{1, 2, 3, 4, 5, 6\}$
Final finite-pool optimization	16 SAA scenarios and 512 randomized-rounding trials per K
Selector	Qwen2.5-7B-Instruct; greedy decoding; cyclic-rotation vote
Initial selector calibration	1,500 balanced panels from 702 distinct tasks
Initial selector strength	$\widehat{\lambda} = 7.2145$, 95% CI $[5.7747, 9.0901]$
Implied pairwise recovery	$\widehat{\lambda}/(1+\widehat{\lambda}) = 87.83\%$
Average selector advantage over uniform choice	34.33 percentage points on the calibration design
Budgeted workflow generation	26 ADAS pricing queries; 20 distinct candidates evaluated
Generated workflows incorporated	0; final optimization bank remains at 18 workflow types
Second selector calibration	$\widehat{\lambda} = 8.1506$, 95% CI $[6.5437, 10.4949]$

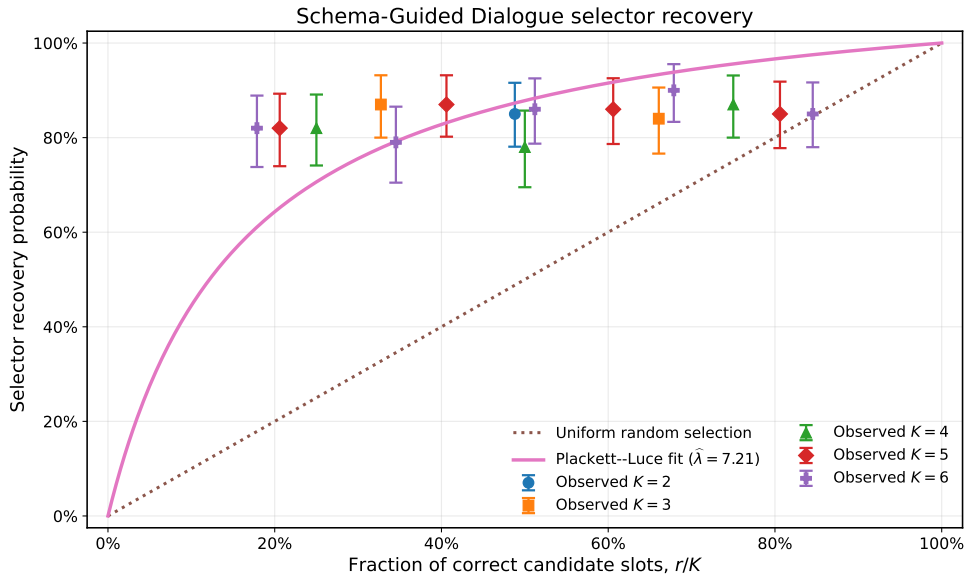


Figure 7 SGD selector recovery. Points report observed recovery on controlled (K, r) panels, vertical bars report 95% bootstrap intervals obtained by resampling tasks, the dotted line is uniform selection, and the solid curve is the fitted Plackett–Luce model. Small horizontal offsets separate observations having the same correct fraction.

and Qwen limited context, one execution each. Its calibrated development accuracy is 92.612%, and its workflow-cost-adjusted value is 0.920758. Thus, although repetition is allowed, the selected

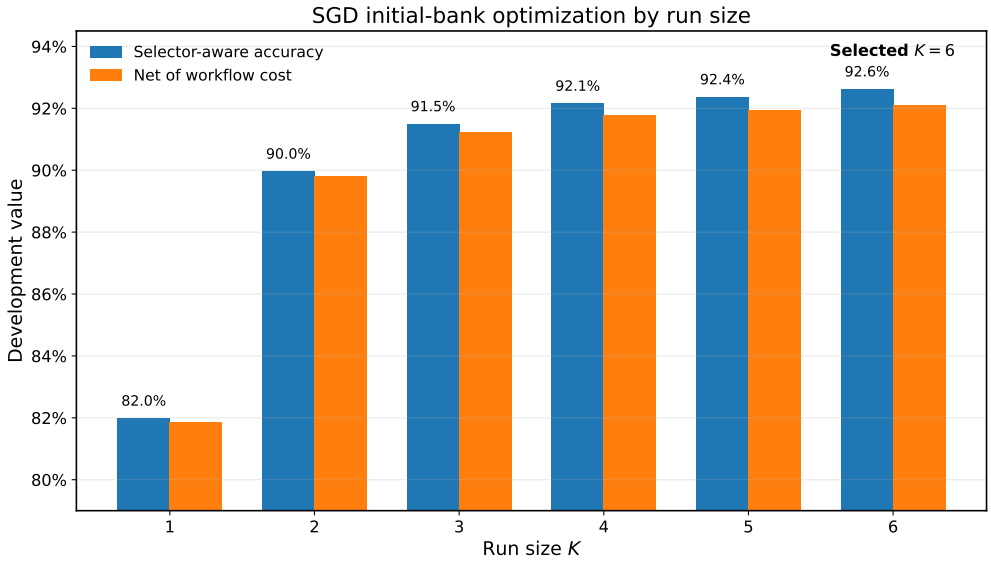


Figure 8 Development performance of the initial SGD workflow bank by run size. Bars report exact plug-in selector-aware accuracy and the same value after subtracting recurring workflow cost. The primary search selects $K = 6$.

SGD portfolio uses six distinct workflow types. The separately optimized no-repeat solution has essentially the same development value.

Budgeted dual-guided workflow generation. We apply the same practical stochastic ellipsoid-generation procedure as in the ABCD experiment. For each K, workflow generation uses one common-uniform SAA scenario, checks the explicit and previously materialized workflow constraints before invoking ADAS, and evaluates every distinct new workflow with five independent executions on each development task. The prespecified paper budget permits 20 distinct new workflow evaluations, allocated across the six run sizes. The SGD run makes 26 ADAS pricing queries and evaluates all 20 candidate workflows. None satisfies the stochastic pricing criterion required for incorporation at the queried dual points, so the optimization bank remains at its original 18 workflow types. Each fixed-K search reaches its candidate-evaluation budget; this result is therefore evidence from a budgeted search, not a claim that no useful workflow exists anywhere in the implicit class.

Because the workflow bank does not change, the pipeline's second calibration is a repeated calibration of the same 18-workflow candidate distribution rather than an expanded-bank calibration. It gives $\widehat{\lambda} = 8.1506$ with 95% confidence interval $[6.5437, 10.4949]$, overlapping the initial estimate. We use this second estimate for the final model-based predictions below; the held-out deployment results additionally run the selector directly and do not rely solely on the parametric recovery model.

Table 6 Fresh held-out SGD deployment on 800 tasks. PL accuracy is the plug-in Plackett–Luce prediction; actual accuracy uses the blind deterministic Qwen selector. Brackets report 95% Wilson intervals. Workflow and selector costs are per task, and actual total-system net subtracts both.

Method	K	PL accuracy	Actual accuracy (95% CI)	Random accuracy	Oracle coverage	Workflow cost	Selector cost	Actual total-system net
Best initial singleton	1	0.8470	0.8525 [0.8262, 0.8754]	0.8470	0.8470	0.001143	0.000000	0.851357
Optimized initial bank / final repeat portfolio	6	0.9471	0.9275 [0.9074, 0.9435]	0.8315	0.9790	0.005357	0.006514	0.915629
Final no-repeat benchmark	6	0.9454	0.9238 [0.9033, 0.9402]	0.8303	0.9772	0.005357	0.006514	0.911879

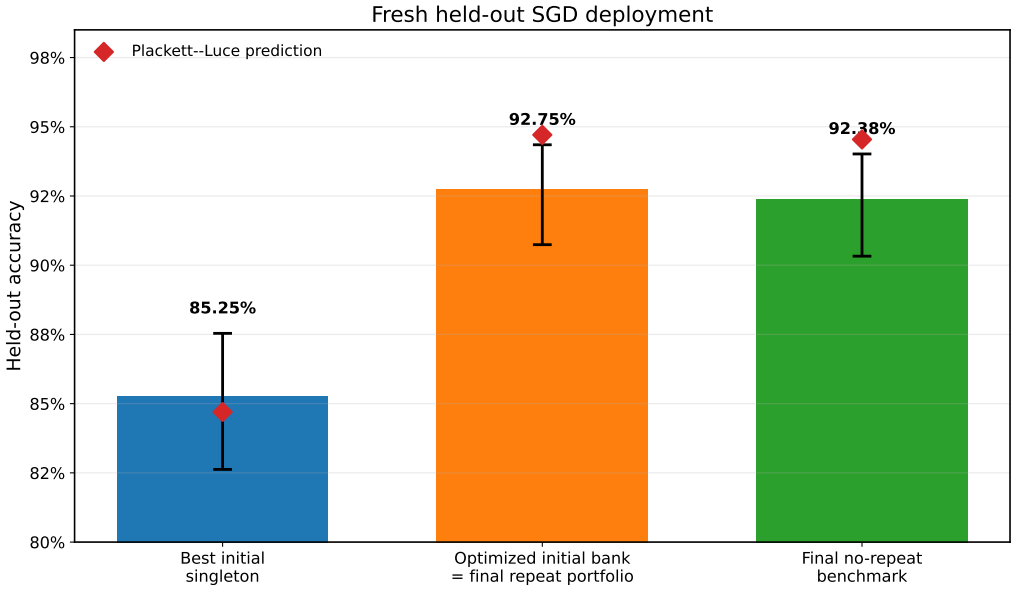


Figure 9 Fresh held-out SGD accuracy. Bars report actual deterministic selector accuracy, error bars show 95% Wilson intervals over 800 tasks, and diamonds report the corresponding plug-in Plackett–Luce predictions.

Fresh held-out deployment. We freeze the reported portfolios, collect five new stochastic executions per selected workflow on each of 800 official test tasks, and run the deterministic Qwen selector using cyclic candidate-order rotations and rotation vote. Table 6 and figure 9 report the results. Actual selector accuracy rises from 85.250% for the best initial singleton to 92.750% for the optimized initial-bank portfolio, an increase of 7.500 percentage points or 8.8% relative to the singleton. Since the budgeted ADAS search incorporates no new workflow, the final repeat-allowed portfolio is the same optimized initial-bank portfolio and has the same held-out accuracy. The no-repeat benchmark reaches 92.375%.

The calibrated model predicts the same ordering, with selector-aware accuracy 84.700% for the singleton, 94.708% for the optimized repeat-allowed portfolio, and 94.543% for the no-repeat benchmark. For the optimized portfolio, uniform selection among the realized candidate slots would attain 83.150%, while oracle coverage is 97.904%. After subtracting both workflow cost and realized selector cost, actual total-system net value rises from 0.851357 for the singleton to 0.915629 for the optimized portfolio.

Multiplicity. Within the selector-calibrated range $K \leq 6$, the repeat-allowed solution uses only distinct workflow types, and the repeat and no-repeat values are effectively identical. Repetition first appears at $K = 7$ in the extended model-based sensitivity analysis and produces only small gains. Thus, SGD supports the value of portfolio diversity and post-output selection but, unlike ABCD, does not provide meaningful evidence that multiplicity is valuable at the endogenous deployment solution.

G.2. HotpotQA

Task construction, data, and answer scoring. HotpotQA is an open-domain question-answering benchmark designed to require reasoning across multiple pieces of evidence (Yang et al. 2018). We use the distractor configuration. Each workflow receives the question together with all supplied Wikipedia passages and must return a concise answer span or short phrase. Supporting-fact annotations are retained only as audit metadata and are not shown to candidate workflows or to the selector.

The public distractor test labels are not distributed. We therefore sample 800 workflow-development questions from the official training split and form disjoint selector-calibration and held-out samples of 1,004 and 800 questions from the labeled distractor validation split. These samples contain 733, 913, and 741 distinct normalized answers, respectively. The largest answer share is 2.63% in the development sample, 3.98% in the calibration sample, and 2.63% in the held-out sample. Answers are evaluated using the normalized exact-match rule implemented in the notebooks: text is lowercased, articles and nonalphanumeric punctuation are removed, and whitespace is collapsed before comparison with the reference answer.

Initial workflow bank and stochastic execution. We use the same 18-workflow initial bank as in the other domains, obtained by crossing Qwen2.5-3B-Instruct, Mistral-7B-Instruct-v0.3, and Granite-3.3-8B-Instruct with the direct, evidence-first, decomposition, verify-and-revise, alternatives, and limited-context prompting strategies. Every workflow–question pair is executed independently $L = 5$ times at temperature one.

The best estimated standalone workflow on the development sample is Granite evidence first, with one-execution accuracy 44.950%. If every workflow in the initial bank is executed once independently, expected oracle coverage is 77.578%, showing substantial complementarity among the candidate generators. The answer parser succeeds on 88.468% of executions, and the five stochastic draws produce 2.908 distinct normalized answers per workflow–question cell on average, confirming substantial within-cell stochastic variation. Workflow-level estimates are stable despite noisy task-level probabilities. Relative to the full $L = 5$ ranking, rankings based on the first one, two, and three executions have Spearman correlations 0.939, 0.993, and 0.990; all five of the highest-ranked workflows remain in the top five. The largest absolute change in a workflow's average accuracy is 3.000, 0.875, and 0.633 percentage points, respectively.

Table 7 HotpotQA design, selector calibration, and workflow-generation summary.

Quantity	Setting or estimate
Development / calibration / held-out questions	800/1,004/800
Data split	Development from official train; disjoint calibration and held-out subsets from labeled distractor validation
Observed normalized answers	733/913/741
Initial workflow bank	18 workflows: 3 generator models × 6 prompting strategies
Stochastic workflow execution	$L = 5$ independent draws per workflow–question cell; temperature one
Run-size search	$K \in \{1, 2, 3, 4, 5, 6\}$
Final finite-pool optimization	16 SAA scenarios and 512 randomized-rounding trials per K
Selector	Qwen2.5-7B-Instruct; greedy decoding; cyclic-rotation vote
Initial selector calibration	1,500 balanced panels from 633 distinct questions
Initial selector strength	$\widehat{\lambda} = 1.7062$, 95% CI $[1.4557, 2.0068]$
Implied pairwise recovery	$\widehat{\lambda}/(1+\widehat{\lambda}) = 63.05\%$
Average selector advantage over uniform choice	10.87 percentage points on the calibration design
Budgeted workflow generation	26 ADAS pricing queries; 20 distinct candidates evaluated
Generated workflows incorporated	1; final optimization bank contains 19 workflow types
Expanded-bank selector strength	$\widehat{\lambda} = 1.7529$, 95% CI $[1.5028, 2.0499]$

Selector calibration. For each $K \in \{2, \ldots, 6\}$ and each interior correct count $r \in \{1, \ldots, K-1\}$, we construct 100 feasible panels containing exactly r correct candidate slots and $K - r$ incorrect slots. The selector observes the question, all supplied passages, and the candidate answers, but not workflow identity, cost, or correctness. The primary rotation-vote calibration contains 1,500 panels from 633 distinct questions and has a 100% base-panel parse rate.

Maximum-likelihood estimation of the Plackett–Luce recovery model gives $\widehat{\lambda} = 1.7062$ with a 95% confidence interval of $[1.4557, 2.0068]$, obtained by resampling tasks. The fitted selector therefore chooses the correct answer with probability 63.05% when exactly one of two candidates is correct. Averaged over the calibration design, selector recovery exceeds uniform selection by 10.87 percentage points. Selector recovery is positive but substantially weaker than in the two service-operation domains, consistent with the greater difficulty of recognizing a correct free-form multi-hop answer among plausible alternatives.

Initial-pool optimization. For each $K \in \{1, \ldots, 6\}$, we apply the common 16-scenario SAA and 512-trial randomized-rounding procedure described in the main text and then evaluate each distinct integral portfolio using the corresponding exact plug-in calculation. The best initial-bank solution has $K = 6$ and assigns five execution slots to Granite evidence first and one slot to Granite decomposition. Its calibrated development accuracy is 48.270%, compared with 44.950% for the best singleton, and its workflow-cost-adjusted value is 0.475840.

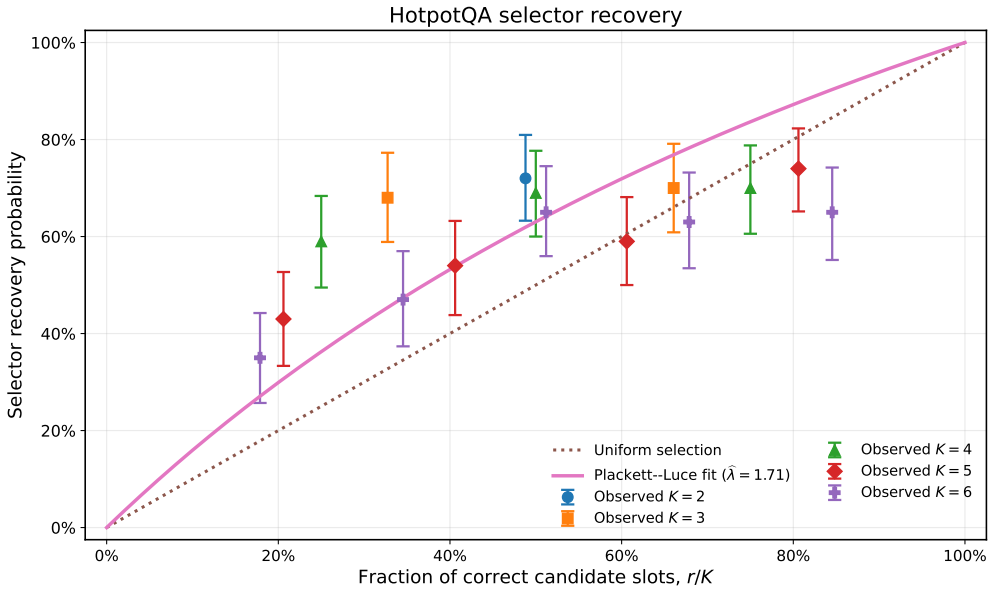


Figure 10 HotpotQA selector recovery. Points report observed recovery on controlled (K, r) panels, vertical bars report 95% bootstrap intervals obtained by resampling tasks, the dotted line is uniform selection, and the solid curve is the fitted Plackett–Luce model. Small horizontal offsets separate observations having the same correct fraction.

Multiplicity is already valuable in the initial bank. When repeated workflow types are prohibited, the best no-repeat solution is attained at $K = 3$ by Granite alternatives, Granite decomposition, and Granite evidence first and has workflow-cost-adjusted value 0.466358. Thus, allowing repetition increases the initial-bank endogenous optimum by 0.009482 in accuracy-equivalent net-value units.

Budgeted dual-guided workflow generation. We apply the same practical stochastic ellipsoid-generation procedure used in the ABCD and SGD experiments. For each K, generation uses one common-uniform SAA scenario, checks explicit and already materialized workflow constraints before invoking ADAS, and evaluates every distinct new workflow using five independent executions on each development question. The prespecified paper budget permits 20 distinct workflow evaluations across the six run sizes.

The search makes 26 ADAS pricing queries, evaluates all 20 distinct candidates, and incorporates one workflow during the $K = 2$ search, expanding the final optimization bank from 18 to 19 workflow types. We label this workflow G1; its frozen ADAS name is `FinalizedMultiHopQASolver`. Each fixed-K search reaches its prespecified candidate-evaluation budget, so the result should be interpreted as a budgeted ellipsoid search rather than as a claim that the complete implicit workflow class has been exhausted.

After adding G1, we recalibrate the deterministic Qwen selector on the expanded candidate distribution. The estimate increases modestly to $\widehat{\lambda} = 1.7529$ with 95% confidence interval $[1.5028, 2.0499]$.

Table 8 Executable structure of the ADAS-generated HotpotQA workflow incorporated into the final bank. All three model calls use GPT-4o-mini through the ADAS `LLMAgentBase` interface.

Workflow	Executable structure	Mean calls	Cost
G1	A first GPT-4o-mini call extracts evidence from the supplied passages. A second call verifies the relevance and accuracy of that evidence. A third call receives the question, passages, and verified evidence and derives the final answer. If either evidence stage returns no usable output, the workflow returns a fixed insufficient-evidence response.	3.0	0.003

Notes. G1 is the frozen ADAS program `adas_ell_5860177799ff` / `FinalizedMultiHopQASolver`. Cost is recurring normalized execution cost per complete workflow run.

Table 9 Key HotpotQA development portfolios. Accuracy is the exact plug-in Plackett–Luce value of the reported integral portfolio. Net value subtracts recurring workflow cost but not selector cost, matching the primary optimization objective.

Method	K	Portfolio	Accuracy	Net value
Best initial singleton	1	Granite evidence first	0.449500	0.448357
Optimized initial bank	6	Granite decomposition + 5× Granite evidence first	0.482697	0.475840
Generated workflow, no repeat	1	G1	0.661000	0.658000
Final repeat-allowed portfolio	2	2×G1	0.668631	0.662631

Expanded-bank reoptimization and multiplicity. G1 changes the portfolio substantially. As a standalone workflow, it has estimated development accuracy 66.100%, compared with 44.950% for the best initial workflow. Under the primary objective, which subtracts recurring workflow cost but not selector cost, the final repeat-allowed optimum has $K = 2$ and assigns both execution slots to G1. Its calibrated development accuracy is 66.863%, and its workflow-cost-adjusted value is 0.662631. The endogenous no-repeat optimum is one execution of G1, with calibrated accuracy 66.100% and workflow-cost-adjusted value 0.658000.

At fixed $K = 2$, multiplicity is valuable under the primary workflow-cost objective. When repetition is prohibited, the best $K = 2$ portfolio combines G1 with Granite evidence first and has value 0.596823. Allowing two independent executions of G1 raises that fixed-size value by 0.065808. After optimizing run size separately under each formulation, the repeat-allowed optimum remains 0.004631 above the no-repeat optimum.

Fresh held-out deployment. We freeze the reported portfolios, collect five new stochastic executions per selected workflow on each of 800 held-out questions, and then deploy the Qwen2.5-7B-Instruct selector deterministically using cyclic candidate-order rotations and rotation vote. Table 10 and figure 12 report both the plug-in Plackett–Luce prediction and the realized selector performance.

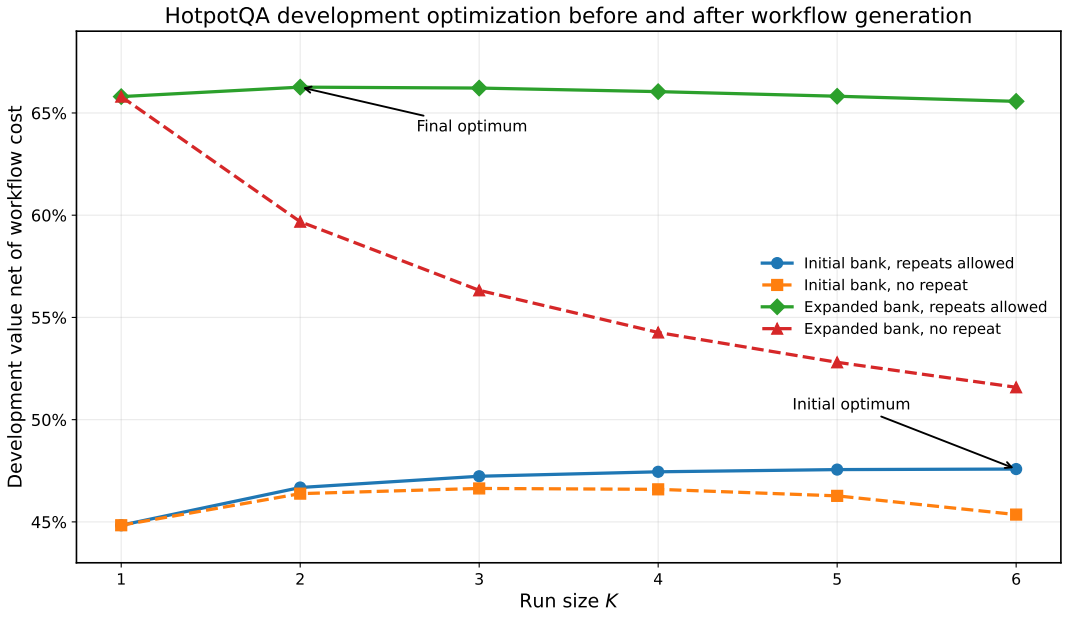


Figure 11 HotpotQA development value before and after workflow generation. Lines compare repeat-allowed and no-repeat optimization over the initial and expanded workflow banks. Workflow generation shifts the selected run size from $K = 6$ to $K = 2$ and makes repeated execution of G1 optimal under the primary workflow-cost objective.

Actual selector accuracy is 30.250% for the best initial singleton and 31.125% for the optimized initial-bank portfolio. After workflow generation, accuracy rises to 55.250% for the two-copy G1 portfolio and 54.875% for the single-G1 no-repeat benchmark. Relative to the best initial singleton, the full repeat-allowed procedure gains 25.000 percentage points, or 82.6%. Workflow generation contributes 24.125 percentage points relative to the optimized initial portfolio. The fitted recovery model is especially accurate for the final repeated portfolio, predicting 55.017% compared with the realized 55.250%.

The held-out comparison also clarifies the value and cost of multiplicity. A second independent G1 execution raises actual accuracy by 0.375 percentage points relative to one G1 execution. Under the primary objective, which charges workflow cost but not selector cost, the two-copy portfolio has slightly higher net value: 0.546500 versus 0.545750. Once realized selector cost is also subtracted, however, the one-copy benchmark has slightly higher total-system net value, 0.545750 versus 0.544329. Thus, HotpotQA provides evidence that multiplicity can improve accuracy while also illustrating that the incremental gain may not justify the additional post-output selection cost.

Table 10 Fresh held-out HotpotQA deployment on 800 questions. PL accuracy is the plug-in Plackett–Luce prediction; actual accuracy uses the blind deterministic Qwen selector. Brackets report 95% Wilson intervals. Workflow and selector costs are per question, and actual total-system net subtracts both.

Method	K	PL accuracy	Actual accuracy (95% CI)	Random accuracy	Oracle coverage	Workflow cost	Selector cost	Actual total-system net
Best initial singleton	1	0.2980	0.3025 [0.2717, 0.3352]	0.2980	0.2980	0.001143	0.000000	0.301357
Optimized initial bank	6	0.3331	0.3113 [0.2801, 0.3442]	0.2997	0.4642	0.006857	0.006514	0.297879
Final ADAS portfolio, repeats	2	0.5502	0.5525 [0.5179, 0.5866]	0.5430	0.5692	0.006000	0.002171	0.544329
Final ADAS portfolio, no repeat	1	0.5430	0.5488 [0.5141, 0.5829]	0.5430	0.5430	0.003000	0.000000	0.545750

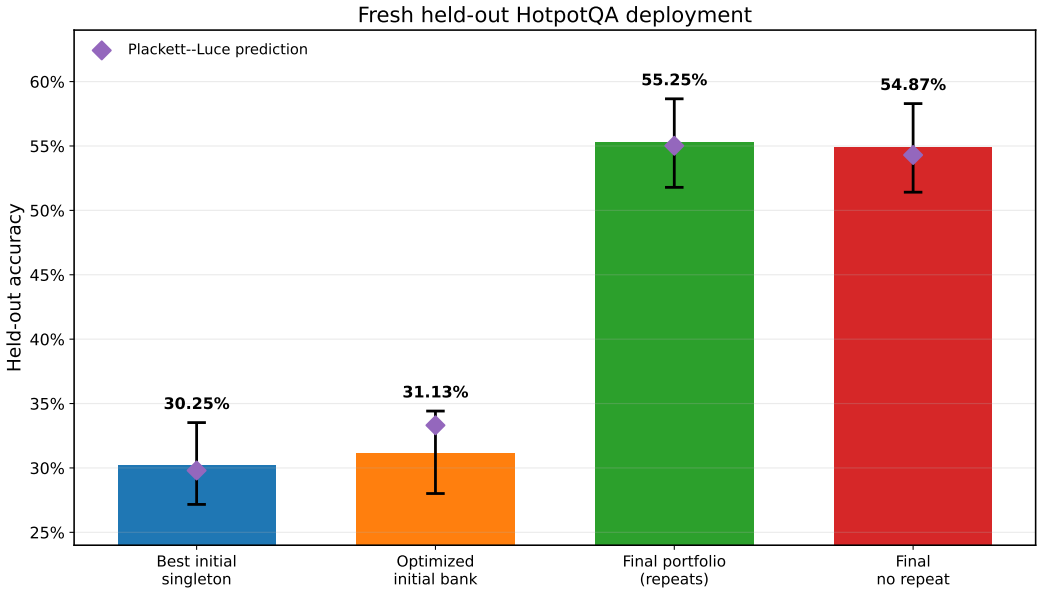


Figure 12 Fresh held-out HotpotQA accuracy. Bars report actual deterministic-selector accuracy, error bars show 95% Wilson intervals over 800 questions, and diamonds report the corresponding plug-in Plackett–Luce predictions.